Modul 1-6 Docker

Dosen pengampu: Dr Ferry Astika Saputra ST, M.Sc

Disusun oleh:

Faiz Ikhsan Ashary 3124500009

2 D3 Teknik Informatika A

PROGRAM STUDI D3 TEKNIK INFORMATIKA

POLITEKNIK ELEKTRONIKA NEGERI

SURABAYA

2025 2026

MODUL 1: Docker dan Instalasi

Langkah 0: Persiapan Environment

Pastikan VM/Host sudah terkoneksi internet untuk download image dari Docker Hub.

```
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

hisyam@HISYAMpc:~$ ping -c 3 google.com
PING google.com (142.250.9.113) 56(84) bytes of data.

--- google.com ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2004ms

hisyam@HISYAMpc:~$ sudo apt update && sudo apt upgrade -y
[sudo] password for hisyam:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1625 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [259 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.9 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [11.0 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1182 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [227 kB]
```

ping -c 3 google.com digunakan untuk memastikan koneksi internet aktif karena Docker perlu download image dari Docker Hub

Langkah 1: Instalasi Docker Engine di Ubuntu 22.04

1.1 Hapus versi lama (jika ada)

```
hisyam@HISYAMpc:~$ sudo apt remove -y docker docker-engine docker.io containerd runc 2>/dev/null
[sudo] password for hisyam:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Package 'docker' is not installed, so not removed
```

1.2 Instal dependensi

```
hisyam@HISYAMpc:~$ sudo apt install -y \
    ca-certificates \
    curl \
    gnupg \
    lsb-release
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
ca-certificates is already the newest version (20240203).
ca-certificates set to manually installed.
curl is already the newest version (8.5.0-2ubuntu10.8).
curl set to manually installed.
gnupg is already the newest version (2.4.4-2ubuntu17.4).
gnupg set to manually installed.
lsb-release is already the newest version (12.0-2).
lsb-release set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

Menginstall dependensi yang dibutuhkan untuk proses instalasi Docker, yaitu ca-certificates untuk verifikasi SSL, curl untuk download file, gnupg untuk verifikasi GPG key, dan lsb-release untuk mendeteksi versi Ubuntu.

1.3 Tambahkan Docker GPG key dan repository

Buat direktori keyrings

```
hisyam@HISYAMpc:~$ # Buat direktori keyrings
sudo install -m 0755 -d /etc/apt/keyrings
hisyam@HISYAMpc:~$ sudo install -m 0755 -d /etc/apt/keyrings
```

Download GPG key Docker

```
hisyam@HISYAMpc:~$ curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
    sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
hisyam@HISYAMpc:~$ sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

Tambahkan repository Docker

```
hisyam@HISYAMpc:~$ echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Menambahkan GPG key resmi Docker agar apt bisa memverifikasi keaslian package Docker, lalu mendaftarkan repository resmi Docker ke sistem.

1.4 Instal Docker Engine

```
hisyam@HISYAMpc:~$ sudo apt update
[sudo] password for hisyam:
Get:1 https://download.docker.com/linux/ubuntu noble InRelease [48.5 kB]
Get:2 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [52.9 kB]
```

```
hisyam@HISYAMpc:~$ sudo apt install -y \
    docker-ce \
    docker-ce-cli \
    containerd.io \
    docker-buildx-plugin \
    docker-compose-plugin
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  docker-ce-rootless-extras iptables libip4tc2 libip6tc2 libnetfilter-conntrack3 libnfnetlink0 libnftables1 libnftnl11
  libslirp0 nftables pigz slirp4netns
Suggested packages:
  cgroupfs-mount | cgroup-lite docker-model-plugin firewalld
The following NEW packages will be installed:
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin iptables
  libip4tc2 libip6tc2 libnetfilter-conntrack3 libnfnetlink0 libnftables1 libnftnl11 libslirp0 nftables pigz
  slirp4netns
```

Menginstall komponen Docker Engine yaitu docker-ce (Docker Community Edition), docker-ce-cli (CLI tool), containerd.io (container runtime), docker-buildx-plugin (untuk build multi-platform), dan docker-compose-plugin (untuk menjalankan multi-container).

1.5 Konfigurasi user non-root

Tambahkan user saat ini ke group docker

```
hisyam@HISYAMpc:~$ sudo usermod -aG docker $USER
```

Aktifkan group baru (atau logout/login)

```
hisyam@HISYAMpc:~$ newgrp docker
```

Menambahkan user ke group docker agar bisa menjalankan perintah docker tanpa sudo setiap saat.

1.6 Verifikasi instalasi

Cek versi Docker

```
hisyam@HISYAMpc:~$ docker version
Client: Docker Engine - Community
 Version:           29.4.2
 API version:       1.54
 Go version:        go1.26.2
 Git commit:        055a478
 Built:             Fri May  1 10:24:01 2026
 OS/Arch:           linux/amd64
 Context:           default

Server: Docker Engine - Community
 Engine:
  Version:          29.4.2
  API version:      1.54 (minimum version 1.40)
  Go version:       go1.26.2
  Git commit:       d329809
  Built:            Fri May  1 10:24:01 2026
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          v2.2.3
  GitCommit:        77c84241c7cbdd9b4eca2591793e3d4f4317c590
 runc:
  Version:          1.3.5
  GitCommit:        v1.3.5-0-g488fc13e
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
```

Cek info Docker Engine

```
hisyam@HISYAMpc:~$ docker info
Client: Docker Engine - Community
 Version:    29.4.2
 Context:    default
 Debug Mode: false
 Plugins:
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.33.0
    Path:     /usr/libexec/docker/cli-plugins/docker-buildx
  compose: Docker Compose (Docker Inc.)
    Version:  v5.1.3
    Path:     /usr/libexec/docker/cli-plugins/docker-compose
```

Pastikan service berjalan

```
hisyam@HISYAMpc:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
     Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-05-04 13:52:04 WIB; 4min 7s ago
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 5817 (dockerd)
      Tasks: 14
     Memory: 36.9M (peak: 40.8M)
        CPU: 540ms
     CGroup: /system.slice/docker.service
             └─5817 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
```

Test container pertama

```
hisyam@HISYAMpc:~$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Verifikasi bahwa Docker sudah terinstall dengan benar dan container hello-world berhasil dijalankan.

Langkah 2: Instalasi Docker Desktop di Windows 10/11

Langkah ini hanya sebagai referensi, tidak wajib dikerjakan. Docker Desktop menggunakan WSL2 sebagai backend untuk menjalankan kernel Linux di Windows.

Buka PowerShell sebagai Administrator:

Aktifkan WSL

```
PS C:\WINDOWS\system32> wsl --install
A distribution with the supplied name already exists. Use --name to chose a different name.
Error code: Wsl/InstallDistro/ERROR_ALREADY_EXISTS
PS C:\WINDOWS\system32>
```

Set WSL2 sebagai default

```
PS C:\WINDOWS\system32> wsl --set-default-version 2
For information on key differences with WSL 2 please visit https://aka.ms/wsl2
The operation completed successfully.
```

Verifikasi

```
PS C:\WINDOWS\system32> wsl --list --verbose
  NAME                   STATE           VERSION
* Ubuntu                 Running         2
  docker-desktop         Running         2
```

Restart komputer jika diminta.

2.2 Download dan Instal Docker Desktop

Buka https://www.docker.com/products/docker-desktop/

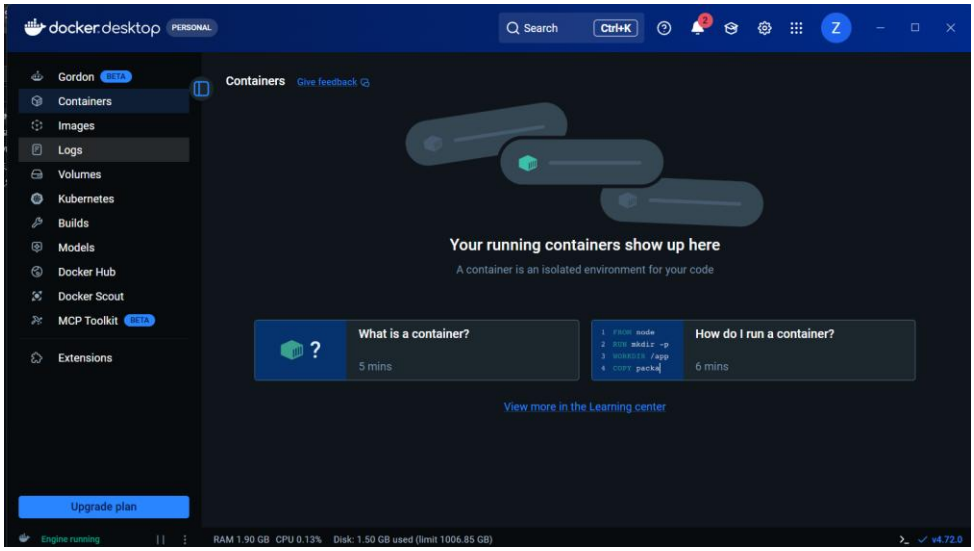
Download Docker Desktop for Windows

Jalankan installer, centang "Use WSL 2 instead of Hyper-V"

Klik Install → tunggu selesai → Restart jika diminta

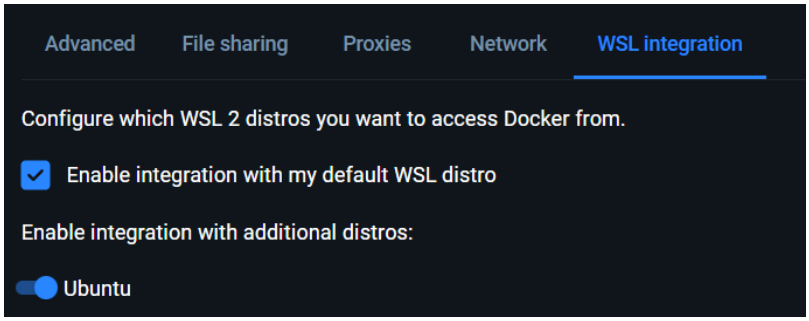
2.3 Konfigurasi Docker Desktop

Buka Docker Desktop dari Start Menu



Masuk ke Settings → General → pastikan "Use the WSL 2 based engine" aktif

Masuk ke Settings → Resources → WSL Integration → aktifkan distro yang diinginkan



2.4 Verifikasi dari PowerShell / WSL Terminal

Dari PowerShell

```
PS C:\Users\HISYAM> docker version
Client:
 Version:           29.4.2
 API version:       1.54
 Go version:        go1.26.2
 Git commit:        055a478
 Built:             Fri May  1 01:27:10 2026
 OS/Arch:           windows/amd64
 Context:           desktop-linux

Server: Docker Desktop 4.72.0 (225998)
 Engine:
  Version:          29.4.2
  API version:      1.54 (minimum version 1.40)
  Go version:       go1.26.2
  Git commit:       d329809
  Built:            Fri May  1 01:24:36 2026
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          v2.2.3
```

```
PS C:\Users\HISYAM> docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:f9078146db2e05e794366b1bfe584a14ea6317f44027d10ef7dad65279026885
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Dari WSL terminal (Ubuntu)

```
hisyam@HISYAMpc:~$ docker version
Client: Docker Engine - Community
 Version:           29.4.2
 API version:       1.54
 Go version:        go1.26.2
 Git commit:        055a478
 Built:             Fri May  1 10:24:01 2026
 OS/Arch:           linux/amd64
 Context:           default

Server: Docker Desktop 4.72.0 (225998)
 Engine:
  Version:          29.4.2
  API version:      1.54 (minimum version 1.40)
  Go version:       go1.26.2
  Git commit:       d329809
  Built:            Fri May  1 01:24:36 2026
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          v2.2.3
  GitCommit:        77c84241c7cbdd9b4eca2591793e3d4f4317c590
```

```
hisyam@HISYAMpc:~$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
```

NB : Langkah berikut hanya sebagai referensi selanjutnya tetap menggunakan wsl

3.1 Pull image dari Docker Hub

Pull image nginx versi latest

```
hisyam@HISYAMpc:~$ docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
85c66128325a: Pull complete
801a1ad15b4e: Pull complete
ff048f1f2159: Pull complete
677c63196868: Pull complete
4677c2a9a3d4: Pull complete
3531af2bc2a9: Pull complete
ce776bbcda0d: Pull complete
54c38c75806e: Download complete
69989ccd189b: Download complete
Digest: sha256:6e23479198b998e5e25921dff8455837c7636a67111a04a635cf1bb363d199dc
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest
```

Pull image nginx versi spesifik

```
hisyam@HISYAMpc:~$ docker pull nginx:1.26
1.26: Pulling from library/nginx
4fd410795c0f: Pull complete
5e98d206134b: Pull complete
d44088bb6ae8: Pull complete
9ebfb40fb06b: Pull complete
8a628cdd7ccc: Pull complete
6923759e66ab: Pull complete
7a0654aeb922: Pull complete
f17adcc09044: Download complete
5c2c3686e536: Download complete
Digest: sha256:41b194461e4bae16f9b25d68b0976ed4735b89ca625c89aad88e1c1c3b7e8860
Status: Downloaded newer image for nginx:1.26
docker.io/library/nginx:1.26
```

Pull image Ubuntu 22.04

```
hisyam@HISYAMpc:~$ docker pull ubuntu:22.04
22.04: Pulling from library/ubuntu
f63eb04151bc: Pull complete
bf0d6867143e: Download complete
Digest: sha256:962f6cadeae0ea6284001009daa4cc9a8c37e75d1f5191cf0eb83fe565b63dd7
Status: Downloaded newer image for ubuntu:22.04
docker.io/library/ubuntu:22.04
```

Pull image Alpine (sangat ringan, ~7MB)

```
hisyam@HISYAMpc:~$ docker pull alpine:3.20
3.20: Pulling from library/alpine
25f1d6b1951a: Pull complete
3a030ca3f633: Download complete
d4050d56ebf2: Download complete
Digest: sha256:d9e853e87e55526f6b2917df91a2115c36dd7c696a35be12163d44e6e2a4b6bc
Status: Downloaded newer image for alpine:3.20
docker.io/library/alpine:3.20
```

Pull image — docker pull nginx mendownload image nginx dari Docker Hub ke lokal. Tag spesifik seperti nginx:1.26 memastikan versi yang digunakan selalu sama.

3.2 Manajemen image

List semua image lokal

```
hisyam@HISYAMpc:~$ docker images

IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
alpine:3.20          d9e853e87e55       12.2MB         3.71MB
hello-world:latest   f9078146db2e       25.9kB         9.49kB   U
nginx:1.26           41b194461e4b        282MB         75.2MB
nginx:latest         6e23479198b9        240MB         65.8MB
ubuntu:22.04         962f6cadeae0        119MB         31.7MB
```

docker images - menampilkan semua image yang tersimpan di lokal beserta ukuran dan tag-nya

List dengan format custom

```
hisyam@HISYAMpc:~$ docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"
REPOSITORY    TAG       SIZE
nginx         latest    240MB
alpine        3.20      12.2MB
ubuntu        22.04     119MB
hello-world   latest    25.9kB
nginx         1.26      282MB
```

Inspect detail image

```
hisyam@HISYAMpc:~$ docker image inspect nginx
[
    {
        "Id": "sha256:6e23479198b998e5e25921dff8455837c7636a67111a04a635cf1bb363d199dc",
        "RepoTags": [
            "nginx:latest"
        ],
        "RepoDigests": [
            "nginx@sha256:6e23479198b998e5e25921dff8455837c7636a67111a04a635cf1bb363d199dc"
        ],
        "Comment": "buildkit.dockerfile.v0",
        "Created": "2026-04-22T01:22:07.750510155Z",
        "Config": {
            "ExposedPorts": {
                "80/tcp": {}
            },
            "Env": [
                "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
                "NGINX_VERSION=1.29.8",
                "NJS_VERSION=0.9.6",
                "NJS_RELEASE=1~trixie",
                "ACME_VERSION=0.3.1",
                "PKG_RELEASE=1~trixie",
                "DYNPKG_RELEASE=1~trixie"
            ]
```

docker image inspect nginx — menampilkan detail lengkap image seperti layer, environment variable, dan konfigurasi

Lihat history layer sebuah image

```
hisyam@HISYAMpc:~$ docker image history nginx
IMAGE          CREATED       CREATED BY                                      SIZE      COMMENT
6e23479198b9   2 weeks ago   CMD ["nginx" "-g" "daemon off;"]                0B        buildkit.dockerfile.v0
<missing>      2 weeks ago   STOPSIGNAL SIGQUIT                              0B        buildkit.dockerfile.v0
<missing>      2 weeks ago   EXPOSE map[80/tcp:{}]                           0B        buildkit.dockerfile.v0
<missing>      2 weeks ago   ENTRYPOINT ["/docker-entrypoint.sh"]            0B        buildkit.dockerfile.v0
<missing>      2 weeks ago   COPY 30-tune-worker-processes.sh /docker-ent…   16.4kB    buildkit.dockerfile.v0
<missing>      2 weeks ago   COPY 20-envsubst-on-templates.sh /docker-ent…   12.3kB    buildkit.dockerfile.v0
<missing>      2 weeks ago   COPY 15-local-resolvers.envsh /docker-entryp…   12.3kB    buildkit.dockerfile.v0
<missing>      2 weeks ago   COPY 10-listen-on-ipv6-by-default.sh /docker…   12.3kB    buildkit.dockerfile.v0
<missing>      2 weeks ago   COPY docker-entrypoint.sh / # buildkit          8.19kB    buildkit.dockerfile.v0
<missing>      2 weeks ago   RUN /bin/sh -c set -x     && groupadd --syst…   86.7MB    buildkit.dockerfile.v0
<missing>      2 weeks ago   ENV DYNPKG_RELEASE=1~trixie                     0B        buildkit.dockerfile.v0
```

docker image history nginx — menampilkan riwayat setiap layer yang membentuk image tersebut

Hapus image

```
hisyam@HISYAMpc:~$ docker rmi alpine:3.20
Untagged: alpine:3.20
Deleted: sha256:d9e853e87e55526f6b2917df91a2115c36dd7c696a35be12163d44e6e2a4b6bc
```

docker rmi alpine:3.20 — menghapus image dari lokal

Hapus semua image yang tidak digunakan

```
hisyam@HISYAMpc:~$ docker image prune -a
WARNING! This will remove all images without at least one container associated to t
Are you sure you want to continue? [y/N] y
Deleted Images:
untagged: nginx:latest
deleted: sha256:6e23479198b998e5e25921dff8455837c7636a67111a04a635cf1bb363d199dc
deleted: sha256:296499281873ebe2636681c6957776d29a31858d93c3e39b654b5aade87ed70a
deleted: sha256:422a5eaa9cd7814933c1fe4171d9675bf78ddff2afc18b4f175a86def04949d3
untagged: ubuntu:22.04
deleted: sha256:962f6cadeae0ea6284001009daa4cc9a8c37e75d1f5191cf0eb83fe565b63dd7
deleted: sha256:14be402d3f1eeeb5e7da73d3260322c68e7b51c88388f53e88eb21d6450bd520
deleted: sha256:268e076660f3cd225015df43cfe7198e5f787f24db00338d433299687237e538
untagged: nginx:1.26
deleted: sha256:41b194461e4bae16f9b25d68b0976ed4735b89ca625c89aad88e1c1c3b7e8860
deleted: sha256:c8042489f41ddaf05b7dfbe1e32d5227f94f1b002aca8e3b7b9c9d0cee6b0fed
deleted: sha256:c28381c5787d56e9bf825ecd4b8b1f1a64caec040b959a54674ac5a43954cfd3

Total reclaimed space: 172.7MB
```

docker image prune -a — membersihkan semua image yang tidak dipakai container manapun

Langkah 4: Menjalankan dan Mengelola Container

4.1 Menjalankan container dasar

Jalankan container nginx (foreground — tekan Ctrl+C untuk stop)

```
hisyam@HISYAMpc:~$ docker run nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
801a1ad15b4e: Pull complete
677c63196868: Pull complete
85c66128325a: Pull complete
3531af2bc2a9: Pull complete
ce776bbcda0d: Pull complete
ff048f1f2159: Pull complete
4677c2a9a3d4: Pull complete
54c38c75806e: Download complete
69989ccd189b: Download complete
Digest: sha256:6e23479198b998e5e25921dff8455837c7636a67111a04a635cf1bb363d199dc
Status: Downloaded newer image for nginx:latest
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/05/08 17:33:21 [notice] 1#1: using the "epoll" event method
```

docker run nginx — menjalankan container di foreground, terminal akan ter-block

Jalankan nginx di background (detached mode)

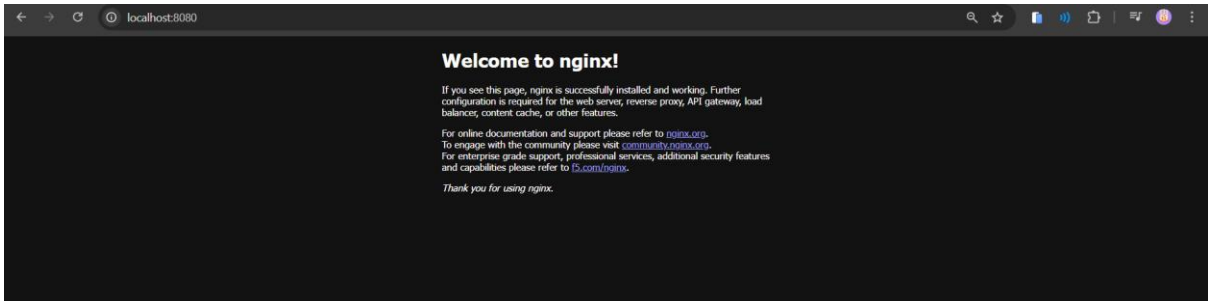
```
hisyam@HISYAMpc:~$ docker run -d --name web-server nginx
8d9debddfb1bf9d38dce62710a00b7a25a64b816d7b13d0458c5d58d54d6f0ec
```

docker run -d --name web-server nginx — flag -d menjalankan container di background (detached), --name memberi nama container

Jalankan nginx dengan port mapping (host:container)

```
hisyam@HISYAMpc:~$ docker run -d --name web-public -p 8080:80 nginx
4e32916b2d05496357cc44d3e974a23749141a077450dcc630f454c4e18635e4
```

Buka browser ke http://localhost:8080 → akan tampil halaman default Nginx.



docker run -d -p 8080:80 nginx — flag -p 8080:80 memetakan port 8080 di host ke port 80 di container, sehingga bisa diakses via browser

4.2 Container interaktif

Jalankan Ubuntu container dengan bash interaktif

```
hisyam@HISYAMpc:~$ docker run -it --name ubuntu-test ubuntu:22.04 /bin/bash
Unable to find image 'ubuntu:22.04' locally
22.04: Pulling from library/ubuntu
f63eb04151bc: Pull complete
bf0d6867143e: Download complete
Digest: sha256:962f6cadeae0ea6284001009daa4cc9a8c37e75d1f5191cf0eb83fe565b63dd7
Status: Downloaded newer image for ubuntu:22.04
```

docker run -it ubuntu:22.04 /bin/bash — flag -i (interactive) dan -t (pseudo-TTY) memungkinkan kita masuk ke dalam shell container dan menjalankan perintah seperti di Linux biasa

Di dalam container:

```
root@1e625beec857:/# cat /etc/os-release
PRETTY_NAME="Ubuntu 22.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
VERSION="22.04.5 LTS (Jammy Jellyfish)"
VERSION_CODENAME=jammy
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=jammy
```

```
root@1e625beec857:/# apt update && apt install -y curl
Hit:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:3 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
2 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
curl is already the newest version (7.81.0-1ubuntu1.24).
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
```

```
root@1e625beec857:/# apt update && apt install -y curl
Get:1 http://archive.ubuntu.com/ubuntu jammy InRelease [270 kB]
Get:2 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:3 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1294 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy/main amd64 Packages [1792 kB]
Get:7 http://archive.ubuntu.com/ubuntu jammy/restricted amd64 Packages [164 kB]
Get:8 http://archive.ubuntu.com/ubuntu jammy/multiverse amd64 Packages [266 kB]
Get:9 http://archive.ubuntu.com/ubuntu jammy/universe amd64 Packages [17.5 MB]
Get:10 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [3915 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [7251 kB]
```

```
curl: (6) Could not resolve host: web-server
root@dea49d1bff7f:/# curl http://172.17.0.2
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

akses container lain (jika di network sama)

```
root@dea49d1bff7f:/# exit
exit
```

keluar (container akan stop)

4.3 Monitoring container

List container yang sedang berjalan

```
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS
     NAMES
10fc88e80d6d   nginx     "/docker-entrypoint.…"   2 minutes ago    Up 2 minutes    80/tcp
     web-server
4e32916b2d05   nginx     "/docker-entrypoint.…"   13 minutes ago   Up 13 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/t
cp   web-public
```

docker ps — menampilkan container yang sedang berjalan

List SEMUA container (termasuk stopped)

```
hisyam@HISYAMpc:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS                      PORTS
                  NAMES
dea49d1bff7f   ubuntu:22.04   "/bin/bash"              2 minutes ago    Exited (0) 44 seconds ago
                  ubuntu-test
10fc88e80d6d   nginx          "/docker-entrypoint.…"   3 minutes ago    Up 3 minutes                80/tcp
                  web-server
738a0b4600c7   nginx          "/docker-entrypoint.…"   4 minutes ago    Exited (0) 4 minutes ago
                  confident_spence
4e32916b2d05   nginx          "/docker-entrypoint.…"   14 minutes ago   Up 14 minutes               0.0.0.0:8080->80/tcp
, [::]:8080->80/tcp   web-public
b134eb63bd52   nginx          "/docker-entrypoint.…"   15 minutes ago   Exited (0) 14 minutes ago
                  gifted_goodall
71811df6ff8a   hello-world    "/hello"                 23 minutes ago   Exited (0) 23 minutes ago
                  loving_fermi
fa71cc3e56ac   hello-world    "/hello"                 24 minutes ago   Exited (0) 24 minutes ago
                  affectionate_kilby
```

docker ps -a — menampilkan semua container termasuk yang sudah stopped

Lihat log container

docker logs web-server

```
hisyam@HISYAMpc:~$ docker logs web-server
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/05/08 17:45:29 [notice] 1#1: using the "epoll" event method
2026/05/08 17:45:29 [notice] 1#1: nginx/1.29.8
2026/05/08 17:45:29 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/05/08 17:45:29 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/05/08 17:45:29 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/05/08 17:45:29 [notice] 1#1: start worker processes
2026/05/08 17:45:29 [notice] 1#1: start worker process 29
2026/05/08 17:45:29 [notice] 1#1: start worker process 30
2026/05/08 17:45:29 [notice] 1#1: start worker process 31
```

docker logs web-server — menampilkan log output container

```
hisyam@HISYAMpc:~$ docker logs -f web-server    # follow (real-time)
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/05/08 17:45:29 [notice] 1#1: using the "epoll" event method
2026/05/08 17:45:29 [notice] 1#1: nginx/1.29.8
2026/05/08 17:45:29 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/05/08 17:45:29 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/05/08 17:45:29 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/05/08 17:45:29 [notice] 1#1: start worker processes
2026/05/08 17:45:29 [notice] 1#1: start worker process 29
2026/05/08 17:45:29 [notice] 1#1: start worker process 30
2026/05/08 17:45:29 [notice] 1#1: start worker process 31
2026/05/08 17:45:29 [notice] 1#1: start worker process 32
2026/05/08 17:45:29 [notice] 1#1: start worker process 33
```

docker logs -f web-server   # follow (real-time)

docker logs --tail 20 web-server # 20 baris terakhir

```
^Chisyam@HISYAMpc:~docker logs --tail 20 web-server  # 20 baris terakhir
2026/05/08 17:45:29 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/05/08 17:45:29 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/05/08 17:45:29 [notice] 1#1: start worker processes
2026/05/08 17:45:29 [notice] 1#1: start worker process 29
2026/05/08 17:45:29 [notice] 1#1: start worker process 30
2026/05/08 17:45:29 [notice] 1#1: start worker process 31
2026/05/08 17:45:29 [notice] 1#1: start worker process 32
2026/05/08 17:45:29 [notice] 1#1: start worker process 33
2026/05/08 17:45:29 [notice] 1#1: start worker process 34
2026/05/08 17:45:29 [notice] 1#1: start worker process 35
2026/05/08 17:45:29 [notice] 1#1: start worker process 36
2026/05/08 17:45:29 [notice] 1#1: start worker process 37
2026/05/08 17:45:29 [notice] 1#1: start worker process 38
2026/05/08 17:45:29 [notice] 1#1: start worker process 39
2026/05/08 17:45:29 [notice] 1#1: start worker process 40
2026/05/08 17:45:29 [notice] 1#1: start worker process 41
2026/05/08 17:45:29 [notice] 1#1: start worker process 42
2026/05/08 17:45:29 [notice] 1#1: start worker process 43
2026/05/08 17:45:29 [notice] 1#1: start worker process 44
172.17.0.4 - - [08/May/2026:17:47:16 +0000] "GET / HTTP/1.1" 200 896 "-" "curl/7.81.0" "-"
```

Lihat resource usage (CPU, Memory, I/O)

docker stats

```
CONTAINER ID   NAME         CPU %     MEM USAGE / LIMIT     MEM %     NET I/O           BLOCK I/O         PIDS
10fc88e80d6d   web-server   0.00%     13.26MiB / 7.664GiB   0.17%     1.67kB / 1.68kB   4.03MB / 12.3kB   17
4e32916b2d05   web-public   0.00%     13.33MiB / 7.664GiB   0.17%     4.17kB / 3.23kB   12.3kB / 12.3kB   17
```

menampilkan penggunaan CPU, memory, dan I/O secara real-time

Lihat detail container

docker inspect web-server

```
^Chisyam@HISYAMpc:~docker inspect web-server
[
    {
        "Id": "10fc88e80d6d2ba041bd16e1137826d848e7cdf0da733756a0c580a1f52d88bc",
        "Created": "2026-05-08T17:45:29.589363351Z",
        "Path": "/docker-entrypoint.sh",
        "Args": [
            "nginx",
            "-g",
            "daemon off;"
        ],
        "State": {
            "Status": "running",
            "Running": true,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 4552,
            "ExitCode": 0,
            "Error": "",
            "StartedAt": "2026-05-08T17:45:29.687416453Z",
            "FinishedAt": "0001-01-01T00:00:00Z"
        },
```

docker inspect — menampilkan detail lengkap container termasuk IP address dan network

4.4 Interaksi dengan container berjalan

Buat file index.html

```
hisyam@HISYAMpc:~$ echo "<h1>Test</h1>" > index.html
```

Eksekusi perintah di container yang sedang berjalan

docker exec web-server cat /etc/nginx/nginx.conf

```
hisyam@HISYAMpc:~$ docker exec web-server cat /etc/nginx/nginx.conf

user  nginx;
worker_processes  auto;

error_log  /var/log/nginx/error.log notice;
pid        /run/nginx.pid;


events {
    worker_connections  1024;
}


http {
    include       /etc/nginx/mime.types;
    default_type  application/octet-stream;

    log_format  main  '$remote_addr - $remote_user [$time_local] "$request" '
                      '$status $body_bytes_sent "$http_referer" '
```

docker exec web-server cat /etc/nginx/nginx.conf - menjalankan perintah di dalam container yang sedang berjalan tanpa masuk ke shell-nya

Masuk ke shell container yang sedang berjalan

docker exec -it web-server /bin/bash

```
hisyam@HISYAMpc:~$ docker exec -it web-server /bin/bash
root@10fc88e80d6d:/#
```

docker exec -it web-server /bin/bash — masuk ke shell container yang sedang berjalan

Copy file dari host ke container

docker cp index.html web-server:/usr/share/nginx/html/index.html

```
root@10fc88e80d6d:/# exit
exit
hisyam@HISYAMpc:~$ docker cp index.html web-server:/usr/share/nginx/html/index.html
Successfully copied 14B (transferred 2.05kB) to web-server:/usr/share/nginx/html/index.html
```

Copy file dari container ke host

docker cp web-server:/etc/nginx/nginx.conf ./nginx.conf

```
hisyam@HISYAMpc:~$ docker cp web-server:/etc/nginx/nginx.conf ./nginx.conf
Successfully copied 644B (transferred 2.56kB) to /home/hisyam/nginx.conf
```

menyalin file antara host dan container, berguna untuk mengubah konten tanpa rebuild image

4.5 Lifecycle management

Stop container

docker stop web-server

```
hisyam@HISYAMpc:~$ docker stop web-server
web-server
```

docker stop — mengirim sinyal SIGTERM ke container, memberi waktu proses untuk berhenti dengan baik

Start container yang sudah di-stop

docker start web-server

```
hisyam@HISYAMpc:~$ docker start web-server
web-server
```

docker start — menjalankan ulang container yang sudah stopped

Restart container

docker restart web-server

```
hisyam@HISYAMpc:~$ docker restart web-server
web-server
```

Kill container (force stop — SIGKILL)

docker kill web-server

```
hisyam@HISYAMpc:~$ docker kill web-server
web-server
```

- docker kill — mengirim SIGKILL langsung, container berhenti paksa

Hapus container (harus dalam keadaan stopped)

docker stop web-server && docker rm web-server

```
hisyam@HISYAMpc:~$ docker stop web-server && docker rm web-server
web-server
web-server
```

Hapus container secara paksa (meskipun masih running)

docker rm -f web-server

```
hisyam@HISYAMpc:~$ docker rm -f web-server
web-server
```

- docker rm -f — menghapus container secara paksa meski masih running

Hapus SEMUA stopped container

docker container prune

```
hisyam@HISYAMpc:~$ docker container prune
WARNING! This will remove all stopped containers.
Are you sure you want to continue? [y/N] y
Deleted Containers:
dea49d1bff7fb4c8ef958a43608038d048727d63150cadd4675826353bcce83d
738a0b4600c7a34386d4b4a082a8f4921f168999e6a0de23d3426a2e236542d8
b134eb63bd52938af439680d0a7237cca5a88ce28b9186802f276c3150785605
71811df6ff8a46ead327f6af4218077d2460b7b8d7dd34486e96899daa896705
fa71cc3e56ac3f242c7ab63cf43a75b1822719f057cfd3750581829597918a81
```

- docker container prune — menghapus semua container yang stopped sekaligus

Langkah 5: Membuat Custom Image dengan Dockerfile

5.1 Buat project directory

mkdir -p ~/docker-lab/custom-web && cd ~/docker-lab/custom-web

```
hisyam@HISYAMpc:~$ mkdir -p ~/docker-lab/custom-web && cd ~/docker-lab/custom-web
```

5.2 Buat halaman web

```
hisyam@HISYAMpc:~/docker-lab/custom-web$ cat > index.html << 'EOF'
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>Docker Lab - PENS</title>
    <style>
        body { font-family: Arial, sans-serif; text-align: center; padding: 50p
               background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
               color: white; }
        .container { background: rgba(255,255,255,0.1); border-radius: 15px;
                     padding: 40px; max-width: 600px; margin: 0 auto; }
        h1 { font-size: 2.5em; }
        .info { background: rgba(0,0,0,0.2); padding: 15px; border-radius: 8px;
                margin-top: 20px; text-align: left; }
    </style>
</head>
<body>
```

5.3 Buat Dockerfile

```
hisyam@HISYAMpc:~/docker-lab/custom-web$ cat > Dockerfile << 'EOF'
> FROM nginx:1.26-alpine
> # Metadata
LABEL maintainer="admin@pens.ac.id"
LABEL description="Custom Nginx untuk praktikum Docker PENS"
LABEL version="1.0"
> # Hapus halaman default dan ganti dengan halaman custom
RUN rm -rf /usr/share/nginx/html/*
COPY index.html /usr/share/nginx/html/index.html
> # Expose port 80
EXPOSE 80
> # Command default (inherited dari base image, tapi kita tulis eksplisit)
CMD ["nginx", "-g", "daemon off;"]
> EOF
```

5.1-5.3 Membuat project directory, file index.html sebagai halaman web custom, dan Dockerfile yang berisi instruksi untuk membangun image.

Penjelasan isi Dockerfile:

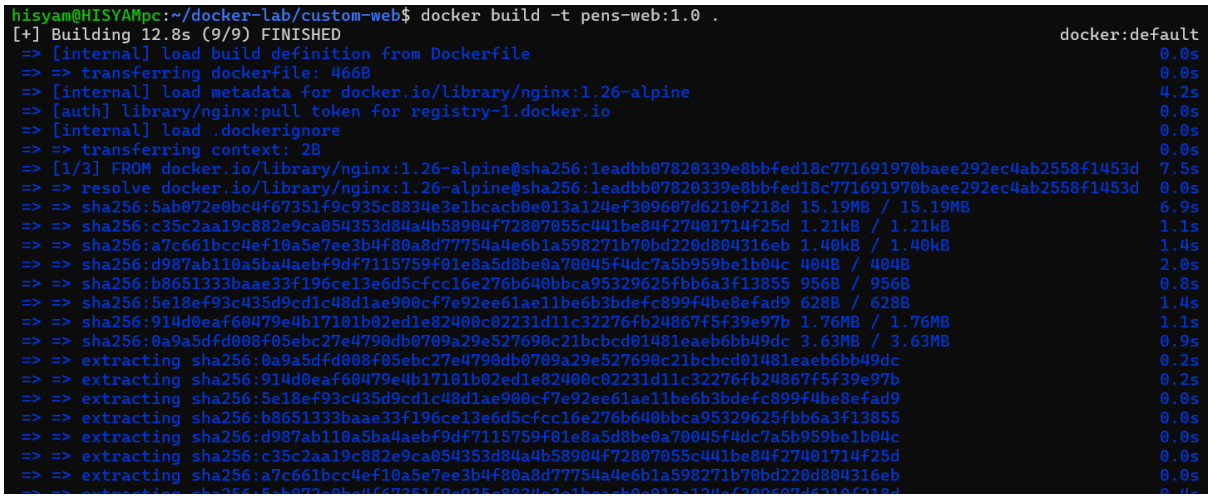
- FROM nginx:1.26-alpine — menggunakan base image nginx versi alpine yang ringan (~12MB)

- LABEL — metadata informasi image (tidak mempengaruhi fungsi)
- RUN rm -rf /usr/share/nginx/html/* — menghapus halaman default nginx
- COPY index.html ... — menyalin halaman custom ke dalam image
- EXPOSE 80 — dokumentasi bahwa container menggunakan port 80
- CMD — perintah yang dijalankan saat container start

5.4 Build dan jalankan

Build image (titik di akhir = build context = direktori saat ini)

docker build -t pens-web:1.0 .

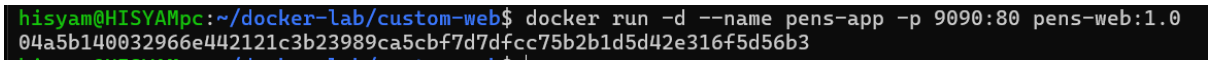


Verifikasi image berhasil dibuat

docker images | grep pens-web

Jalankan container dari image custom

docker run -d --name pens-app -p 9090:80 pens-web:1.0

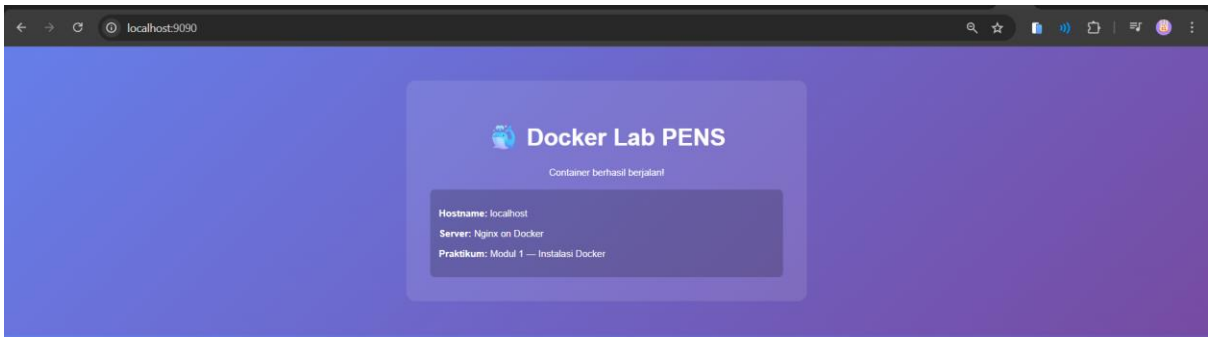


Test

curl http://localhost:9090

```
hisyam@HISYAMpc:~/docker-lab/custom-web$ curl http://localhost:9090
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>Docker Lab - PENS</title>
    <style>
        body { font-family: Arial, sans-serif; text-align: center; padding: 50px;
               background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
               color: white; }
        .container { background: rgba(255,255,255,0.1); border-radius: 15px;
                     padding: 40px; max-width: 600px; margin: 0 auto; }
        h1 { font-size: 2.5em; }
        .info { background: rgba(0,0,0,0.2); padding: 15px; border-radius: 8px;
                margin-top: 20px; text-align: left; }
    </style>
</head>
<body>
    <div class="container">
```

Buka browser ke http://localhost:9090.



docker build -t pens-web:1.0 . — membangun image dari Dockerfile. Titik . menunjukkan build context adalah direktori saat ini. Setiap instruksi di Dockerfile menghasilkan satu layer baru.

5.5 Lihat layer image

Bandingkan layer antara base image dan custom image

docker image history nginx:1.26-alpine

```
hisyam@HISYAMpc:~/docker-lab/custom-web$ docker image history nginx:1.26-alpine
IMAGE          CREATED        CREATED BY                                      SIZE      COMMENT
1eadbb078203   15 months ago   RUN /bin/sh -c set -x     && apkArch="$(cat …   37.7MB    buildkit.dockerfile.v0
<missing>      15 months ago   ENV NJS_RELEASE=1                               0B        buildkit.dockerfile.v0
<missing>      15 months ago   ENV NJS_VERSION=0.8.9                           0B        buildkit.dockerfile.v0
<missing>      15 months ago   CMD ["nginx" "-g" "daemon off;"]                0B        buildkit.dockerfile.v0
<missing>      15 months ago   STOPSIGNAL SIGQUIT                              0B        buildkit.dockerfile.v0
<missing>      15 months ago   EXPOSE map[80/tcp:{}]                           0B        buildkit.dockerfile.v0
<missing>      15 months ago   ENTRYPOINT ["/docker-entrypoint.sh"]            0B        buildkit.dockerfile.v0
<missing>      15 months ago   COPY 30-tune-worker-processes.sh /docker-ent…   16.4kB    buildkit.dockerfile.v0
<missing>      15 months ago   COPY 20-envsubst-on-templates.sh /docker-ent…   12.3kB    buildkit.dockerfile.v0
<missing>      15 months ago   COPY 15-local-resolvers.envsh /docker-entryp…   12.3kB    buildkit.dockerfile.v0
<missing>      15 months ago   COPY 10-listen-on-ipv6-by-default.sh /docker…   12.3kB    buildkit.dockerfile.v0
<missing>      15 months ago   COPY docker-entrypoint.sh / # buildkit          8.19kB    buildkit.dockerfile.v0
<missing>      15 months ago   RUN /bin/sh -c set -x     && addgroup -g 101…   5.28MB    buildkit.dockerfile.v0
<missing>      15 months ago   ENV PKG_RELEASE=2                               0B        buildkit.dockerfile.v0
```

docker image history pens-web:1.0

```
hisyam@HISYAMpc:~/docker-lab/custom-web$ docker image history pens-web:1.0
IMAGE          CREATED         CREATED BY                                      SIZE      COMMENT
bed6315bdc13   3 minutes ago   CMD ["nginx" "-g" "daemon off;"]                0B        buildkit.dockerfile.v0
<missing>      3 minutes ago   EXPOSE [80/tcp]                                 0B        buildkit.dockerfile.v0
<missing>      3 minutes ago   COPY index.html /usr/share/nginx/html/index.…   24.6kB    buildkit.dockerfile.v0
<missing>      3 minutes ago   RUN /bin/sh -c rm -rf /usr/share/nginx/html/…   20.5kB    buildkit.dockerfile.v0
<missing>      3 minutes ago   LABEL version=1.0                               0B        buildkit.dockerfile.v0
<missing>      3 minutes ago   LABEL description=Custom Nginx untuk praktik…   0B        buildkit.dockerfile.v0
<missing>      3 minutes ago   LABEL maintainer=admin@pens.ac.id               0B        buildkit.dockerfile.v0
<missing>      15 months ago   RUN /bin/sh -c set -x     && apkArch="$(cat …   37.7MB    buildkit.dockerfile.v0
<missing>      15 months ago   ENV NJS_RELEASE=1                               0B        buildkit.dockerfile.v0
<missing>      15 months ago   ENV NJS_VERSION=0.8.9                           0B        buildkit.dockerfile.v0
<missing>      15 months ago   CMD ["nginx" "-g" "daemon off;"]                0B        buildkit.dockerfile.v0
<missing>      15 months ago   STOPSIGNAL SIGQUIT                              0B        buildkit.dockerfile.v0
<missing>      15 months ago   EXPOSE map[80/tcp:{}]                           0B        buildkit.dockerfile.v0
<missing>      15 months ago   ENTRYPOINT ["/docker-entrypoint.sh"]            0B        buildkit.dockerfile.v0
<missing>      15 months ago   COPY 30-tune-worker-processes.sh /docker-ent…   16.4kB    buildkit.dockerfile.v0
<missing>      15 months ago   COPY 20-envsubst-on-templates.sh /docker-ent…   12.3kB    buildkit.dockerfile.v0
<missing>      15 months ago   COPY 15-local-resolvers.envsh /docker-entryp…   12.3kB    buildkit.dockerfile.v0
<missing>      15 months ago   COPY 10-listen-on-ipv6-by-default.sh /docker…   12.3kB    buildkit.dockerfile.v0
```

Membandingkan layer nginx:1.26-alpine dan pens-web:1.0 - layer dari base image di-share, hanya layer baru yang ditambahkan oleh Dockerfile yang berbeda.

Langkah 6: Docker System Cleanup

Lihat disk usage Docker

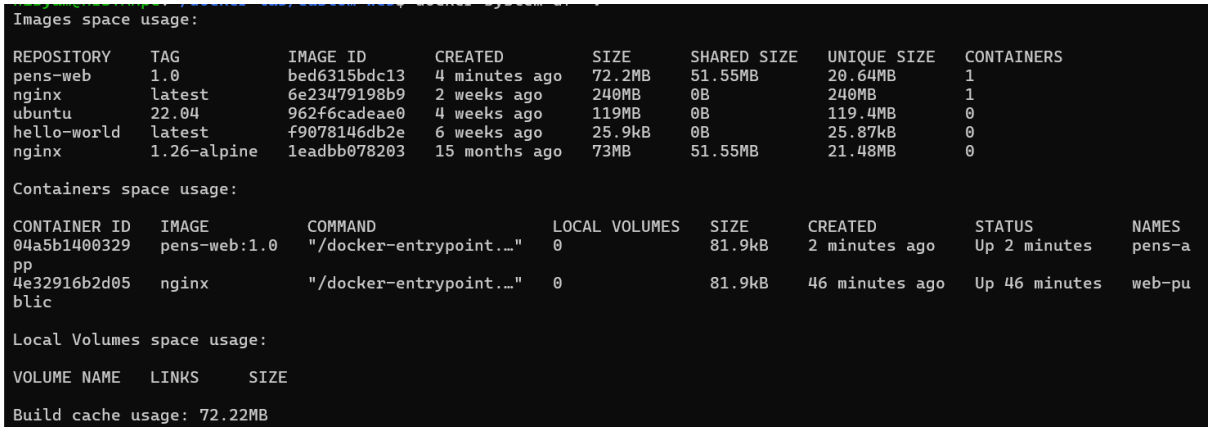
docker system df

```
hisyam@HISYAMpc:~/docker-lab/custom-web$ docker system df
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          5         2         432.9MB   140.9MB (32%)
Containers      2         2         163.8kB   0B (0%)
Local Volumes   0         0         0B        0B
Build Cache     13        0         72.22MB   20.48kB
```

docker system df — menampilkan total disk yang digunakan Docker (image, container, volume, cache)

Lihat detail disk usage

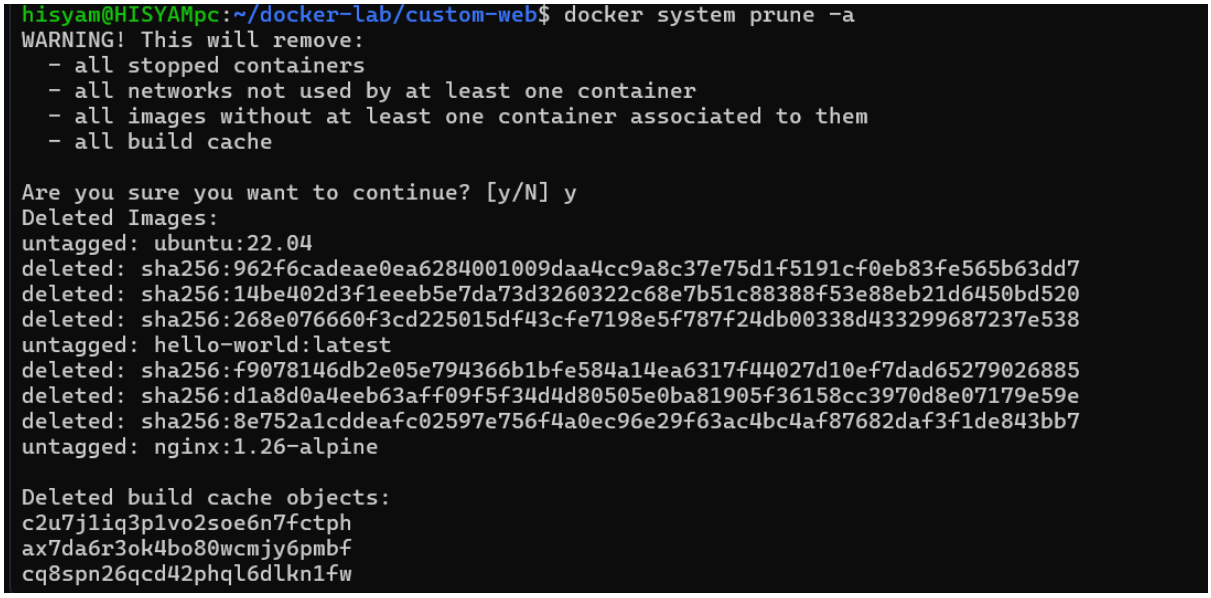
docker system df -v



```
Images space usage:

REPOSITORY    TAG           IMAGE ID       CREATED         SIZE      SHARED SIZE   UNIQUE SIZE   CONTAINERS
pens-web      1.0           bed6315bdc13   4 minutes ago   72.2MB    51.55MB       20.64MB       1
nginx         latest        6e23479198b9   2 weeks ago     240MB     0B            240MB         1
ubuntu        22.04         962f6cadeae0   4 weeks ago     119MB     0B            119.4MB       0
hello-world   latest        f9078146db2e   6 weeks ago     25.9kB    0B            25.87kB       0
nginx         1.26-alpine   1eadbb078203   15 months ago   73MB      51.55MB       21.48MB       0

Containers space usage:

CONTAINER ID   IMAGE          COMMAND                  LOCAL VOLUMES   SIZE      CREATED          STATUS          NAMES
04a5b1400329   pens-web:1.0   "/docker-entrypoint.…"   0               81.9kB    2 minutes ago    Up 2 minutes    pens-a
pp
4e32916b2d05   nginx          "/docker-entrypoint.…"   0               81.9kB    46 minutes ago   Up 46 minutes   web-pu
blic

Local Volumes space usage:

VOLUME NAME   LINKS     SIZE

Build cache usage: 72.22MB
```

Hapus semua resource yang tidak digunakan (container, image, network, cache)

docker system prune -a



```
hisyam@HISYAMpc:~/docker-lab/custom-web$ docker system prune -a
WARNING! This will remove:
  - all stopped containers
  - all networks not used by at least one container
  - all images without at least one container associated to them
  - all build cache

Are you sure you want to continue? [y/N] y
Deleted Images:
untagged: ubuntu:22.04
deleted: sha256:962f6cadeae0ea6284001009daa4cc9a8c37e75d1f5191cf0eb83fe565b63dd7
deleted: sha256:14be402d3f1eeeb5e7da73d3260322c68e7b51c88388f53e88eb21d6450bd520
deleted: sha256:268e076660f3cd225015df43cfe7198e5f787f24db00338d433299687237e538
untagged: hello-world:latest
deleted: sha256:f9078146db2e05e794366b1bfe584a14ea6317f44027d10ef7dad65279026885
deleted: sha256:d1a8d0a4eeb63aff09f5f34d4d80505e0ba81905f36158cc3970d8e07179e59e
deleted: sha256:8e752a1cddeafc02597e756f4a0ec96e29f63ac4bc4af87682daf3f1de843bb7
untagged: nginx:1.26-alpine

Deleted build cache objects:
c2u7j1iq3p1vo2soe6n7fctph
ax7da6r3ok4bo80wcmjy6pmbf
cq8spn26qcd42phql6dlkn1fw
```

docker system prune -a — membersihkan semua resource Docker yang tidak digunakan: stopped container, unused image, dangling cache. Berguna untuk membebaskan ruang disk setelah praktikum selesai.

Konfirmasi dengan -f (force, tanpa prompt)

docker system prune -a -f

```
hisyam@HISYAMpc:~/docker-lab/custom-web$ docker system prune -a -f
Total reclaimed space: 0B
```

5. PERTANYAAN

Pre-Lab (jawab sebelum praktikum)

1. Sebutkan minimal 3 perbedaan antara Virtual Machine dan Container.

Virtual Machine	**Container**
Memiliki guest OS sendiri	Berbagi kernel host OS
Lebih berat karena membutuhkan resource besar	Lebih ringan dan cepat
Booting lebih lama	Startup sangat cepat
Isolasi lebih kuat	Isolasi lebih ringan dibanding VM
Ukuran file biasanya besar	Ukuran lebih kecil

2. Apa fungsi dari containerd dan runc dalam arsitektur Docker?

   Containerd

   - Bertugas mengelola lifecycle container.
   - Mengatur image, storage, networking, dan proses container.
   - Menjadi perantara antara Docker Engine dan runtime rendah.

   Runc

   - Runtime tingkat rendah untuk menjalankan container.
   - Bertugas membuat dan menjalankan container sesuai standar OCI (Open Container Initiative).
   - Digunakan oleh containerd untuk mengeksekusi container.

3. Mengapa Docker membutuhkan kernel Linux? Bagaimana Docker Desktop di Windows mengatasi hal ini?

   Docker menggunakan fitur kernel Linux seperti:

   - namespaces
   - cgroups
   - union filesystem

   Fitur tersebut digunakan untuk isolasi dan manajemen resource container.

   Karena Windows tidak memiliki kernel Linux secara native, Docker Desktop di Windows menggunakan:

   - **WSL2 (Windows Subsystem for Linux 2)** atau
   - virtual machine ringan

   untuk menjalankan kernel Linux di dalam Windows sehingga container Docker tetap dapat berjalan.

4. Apa keuntungan layered filesystem pada Docker Image?

   - Menghemat storage karena layer yang sama dapat digunakan bersama.
   - Proses build lebih cepat dengan cache layer.
   - Mempermudah distribusi image karena hanya layer baru yang dikirim.
   - Memudahkan rollback atau update image.
   - Image menjadi lebih modular dan efisien.

5. Jelaskan perbedaan antara docker run dan docker exec.

   docker run

- Membuat dan menjalankan container baru dari image.

  docker exec

- Menjalankan perintah pada container yang sudah berjalan.
- Tidak membuat container baru.

Post-Lab (jawab setelah praktikum)

1. Bandingkan output docker image history nginx dengan docker image history pens-web:1.0. Layer mana saja yang di-share?

nginx (latest) dan pens-web:1.0 menggunakan **base image yang berbeda**:

- nginx latest → berbasis **Debian** (terlihat dari debian.sh dan ukuran 87.4MB di layer terbawah)
- pens-web:1.0 → berbasis **Alpine** nginx:1.26-alpine (terlihat dari alpine-minirootfs di layer terbawah, ukuran 8.47MB)

Karena base image berbeda, **tidak ada layer yang di-share** antara keduanya.

Layer tambahan milik pens-web:1.0 yang tidak ada di nginx latest:

- COPY index.html → halaman web custom
- RUN rm -rf /usr/share/nginx/html/ → hapus halaman default
- 3 LABEL → metadata maintainer, description, version

2. Apa yang terjadi pada data di dalam container setelah container dihapus dengan docker rm? Bagaimana solusinya?

Data di dalam container **hilang permanen** karena container layer bersifat writable tapi tidak persisten. Solusinya menggunakan **Docker Volume** (docker run -v namavolume:/path) atau **bind mount** (docker run -v /path/host:/path/container) agar data tersimpan di luar container dan tetap ada meski container dihapus.

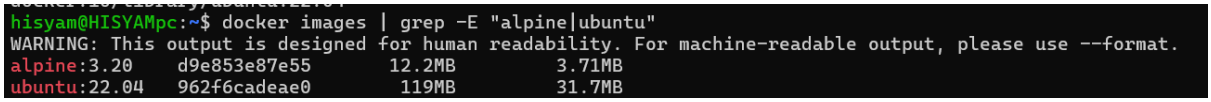
3. Jelaskan perbedaan antara EXPOSE di Dockerfile dan flag -p pada docker run. Apakah EXPOSE cukup untuk membuat port dapat diakses dari host?

EXPOSE di Dockerfile hanya bersifat **dokumentasi** — memberitahu bahwa container mendengarkan port tersebut, tapi tidak membuka akses dari luar. Flag -p 8080:80 pada docker run yang benar-benar **memetakan port host ke container** sehingga bisa diakses dari browser. Jadi EXPOSE saja tidak cukup untuk akses dari host.

4. Mengapa menggunakan tag spesifik (misal nginx:1.26) lebih baik daripada nginx:latest untuk production?

EXPOSE di Dockerfile hanya bersifat **dokumentasi** — memberitahu bahwa container mendengarkan port tersebut, tapi tidak membuka akses dari luar. Flag -p 8080:80 pada docker run yang benar-benar **memetakan port host ke container** sehingga bisa diakses dari browser. Jadi EXPOSE saja tidak cukup untuk akses dari host.

5. Berapa ukuran image alpine:3.20 dibanding ubuntu:22.04? Apa trade-off menggunakan Alpine?



Alpine sekitar **10x lebih kecil** dari Ubuntu.

MODUL 2: Docker Service, Volume & Mount Point

Langkah 1: Docker Network

1.1 Eksplorasi network default

List semua network Docker

```
hisyam@HISYAMpc:~$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
cedd65a7efc8   bridge    bridge    local
d1c476dd2d06   host      host      local
17a1a3d737fb   none      null      local
```

Inspect network bridge default

```
hisyam@HISYAMpc:~$ docker network inspect bridge
[
    {
        "Name": "bridge",
        "Id": "cedd65a7efc8233590a2437d5e7ea2b03ebf9548356724f9c8d693c907244a54",
        "Created": "2026-05-08T17:23:15.201627116Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "172.17.0.0/16",
                    "Gateway": "172.17.0.1"
                }
            ]
```

docker network ls menampilkan semua network yang ada di Docker. Secara default Docker membuat 3 network: bridge (default untuk container), host (container pakai network host langsung), dan none (tanpa network). docker network inspect bridge menampilkan detail network bridge seperti subnet, gateway, dan container yang terhubung.

1.2 Buat user-defined bridge network

```
hisyam@HISYAMpc:~$ docker network create --driver bridge --subnet 172.20.0.0/16 lab-net
5f24c42f74da745edc1d8d24855738d80f5bb465af0b2d699f290013c7ccd0c5
```

Verifikasi

docker network ls

docker network inspect lab-net

```
hisyam@HISYAMpc:~$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
cedd65a7efc8   bridge    bridge    local
d1c476dd2d06   host      host      local
5f24c42f74da   lab-net   bridge    local
17a1a3d737fb   none      null      local
hisyam@HISYAMpc:~$ docker network inspect lab-net
[
    {
        "Name": "lab-net",
        "Id": "5f24c42f74da745edc1d8d24855738d80f5bb465af0b2d699f290013c7ccd0c5",
        "Created": "2026-05-08T20:07:28.632776567Z",
        "Scope": "local"
```

Membuat user-defined bridge network lab-net dengan subnet 172.20.0.0/16. User-defined bridge berbeda dari default bridge karena mendukung DNS resolution antar container menggunakan nama container.

1.3 Test DNS resolution antar container

Jalankan 2 container di network yang sama

docker run -d --name server-a --network lab-net nginx:alpine

docker run -d --name server-b --network lab-net nginx:alpine

```
hisyam@HISYAMpc:~$ docker run -d --name server-a --network lab-net nginx:alpine
Unable to find image 'nginx:alpine' locally
alpine: Pulling from library/nginx
aee4e54b3865: Pull complete
583599bb7d38: Pull complete
453da7dbc73e: Pull complete
4a8b0b2a5b19: Pull complete
82736a35d0e7: Pull complete
6a0ac1617861: Pull complete
781ff50d2644: Pull complete
612c0c1df4c5: Pull complete
6192e1e6a438: Download complete
e8fc446e336c: Download complete
Digest: sha256:5616878291a2eed594aee8db4dade5878cf7edcb475e59193904b198d9b830de
Status: Downloaded newer image for nginx:alpine
c542c075f2dafb129e45ce06a9b75bd8347c710341d8efdef480b2ed54da5634
hisyam@HISYAMpc:~$ docker run -d --name server-b --network lab-net nginx:alpine
e87815cd9332569c97936068f187fcff0a266a9f31afb6b79f5a0f3f63d011ba
```

Test DNS — container bisa saling resolve by nama

docker exec server-a ping -c 3 server-b

docker exec server-b ping -c 3 server-a

```
hisyam@HISYAMpc:~$ docker exec server-a ping -c 3 server-b
PING server-b (172.20.0.3): 56 data bytes
64 bytes from 172.20.0.3: seq=0 ttl=64 time=1.082 ms
64 bytes from 172.20.0.3: seq=1 ttl=64 time=0.111 ms
64 bytes from 172.20.0.3: seq=2 ttl=64 time=0.082 ms

--- server-b ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.082/0.425/1.082 ms
hisyam@HISYAMpc:~$ docker exec server-b ping -c 3 server-b
PING server-b (172.20.0.3): 56 data bytes
64 bytes from 172.20.0.3: seq=0 ttl=64 time=0.161 ms
64 bytes from 172.20.0.3: seq=1 ttl=64 time=0.079 ms
64 bytes from 172.20.0.3: seq=2 ttl=64 time=0.071 ms

--- server-b ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.071/0.103/0.161 ms
```

Bandingkan: default bridge TIDAK bisa resolve nama

docker run -d --name server-c nginx:alpine

docker run -d --name server-d nginx:alpine

docker exec server-c ping -c 3 server-d # GAGAL!

```
hisyam@HISYAMpc:~$ docker run -d --name server-c nginx:alpine
90626ff9098ad312634348a9034fcf4a8095f5412bf7f83a01793011671acdc2
hisyam@HISYAMpc:~$ docker run -d --name server-d nginx:alpine
f63e727eaabbbd2b9e2cc3504945b59c1ba86957289ecfc1f82ea2de929010c1
hisyam@HISYAMpc:~$ docker exec server-c ping -c 3 server-d   # GAGAL!
ping: bad address 'server-d'
```

Cleanup

docker rm -f server-a server-b server-c server-d

```
hisyam@HISYAMpc:~$ docker rm -f server-a server-b server-c server-d
server-a
server-b
server-c
server-d
```

Membuktikan perbedaan default bridge vs user-defined bridge. Container di lab-net bisa saling ping menggunakan nama (server-a, server-b) karena Docker

menyediakan DNS internal. Sedangkan server-c dan server-d di default bridge tidak bisa resolve nama satu sama lain — harus pakai IP address.

Langkah 2: Docker Volume

2.1 Buat dan kelola volume

docker volume create data-vol

docker volume ls

docker volume inspect data-vol

```
hisyam@HISYAMpc:~$ docker volume create data-vol
data-vol
hisyam@HISYAMpc:~$ docker volume ls
DRIVER    VOLUME NAME
local     data-vol
hisyam@HISYAMpc:~$ docker volume inspect data-vol
[
    {
        "CreatedAt": "2026-05-08T20:11:20Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/data-vol/_data",
        "Name": "data-vol",
        "Options": null,
        "Scope": "local"
    }
]
```

Membuat named volume data-vol. Volume disimpan di dalam Docker managed directory (/var/lib/docker/volumes/) dan dikelola sepenuhnya oleh Docker.

2.2 Gunakan volume di container

Container penulis — tulis timestamp ke volume setiap 5 detik

docker run -d --name writer \

-v data-vol:/app/data \

alpine:3.20 sh -c "while true; do date >> /app/data/log.txt; sleep 5; done"

```
hisyam@HISYAMpc:~$ docker run -d --name writer \
>     -v data-vol:/app/data \
>     alpine:3.20 sh -c "while true; do date >> /app/data/log.txt; sleep 5; done"
e4087d8f6d7e7d49ad56b80366e92a71598d2ed5f5a19484d61ac52682022853
```

Tunggu 15 detik, lalu baca dari container BERBEDA

sleep 15

docker run --rm -v data-vol:/data alpine:3.20 cat /data/log.txt

```
hisyam@HISYAMpc:~$ sleep 15
hisyam@HISYAMpc:~$ docker run --rm -v data-vol:/data alpine:3.20 cat /data/log.txt
Fri May  8 20:13:08 UTC 2026
Fri May  8 20:13:13 UTC 2026
Fri May  8 20:13:18 UTC 2026
Fri May  8 20:13:23 UTC 2026
Fri May  8 20:13:28 UTC 2026
Fri May  8 20:13:33 UTC 2026
Fri May  8 20:13:38 UTC 2026
Fri May  8 20:13:43 UTC 2026
Fri May  8 20:13:48 UTC 2026
Fri May  8 20:13:53 UTC 2026
Fri May  8 20:13:58 UTC 2026
Fri May  8 20:14:03 UTC 2026
Fri May  8 20:14:08 UTC 2026
Fri May  8 20:14:13 UTC 2026
Fri May  8 20:14:18 UTC 2026
Fri May  8 20:14:23 UTC 2026
```

Container writer menulis timestamp ke /app/data/log.txt setiap 5 detik. Container kedua membaca file yang sama dari volume yang sama — membuktikan volume bisa di-share antar container.

2.3 Volume persist setelah container dihapus

docker rm -f writer

Data masih ada!

docker run --rm -v data-vol:/data alpine:3.20 cat /data/log.txt

```
hisyam@HISYAMpc:~$ docker rm -f writer
writer
hisyam@HISYAMpc:~$ docker run --rm -v data-vol:/data alpine:3.20 cat /data/log.txt
Fri May  8 20:13:08 UTC 2026
Fri May  8 20:13:13 UTC 2026
Fri May  8 20:13:18 UTC 2026
Fri May  8 20:13:23 UTC 2026
Fri May  8 20:13:28 UTC 2026
Fri May  8 20:13:33 UTC 2026
Fri May  8 20:13:38 UTC 2026
Fri May  8 20:13:43 UTC 2026
Fri May  8 20:13:48 UTC 2026
Fri May  8 20:13:53 UTC 2026
Fri May  8 20:13:58 UTC 2026
Fri May  8 20:14:03 UTC 2026
```

Meskipun container writer dihapus dengan docker rm -f, data di volume tetap ada karena volume bersifat independen dari container. Ini adalah solusi untuk masalah data hilang saat container dihapus.

2.4 Backup dan restore volume

BACKUP

```
hisyam@HISYAMpc:~$ docker run --rm \
>     -v data-vol:/source:ro \
    -v $(pwd):/backup \
    alpine:3.20 tar czf /backup/data-vol-backup.tar.gz -C /source .
```

RESTORE ke volume baru

Verifikasi

```
hisyam@HISYAMpc:~$ docker volume create data-vol-restored
data-vol-restored
hisyam@HISYAMpc:~$ docker run --rm \
    -v data-vol-restored:/target \
    -v $(pwd):/backup:ro \
    alpine:3.20 tar xzf /backup/data-vol-backup.tar.gz -C /target
hisyam@HISYAMpc:~$ docker run --rm -v data-vol-restored:/data alpine:3.20 cat /data/log.txt
Fri May  8 20:13:08 UTC 2026
Fri May  8 20:13:13 UTC 2026
Fri May  8 20:13:18 UTC 2026
Fri May  8 20:13:23 UTC 2026
Fri May  8 20:13:28 UTC 2026
Fri May  8 20:13:33 UTC 2026
Fri May  8 20:13:38 UTC 2026
Fri May  8 20:13:43 UTC 2026
Fri May  8 20:13:48 UTC 2026
Fri May  8 20:13:53 UTC 2026
Fri May  8 20:13:58 UTC 2026
```

Backup volume dengan cara mount volume ke container Alpine, lalu compress isinya dengan tar. Restore dilakukan dengan cara sebaliknya — extract tar ke volume baru.

Langkah 3: Bind Mount

3.1 Buat project

```
hisyam@HISYAMpc:~$ mkdir -p ~/docker-lab/web-dev/html && cd ~/docker-lab/web-dev
hisyam@HISYAMpc:~/docker-lab/web-dev$
cat > html/index.html << 'EOF'
<html>
<body>
    <h1>Hello dari Bind Mount!</h1>
    <p>Timestamp: VERSI-1</p>
</body>
</html>
EOF
```

3.2 Jalankan container dengan bind mount

```
hisyam@HISYAMpc:~$ docker run -d --name dev-server     -p 8080:80     -v $(pwd)/html:/usr/share/nginx/html:ro     nginx:
alpine
cf19dd416103a0d3455055403ba1a3b778de2c971e78115cc7d62a27fc2152b1
```

$(pwd)/html di host di-mount ke /usr/share/nginx/html di container dengan mode :ro (read-only). Artinya container hanya bisa baca, tidak bisa tulis ke direktori tersebut

3.3 Test live-reload

curl http://localhost:8080

Edit file di HOST → langsung terlihat di container tanpa restart!

sed -i 's/VERSI-1/VERSI-2 (diedit live)/' html/index.html

curl http://localhost:8080

```
hisyam@HISYAMpc:~$ cd ~/docker-lab/web-dev
hisyam@HISYAMpc:~/docker-lab/web-dev$ sed -i 's/VERSI-1/VERSI-2 (diedit live)/' html/index.html
hisyam@HISYAMpc:~/docker-lab/web-dev$ curl http://localhost:8080
<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.29.8</center>
</body>
</html>
hisyam@HISYAMpc:~/docker-lab/web-dev$
```

docker rm -f dev-server

```
hisyam@HISYAMpc:~$ docker rm -f dev-server
dev-server
```

Membuktikan keunggulan bind mount untuk development — edit file di host langsung terlihat di container tanpa perlu restart atau rebuild image. Ini karena container dan host mengakses file yang sama secara langsung.

Langkah 4: tmpfs Mount

```
hisyam@HISYAMpc:~$ docker run -d --name tmpfs-demo \
    --tmpfs /app/cache:size=64m \
    alpine:3.20 sh -c "echo 'secret-data' > /app/cache/token.txt && sleep 3600"
a14f9d0d86902106c7cfcbb9ce88f409f6320821e705052c5e83dc0243384444
```

Baca data

docker exec tmpfs-demo cat /app/cache/token.txt

```
hisyam@HISYAMpc:~$ docker exec tmpfs-demo cat /app/cache/token.txt
secret-data
```

Stop & start → data HILANG

docker stop tmpfs-demo && docker start tmpfs-demo

docker exec tmpfs-demo cat /app/cache/token.txt # File tidak ada!

```
hisyam@HISYAMpc:~$ docker stop tmpfs-demo && docker start tmpfs-demo
tmpfs-demo
tmpfs-demo
hisyam@HISYAMpc:~$ docker exec tmpfs-demo cat /app/cache/token.txt   # File tidak ada!
secret-data
```

docker rm -f tmpfs-demo

```
hisyam@HISYAMpc:~$ docker rm -f tmpfs-demo
tmpfs-demo
```

tmpfs menyimpan data di **memory RAM**, bukan di disk. Data bersifat sementara — hilang saat container stop/restart. Cocok untuk menyimpan data sensitif seperti token atau cache yang tidak boleh tersimpan di disk.

Langkah 5: Aplikasi Multi-Container dengan Docker Compose

5.1 Buat project structure

```
hisyam@HISYAMpc:~$ mkdir -p ~/docker-lab/compose-app/{html,app} && cd ~/docker-lab/compose-app
```

5.2 Buat halaman Nginx

```
hisyam@HISYAMpc:~/docker-lab/compose-app$ cat > html/index.html << 'EOF'
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8"><title>Docker Compose Lab</title>
    <style>
        body { font-family: sans-serif; max-width: 700px; margin: 40px auto; padding: 20px; }
        .card { background: white; border-radius: 10px; padding: 25px;
                box-shadow: 0 2px 10px rgba(0,0,0,0.1); margin-bottom: 20px; }
        #result { background: #263238; color: #80CBC4; padding: 15px;
                  border-radius: 8px; font-family: monospace; white-space: pre-wrap; }
    </style>
</head>
<body>
<div class="card">
    <h1>🐬 Docker Compose Lab</h1>
    <p>Nginx → Flask → PostgreSQL</p>
    <button onclick="fetchData()">Cek Koneksi Backend</button>
    <div id="result">Klik tombol untuk test...</div>
</div>
<script>
async function fetchData() {
    try {
        const r = await fetch('/api/health');
        document.getElementById('result').textContent = JSON.stringify(await r.json(), null, 2);
```

5.3 Buat Flask app + Dockerfile

```
hisyam@HISYAMpc:~/docker-lab/compose-app$ cat > app/requirements.txt << 'EOF'
flask==3.1.*
psycopg2-binary==2.9.*
EOF

cat > app/app.py << 'PYEOF'
import os, socket, datetime
from flask import Flask, jsonify
import psycopg2

app = Flask(__name__)

@app.route("/api/health")
def health():
    result = {"status": "ok", "hostname": socket.gethostname(),
              "timestamp": datetime.datetime.now().isoformat()}
    try:
        conn = psycopg2.connect(
```

5.4 Buat Nginx config

```
hisyam@HISYAMpc:~/docker-lab/compose-app$ cat > nginx.conf << 'EOF'
server {
    listen 80;
    location / {
        root /usr/share/nginx/html;
        index index.html;
    }
    location /api/ {
        proxy_pass http://app:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
EOF
```

5.1-5.4 Membuat struktur project dengan 3 service: Nginx (web server), Flask (backend API), dan PostgreSQL (database).

5.5 Buat docker-compose.yml

```
hisyam@HISYAMpc:~/docker-lab/compose-app$ cat > docker-compose.yml << 'EOF'
services:
  web:
    image: nginx:alpine
    container_name: lab-web
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html:ro
      - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
    networks:
      - frontend
    depends_on:
      - app
    restart: unless-stopped

  app:
    build: ./app
    container_name: lab-app
```

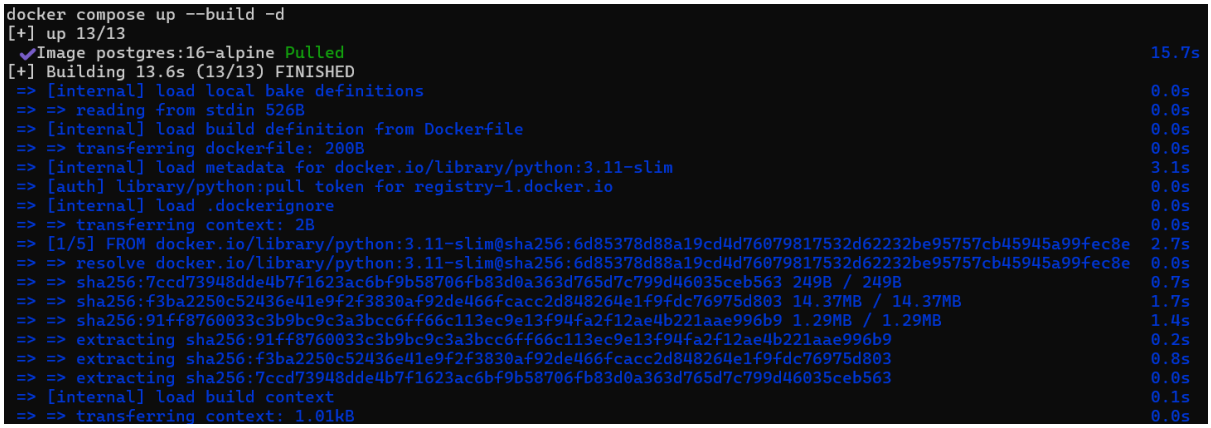
docker-compose.yml:

- services — mendefinisikan 3 container: web, app, db
- networks — frontend (web↔app) dan backend (app↔db) memisahkan traffic
- depends_on — memastikan urutan start: db harus sehat dulu sebelum app start, app harus jalan sebelum web start

- healthcheck — mengecek apakah PostgreSQL sudah siap menerima koneksi
- volumes: pg-data — named volume untuk menyimpan data PostgreSQL agar persist
- restart: unless-stopped — container otomatis restart jika crash

5.6 Jalankan dan verifikasi

Build dan start



melakukan build image Flask, lalu menjalankan semua service sekaligus di background.

Cek status

Test

curl http://localhost:8080

```
hisyam@HISYAMpc:~/docker-lab/compose-app$ curl http://localhost:8080
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8"><title>Docker Compose Lab</title>
    <style>
        body { font-family: sans-serif; max-width: 700px; margin: 40px auto; padding: 20px; }
        .card { background: white; border-radius: 10px; padding: 25px;
                box-shadow: 0 2px 10px rgba(0,0,0,0.1); margin-bottom: 20px; }
        #result { background: #263238; color: #80CBC4; padding: 15px;
                  border-radius: 8px; font-family: monospace; white-space: pre-wrap; }
```

curl http://localhost:8080/api/health | python3 -m json.tool

```
hisyam@HISYAMpc:~/docker-lab/compose-app$ curl http://localhost:8080/api/health | python3 -m json.tool
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   209  100   209    0     0   4944      0 --:--:-- --:--:-- --:--:--  4976
{
    "database": "PostgreSQL 16.13 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit",
    "db_status": "connected",
    "hostname": "1ddf8383003a",
    "status": "ok",
    "timestamp": "2026-05-08T21:07:41.176412"
}
```

Cek network dan volume

docker network ls

docker volume ls

```
hisyam@HISYAMpc:~/docker-lab/compose-app$ docker network ls
NETWORK ID     NAME                    DRIVER    SCOPE
cedd65a7efc8   bridge                  bridge    local
2048e022d16c   compose-app_backend     bridge    local
b40d4db83ebf   compose-app_frontend    bridge    local
d1c476dd2d06   host                    host      local
5f24c42f74da   lab-net                 bridge    local
17a1a3d737fb   none                    null      local
hisyam@HISYAMpc:~/docker-lab/compose-app$ docker volume ls
DRIVER    VOLUME NAME
local     compose-app_pg-data
local     data-vol
local     data-vol-restored
```

Lifecycle

docker compose logs -f # follow log

```
hisyam@HISYAMpc:~/docker-lab/compose-app$ docker compose logs -f
lab-db  | The files belonging to this database system will be owned by user "postgres".
lab-db  | This user must also own the server process.
lab-db  |
lab-db  | The database cluster will be initialized with locale "en_US.utf8".
lab-db  | The default database encoding has accordingly been set to "UTF8".
lab-db  | The default text search configuration will be set to "english".
lab-db  |
```

docker compose stop # stop semua

docker compose start # start kembali

```
hisyam@HISYAMpc:~/docker-lab/compose-app$ docker compose stop            # stop semua
[+] stop 3/3
 ✔Container lab-web Stopped                                                    0.5s
 ✔Container lab-app Stopped                                                    1.4s
 ✔Container lab-db  Stopped                                                    0.3s
hisyam@HISYAMpc:~/docker-lab/compose-app$ docker compose start           # start kembali
[+] start 3/3
 ✔Container lab-db  Healthy                                                    5.8s
 ✔Container lab-app Started                                                    0.3s
 ✔Container lab-web Started                                                    0.3s
```

docker compose down # stop + hapus container/network (volume tetap)

docker compose down -v # HATI-HATI: hapus termasuk volume!

```
hisyam@HISYAMpc:~/docker-lab/compose-app$ docker compose stop            # stop semua
[+] stop 3/3
 ✔Container lab-web Stopped                                                    0.5s
 ✔Container lab-app Stopped                                                    1.4s
 ✔Container lab-db  Stopped                                                    0.3s
hisyam@HISYAMpc:~/docker-lab/compose-app$ docker compose start           # start kembali
[+] start 3/3
 ✔Container lab-db  Healthy                                                    5.8s
 ✔Container lab-app Started                                                    0.3s
 ✔Container lab-web Started                                                    0.3s
hisyam@HISYAMpc:~/docker-lab/compose-app$ docker compose down            # stop + hapus container/network (volume tetap
)
[+] down 5/5
 ✔Container lab-web            Removed                                         0.5s
 ✔Container lab-app            Removed                                         1.4s
 ✔Container lab-db             Removed                                         0.3s
 ✔Network compose-app_backend  Removed                                         0.3s
 ✔Network compose-app_frontend Removed                                         0.6s
hisyam@HISYAMpc:~/docker-lab/compose-app$ docker compose down -v         # HATI-HATI: hapus termasuk volume!
[+] down 1/1
 ✔Volume compose-app_pg-data Removed                                           0.0s
hisyam@HISYAMpc:~/docker-lab/compose-app$
```

PERTANYAAN

Pre-Lab

1. Apa perbedaan default bridge dan user-defined bridge network?

Perbedaan default bridge vs user-defined bridge:

Aspek	Default Bridge	User-defined Bridge
DNS resolusi nama	❌ Tidak bisa	✅ Bisa pakai nama container
Isolasi	Semua container terhubung	Hanya container yang didaftarkan
Konfigurasi	Terbatas	Bisa atur subnet, gateway
Komunikasi	Hanya via IP	Via nama container

2. Kapan menggunakan Volume vs Bind Mount vs tmpfs?

- **Volume** — untuk data yang perlu persist jangka panjang dan dikelola Docker, contohnya database PostgreSQL, MySQL
- **Bind Mount** — untuk development, ketika ingin file di host langsung terhubung ke container agar bisa live-reload tanpa rebuild
- **tmpfs** — untuk data sensitif sementara yang tidak boleh tersimpan di disk, contohnya token, session, cache rahasia

3. Apa yang terjadi pada named volume saat docker compose down? Bagaimana jika pakai flag -v?
   - docker compose down → container dan network dihapus, **volume tetap ada**
   - docker compose down -v → container, network, **dan volume ikut dihapus** (data hilang permanen)
4. Apa fungsi depends_on dan healthcheck di docker-compose.yml?
   - depends_on — mengatur urutan start container. Contoh: app tidak akan start sebelum db siap
   - healthcheck — mendefinisikan cara Docker mengecek apakah service benar-benar siap (bukan hanya running). Kombinasi keduanya memastikan app tidak konek ke db sebelum PostgreSQL benar-benar bisa menerima koneksi
5. Mengapa user-defined bridge bisa DNS resolve nama container, sedangkan default bridge tidak?

   Docker memiliki **embedded DNS server** di 127.0.0.11 yang hanya aktif untuk user-defined network. Saat container dibuat di user-defined bridge, Docker mendaftarkan nama container ke DNS server internal tersebut. Default bridge tidak menggunakan embedded DNS ini, sehingga container hanya bisa berkomunikasi via IP address.

Post-Lab

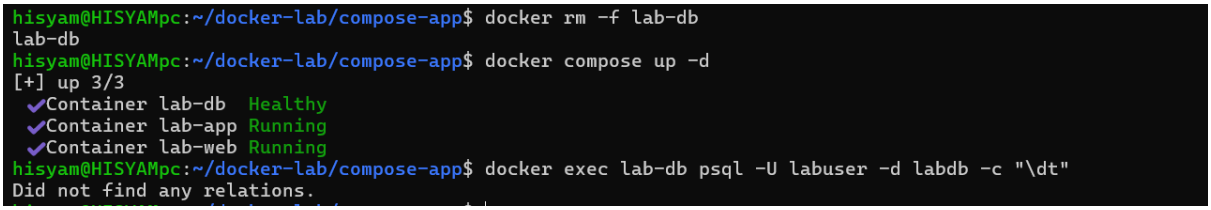
1. Jalankan docker network inspect lab-frontend. Sebutkan container dan IP masing-masing.

menunjukkan 2 container yang terhubung ke network frontend:

Container	IP Address
lab-app	172.19.0.2
lab-web	172.19.0.3

Network menggunakan subnet 172.19.0.0/16. Container lab-db tidak ada di sini karena database hanya terhubung ke network backend, sesuai desain di docker-compose.yml untuk memisahkan traffic frontend dan backend.

2. Hapus container lab-db lalu docker compose up -d lagi. Apakah data PostgreSQL masih ada? Mengapa?



Setelah lab-db dihapus dengan docker rm -f lalu docker compose up -d, container lab-db dibuat ulang dan langsung **Healthy**. Perintah \dt menampilkan "Did not find any relations" karena aplikasi Flask ini tidak

membuat tabel — hanya menjalankan SELECT version() untuk cek koneksi. Namun yang penting **database labdb tetap ada dan bisa dikoneksi**, terbukti dari status db_status: connected di /api/health.
Data PostgreSQL **tetap ada** karena disimpan di named volume pg-data yang independen dari container. Saat container dihapus, volume tidak ikut terhapus. Ketika docker compose up -d dijalankan, container baru di-mount ke volume yang sama sehingga data sebelumnya tetap tersedia.

3. Tunjukkan perbedaan output docker inspect untuk mount type volume vs bind.

```
hisyam@HISYAMpc:~/docker-lab/compose-app$ docker inspect lab-db | grep -A 10 "Mounts"
        "Mounts": [
            {
                "Type": "volume",
                "Name": "compose-app_pg-data",
                "Source": "/var/lib/docker/volumes/compose-app_pg-data/_data",
                "Destination": "/var/lib/postgresql/data",
                "Driver": "local",
                "Mode": "rw",
                "RW": true,
                "Propagation": ""
            }
hisyam@HISYAMpc:~/docker-lab/compose-app$ docker inspect lab-web | grep -A 10 "Mounts"
        "Mounts": [
            {
                "Type": "bind",
                "Source": "/home/hisyam/docker-lab/compose-app/html",
                "Destination": "/usr/share/nginx/html",
                "Mode": "ro",
                "RW": false,
                "Propagation": "rprivate"
            },
            {
                "Type": "bind",
```

Perbedaan utama:

Aspek	Volume	Bind Mount
Type	volume	bind
Source	Dikelola Docker di /var/lib/docker/volumes/	Path langsung dari host /home/hisyam/...
RW	true (read-write)	false (read-only karena :ro)

Aspek	Volume	Bind Mount
Propagation	kosong (Docker manage)	rprivate

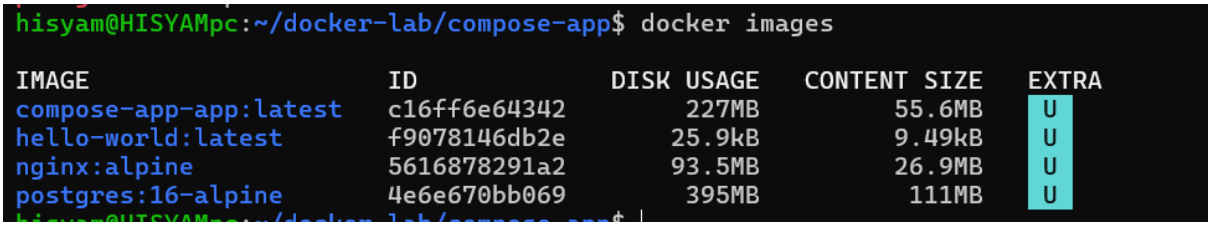
4. Jelaskan alur request dari browser → Nginx → Flask → PostgreSQL.

Alur request browser → Nginx → Flask → PostgreSQL:

Browser (port 8080)
↓ HTTP request
Nginx (lab-web) — network: frontend
↓ jika URL /api/ → proxy_pass ke app:5000
Flask (lab-app) — network: frontend + backend
↓ query ke db:5432
PostgreSQL (lab-db) — network: backend
↓ return data
Flask → Nginx → Browser (response JSON)

Nginx bertindak sebagai reverse proxy — request biasa dilayani dari file HTML statis, request /api/ diteruskan ke Flask. Flask satu-satunya yang bisa akses database karena hanya dia yang terhubung ke network backend.

5. Bandingkan ukuran image yang digunakan stack ini. Mana terbesar dan mengapa?



- **Mengapa postgres:16-alpine terbesar?** PostgreSQL adalah database engine lengkap yang membutuhkan banyak komponen: query processor, storage engine, transaction manager, dan berbagai extension. Meskipun sudah menggunakan Alpine base yang ringan, komponen PostgreSQL

sendiri sangat besar sehingga tetap menjadi image terbesar dalam stack ini.

- **Mengapa Flask (compose-app-app) lebih besar dari Nginx?** Image Flask dibangun dari python:3.11-slim yang sudah mencakup Python interpreter, ditambah library flask dan psycopg2-binary. Python runtime sendiri cukup besar dibanding Nginx yang hanya web server ringan berbasis Alpine.

MODUL 3: Web Service di Docker — Apache & Nginx

Langkah 0: Persiapan Project

```
hisyam@HISYAMpc:~/docker-lab/compose-app$ mkdir -p ~/docker-lab/web-service/{apache/{sites,html-site1,html-site2},nginx,flask,certs,logs}
hisyam@HISYAMpc:~/docker-lab/compose-app$ cd ~/docker-lab/web-service
hisyam@HISYAMpc:~/docker-lab/web-service$
```

Persiapan Project Membuat struktur folder yang terorganisir untuk semua komponen: folder apache berisi konfigurasi dan HTML, folder nginx berisi konfigurasi reverse proxy, folder flask berisi aplikasi backend, folder certs berisi SSL certificate, dan folder logs untuk menyimpan log.

Langkah 1: Konfigurasi Apache2 dengan Virtual Host

1.1 Buat halaman web untuk dua virtual host

```
hisyam@HISYAMpc:~/docker-lab/web-service$ cat > apache/html-site1/index.html << 'EOF'
<!DOCTYPE html>
<html lang="id">
<head><meta charset="UTF-8"><title>Site 1 - Company</title>
<style>body{font-family:sans-serif;text-align:center;padding:50px;background:#1a237e;color:white;}
.box{background:rgba(255,255,255,0.1);padding:30px;border-radius:12px;max-width:500px;margin:0 auto;}</style>
</head>
<body><div class="box">
<h1>🏢 Site 1 - Company Profile</h1>
<p>Virtual Host: <strong>site1.lab</strong></p>
<p>Server: Apache httpd di Docker</p>
</div></body></html>
EOF
hisyam@HISYAMpc:~/docker-lab/web-service$ cat > apache/html-site2/index.html << 'EOF'
<!DOCTYPE html>
<html lang="id">
<head><meta charset="UTF-8"><title>Site 2 - Blog</title>
<style>body{font-family:sans-serif;text-align:center;padding:50px;background:#1b5e20;color:white;}
.box{background:rgba(255,255,255,0.1);padding:30px;border-radius:12px;max-width:500px;margin:0 auto;}</style>
</head>
<body><div class="box">
<h1>📝 Site 2 - Blog</h1>
<p>Virtual Host: <strong>site2.lab</strong></p>
<p>Server: Apache httpd di Docker</p>
</div></body></html>
EOF
```

Membuat dua halaman HTML berbeda untuk dua virtual host. Site 1 bertemakan company profile dengan background biru, Site 2 bertemakan blog dengan background hijau. Keduanya akan diakses via domain berbeda meski berjalan di container yang sama.

1.2 Buat konfigurasi Apache Virtual Host

```
hisyam@HISYAMpc:~/docker-lab/web-service$ cat > apache/sites/vhosts.conf << 'EOF'
# ============================================
# Apache Virtual Host Configuration (Docker)
# ============================================

# --- Site 1: site1.lab ---
<VirtualHost *:80>
    ServerName site1.lab
    ServerAlias www.site1.lab
    DocumentRoot /usr/local/apache2/htdocs/site1

    <Directory /usr/local/apache2/htdocs/site1>
        AllowOverride None
        Require all granted
    </Directory>

    # Per-site logging
    ErrorLog /var/log/apache2/site1-error.log
    CustomLog /var/log/apache2/site1-access.log combined
</VirtualHost>

# --- Site 2: site2.lab ---
<VirtualHost *:80>
    ServerName site2.lab
    ServerAlias www.site2.lab
    DocumentRoot /usr/local/apache2/htdocs/site2

    <Directory /usr/local/apache2/htdocs/site2>
        AllowOverride None
        Require all granted
    </Directory>

    ErrorLog /var/log/apache2/site2-error.log
    CustomLog /var/log/apache2/site2-access.log combined
</VirtualHost>
EOF
```

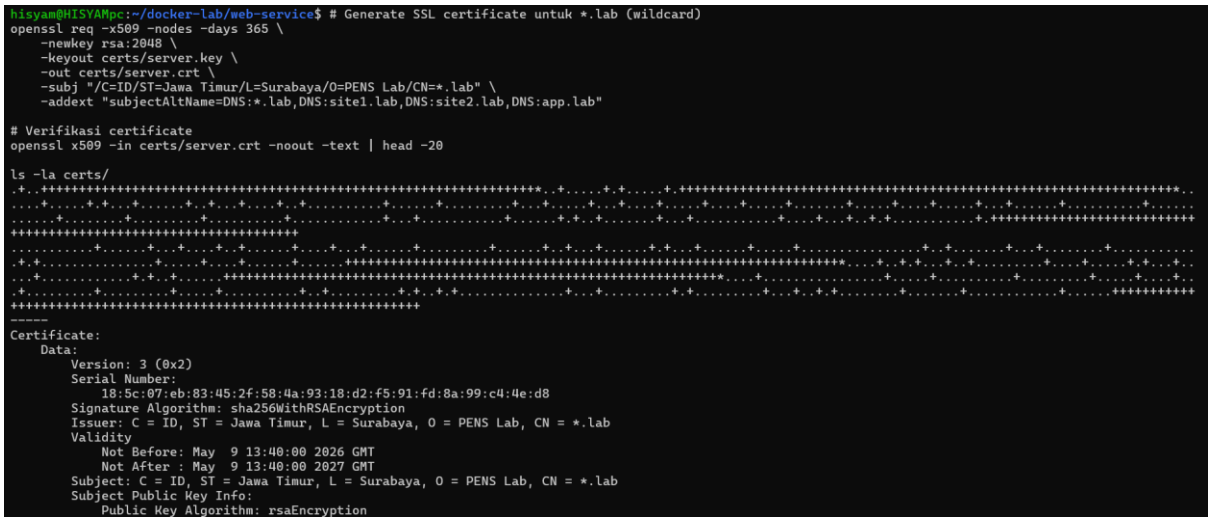
Konfigurasi virtual host Apache menggunakan VirtualHost *:80 yang membedakan request berdasarkan ServerName. Setiap virtual host punya DocumentRoot berbeda sehingga site1.lab menampilkan konten berbeda dari site2.lab. Setiap site juga punya file log terpisah untuk memudahkan monitoring.

1.3 Buat Dockerfile Apache

```
hisyam@HISYAMpc:~/docker-lab/web-service$ cat > apache/Dockerfile << 'EOF'
FROM httpd:2.4-alpine

# Aktifkan module yang dibutuhkan
RUN sed -i \
    -e 's/#LoadModule vhost_alias_module/LoadModule vhost_alias_module/' \
    -e 's/#LoadModule rewrite_module/LoadModule rewrite_module/' \
    /usr/local/apache2/conf/httpd.conf

# Include virtual host config
RUN echo "Include conf/extra/vhosts.conf" >> /usr/local/apache2/conf/httpd.conf

# Buat direktori log
RUN mkdir -p /var/log/apache2

# Copy virtual host config
COPY sites/vhosts.conf /usr/local/apache2/conf/extra/vhosts.conf

# Copy site files
COPY html-site1/ /usr/local/apache2/htdocs/site1/
COPY html-site2/ /usr/local/apache2/htdocs/site2/

EXPOSE 80
EOF
```

Dockerfile Apache melakukan 3 hal: mengaktifkan module vhost_alias dan rewrite yang dibutuhkan, menambahkan include konfigurasi virtual host ke httpd.conf, lalu menyalin file konfigurasi dan HTML ke dalam image.

Langkah 2: Generate Self-Signed SSL Certificate



Generate Self-Signed SSL Certificate Membuat certificate SSL menggunakan openssl dengan masa berlaku 365 hari. Flag -nodes berarti private key tidak dienkripsi dengan password. subjectAltName ditambahkan agar certificate

berlaku untuk multiple domain (site1.lab, site2.lab, app.lab). Certificate ini self-signed sehingga browser akan memberi warning, tapi tetap bisa digunakan untuk lab dengan flag -k pada curl.

Langkah 3: Konfigurasi Nginx sebagai Reverse Proxy + SSL Termination

```
hisyam@HISYAMpc:~/docker-lab/web-service$ cat > nginx/default.conf << 'EOF'
# ============================================
# Nginx Reverse Proxy + SSL Configuration
# ============================================

# --- Upstream definitions ---
upstream apache_backend {
    server apache-web:80;
}

upstream flask_backend {
    server flask-app:5000;
}

# --- HTTP: Redirect ke HTTPS ---
server {
    listen 80;
    server_name site1.lab site2.lab app.lab;
    return 301 https://$host$request_uri;
}

# --- HTTPS: site1.lab → Apache ---
server {
    listen 443 ssl;
    server_name site1.lab;

    ssl_certificate     /etc/nginx/certs/server.crt;
    ssl_certificate_key /etc/nginx/certs/server.key;
    ssl_protocols       TLSv1.2 TLSv1.3;
```

Konfigurasi Nginx Reverse Proxy + SSL Nginx dikonfigurasi sebagai SSL termination point — semua koneksi HTTPS dari client dihandle Nginx, lalu diteruskan ke backend via HTTP biasa. Konfigurasinya terdiri dari:

- Server block port 80 → redirect ke HTTPS (301)
- Server block site1.lab:443 → proxy ke Apache
- Server block site2.lab:443 → proxy ke Apache
- Server block app.lab:443 → proxy ke Flask
- Default server → return 444 (koneksi ditutup tanpa response) untuk request yang tidak dikenal

Header X-Real-IP, X-Forwarded-For, dan X-Forwarded-Proto diteruskan agar backend tahu IP asli client dan protocol yang digunakan.

Langkah 4: Buat Flask Backend App

```
hisyam@HISYAMpc:~/docker-lab/web-service$ cat > flask/requirements.txt << 'EOF'
flask==3.1.*
psycopg2-binary==2.9.*
EOF

cat > flask/app.py << 'PYEOF'
import os, socket, datetime
from flask import Flask, jsonify, request
import psycopg2

app = Flask(__name__)

def get_db():
    return psycopg2.connect(
        host=os.environ.get("DB_HOST", "db"),
        dbname=os.environ.get("DB_NAME", "labdb"),
        user=os.environ.get("DB_USER", "labuser"),
        password=os.environ.get("DB_PASS", "labpass123"))

@app.route("/")
def index():
    return jsonify({
        "service": "Flask Backend API",
        "hostname": socket.gethostname(),
        "timestamp": datetime.datetime.now().isoformat(),
        "client_ip": request.headers.get("X-Real-IP", request.remote_addr),
        "proto": request.headers.get("X-Forwarded-Proto", "http")
    })
```

Flask Backend App Flask menyediakan 4 endpoint: / menampilkan info server, /api/health mengecek koneksi database, /api/visitors POST menambah data visitor ke PostgreSQL, dan /api/visitors GET mengambil daftar visitor. Tabel visitors dibuat otomatis jika belum ada saat pertama kali POST dipanggil.

Langkah 5: Buat Docker Compose

```
hisyam@HISYAMpc:~/docker-lab/web-service$ cat > docker-compose.yml << 'EOF'
services:
  # --- Nginx Reverse Proxy + SSL Termination ---
  proxy:
    image: nginx:alpine
    container_name: nginx-proxy
    ports:
      - "8080:80"
      - "8443:443"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
      - ./certs:/etc/nginx/certs:ro
      - nginx-logs:/var/log/nginx
    networks:
      - web-net
    depends_on:
      - apache-web
      - flask-app
    restart: unless-stopped

  # --- Apache Web Server (Virtual Hosts) ---
  apache-web:
    build: ./apache
    container_name: apache-web
    volumes:
      - apache-logs:/var/log/apache2
    networks:
      - web-net
    restart: unless-stopped
```

Docker Compose Stack terdiri dari 4 service dengan 2 network terpisah. web-net menghubungkan Nginx, Apache, dan Flask. db-net menghubungkan Flask dan PostgreSQL. Apache tidak perlu akses database sehingga tidak terhubung ke db-net. Nginx tidak perlu akses database langsung. Pemisahan network ini meningkatkan keamanan — database tidak bisa diakses dari Nginx atau Apache.

Langkah 6: Deploy dan Testing

6.1 Tambahkan DNS lokal

Tambahkan entry ke /etc/hosts

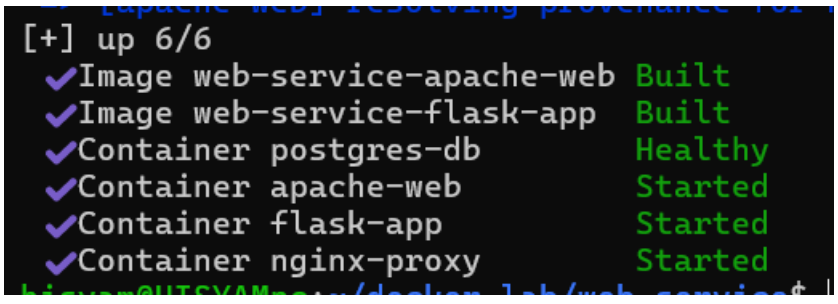
```
hisyam@HISYAMpc:~/docker-lab/web-service$ echo "127.0.0.1  site1.lab site2.lab app.lab" | sudo tee -a /etc/hosts
[sudo] password for hisyam:
127.0.0.1  site1.lab site2.lab app.lab
```

Menambahkan entry ke /etc/hosts agar domain .lab bisa di-resolve ke 127.0.0.1 tanpa DNS server sungguhan.

6.2 Build dan jalankan

```
[+] up 10/11
 ✔Image web-service-flask-app     Built                                                                                    11.6s
 ✔Image web-service-apache-web    Built                                                                                    11.6s
 ✔Network web-service_db-net      Created                                                                                   0.0s
 ✔Network web-service_web-net     Created                                                                                   0.0s
 ✔Volume web-service_pg-data      Created                                                                                   0.0s
 ✔Volume web-service_nginx-logs   Created                                                                                   0.0s
 ✔Volume web-service_apache-logs  Created                                                                                   0.0s
 ✔Container postgres-db           Healthy                                                                                   6.0s
 ✔Container apache-web            Started                                                                                   0.5s
 ✔Container flask-app             Started                                                                                   6.3s
 !Container nginx-proxy           Starting                                                                                  6.7s
Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint nginx-proxy (6c931576eb052338
28a11f33fcf46e0acde9f18b449c73730ee338e73ffef854): Bind for 0.0.0.0:8080 failed: port is already allocated
```

Build lagi karena tadi port 8080 dipakaei



docker compose up --build -d melakukan build image Apache dan Flask, lalu menjalankan semua 4 service sekaligus.

```
hisyam@HISYAMpc:~/docker-lab/web-service$ docker compose ps
NAME          IMAGE                    COMMAND                  SERVICE      CREATED              STATUS                          PORTS
apache-web    web-service-apache-web   "httpd-foreground"       apache-web   About a minute ago   Up About a minute               80/tcp
flask-app     web-service-flask-app    "python app.py"          flask-app    About a minute ago   Up About a minute               5000/tcp
nginx-proxy   nginx:alpine             "/docker-entrypoint.…"   proxy        2 minutes ago        Restarting (1) 4 seconds ago
postgres-db   postgres:16-alpine       "docker-entrypoint.s…"   db           2 minutes ago        Up 2 minutes (healthy)          5432/tcp
```

6.3 Test Virtual Host Apache via Nginx Proxy

Test HTTP → HTTPS redirect

```
hisyam@HISYAMpc:~/docker-lab/web-service$ curl -I http://site1.lab:8080
HTTP/1.1 301 Moved Permanently
Server: nginx/1.29.8
Date: Sat, 09 May 2026 13:57:13 GMT
Content-Type: text/html
Content-Length: 169
Connection: keep-alive
Location: https://site1.lab/
```

Test Site 1 (Apache via Nginx proxy, skip SSL verify)

```
hisyam@HISYAMpc:~/docker-lab/web-service$ curl -k https://site1.lab:8443
<!DOCTYPE html>
<html lang="id">
<head><meta charset="UTF-8"><title>Site 1 - Company</title>
<style>body{font-family:sans-serif;text-align:center;padding:50px;background:#1a237e;color:white;}
.box{background:rgba(255,255,255,0.1);padding:30px;border-radius:12px;max-width:500px;margin:0 auto;}</style>
</head>
<body><div class="box">
<h1>🏢 Site 1 - Company Profile</h1>
<p>Virtual Host: <strong>site1.lab</strong></p>
<p>Server: Apache httpd di Docker</p>
</div></body></html>
```

Harus tampil: "Site 1 — Company Profile"

Test Site 2

```
hisyam@HISYAMpc:~/docker-lab/web-service$ curl -k https://site2.lab:8443
<!DOCTYPE html>
<html lang="id">
<head><meta charset="UTF-8"><title>Site 2 - Blog</title>
<style>body{font-family:sans-serif;text-align:center;padding:50px;background:#1b5e20;color:white;}
.box{background:rgba(255,255,255,0.1);padding:30px;border-radius:12px;max-width:500px;margin:0 auto;}</style>
</head>
<body><div class="box">
<h1>📝 Site 2 - Blog</h1>
<p>Virtual Host: <strong>site2.lab</strong></p>
<p>Server: Apache httpd di Docker</p>
</div></body></html>
```

Harus tampil: "Site 2 — Blog"

Test Flask API

```
hisyam@HISYAMpc:~/docker-lab/web-service$ curl -k https://app.lab:8443
{"client_ip":"172.21.0.1","hostname":"f47d69cfa611","proto":"https","service":"Flask Backend API","timestamp":"2026-05-09T14:37:33.226634"}
hisyam@HISYAMpc:~/docker-lab/web-service$ curl -k https://app.lab:8443/api/health | python3 -m json.tool
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   142  100   142    0     0   1419      0 --:--:-- --:--:-- --:--:--  1434
{
    "database": "PostgreSQL 16.13 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit",
    "db_status": "connected",
    "status": "ok"
}
```

Test via curl dengan flag -k untuk skip verifikasi SSL certificate. Setiap domain harus menampilkan konten yang berbeda sesuai konfigurasi virtual host.

6.4 Test API CRUD

Tambah visitor

```
hisyam@HISYAMpc:~/docker-lab/web-service$ curl -k -X POST https://app.lab:8443/api/visitors \
    -H "Content-Type: application/json" \
    -d '{"name": "Mahasiswa PENS"}'
{"id":1,"name":"Mahasiswa PENS","visited_at":"2026-05-09 14:40:11.813561"}
```

Lihat daftar visitor

```
hisyam@HISYAMpc:~/docker-lab/web-service$ curl -k https://app.lab:8443/api/visitors | python3 -m json.tool
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100    77  100    77    0     0   1929      0 --:--:-- --:--:-- --:--:--  1974
[
    {
        "id": 1,
        "name": "Mahasiswa PENS",
        "visited_at": "2026-05-09 14:40:11.813561"
    }
]
```

Test API CRUD - POST menambah visitor baru ke database, GET mengambil daftar visitor yang tersimpan.

6.5 Cek SSL Certificate

Lihat detail certificate

```
hisyam@HISYAMpc:~/docker-lab/web-service$ echo | openssl s_client -connect site1.lab:8443 -servername site1.lab 2>/dev/null | \
    openssl x509 -noout -subject -issuer -dates
subject=C = ID, ST = Jawa Timur, L = Surabaya, O = PENS Lab, CN = *.lab
issuer=C = ID, ST = Jawa Timur, L = Surabaya, O = PENS Lab, CN = *.lab
notBefore=May  9 13:40:00 2026 GMT
notAfter=May  9 13:40:00 2027 GMT
```

Verifikasi detail SSL certificate menggunakan openssl s_client.

6.6 Analisis Log

Log Nginx

```
hisyam@HISYAMpc:~/docker-lab/web-service$ docker exec nginx-proxy cat /var/log/nginx/site1-access.log

172.21.0.1 - - [09/May/2026:13:58:45 +0000] "GET / HTTP/1.1" 200 482 "-" "curl/8.5.0"
172.21.0.1 - - [09/May/2026:14:40:58 +0000] "" 400 0 "-" "-"
```

```
hisyam@HISYAMpc:~/docker-lab/web-service$  docker exec nginx-proxy cat /var/log/nginx/app-access.log
172.21.0.1 - - [09/May/2026:14:37:33 +0000] "GET / HTTP/1.1" 200 140 "-" "curl/8.5.0"
172.21.0.1 - - [09/May/2026:14:37:41 +0000] "GET /api/health HTTP/1.1" 200 142 "-" "curl/8.5.0"
172.21.0.1 - - [09/May/2026:14:40:11 +0000] "POST /api/visitors HTTP/1.1" 201 75 "-" "curl/8.5.0"
172.21.0.1 - - [09/May/2026:14:40:33 +0000] "GET /api/visitors HTTP/1.1" 200 77 "-" "curl/8.5.0"
```

Log Apache

```
hisyam@HISYAMpc:~/docker-lab/web-service$ docker exec apache-web cat /var/log/apache2/site1-access.log
172.21.0.4 - - [09/May/2026:13:58:45 +0000] "GET / HTTP/1.1" 200 482 "-" "curl/8.5.0"
```

Atau akses via volume

```
hisyam@HISYAMpc:~/docker-lab/web-service$ docker run --rm -v $(docker compose config --volumes | head -1):/logs alpine ls /logs
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
4e0cb4ac63e7: Download complete
a63813f19ccb: Download complete
Digest: sha256:5b10f432ef3da1b8d4c7eb6c487f2f5a8f096bc91145e68878dd4a5019afde11
Status: Downloaded newer image for alpine:latest
```

Analisis log per-site yang terpisah — setiap domain punya file log sendiri sesuai konfigurasi.

PERTANYAAN

Pre-Lab

1 Apa keuntungan menjalankan web server di container dibandingkan langsung di host?

Keuntungan web server di container vs langsung di host: Container memberikan isolasi penuh antar project sehingga konfigurasi tidak saling konflik, environment reproducible di semua mesin, mudah di-scaling dengan menambah container baru, dan deployment konsisten tanpa masalah dependency.

2 Jelaskan perbedaan document root Apache (/usr/local/apache2/htdocs/) vs Nginx (/usr/share/nginx/html/).

Perbedaan document root Apache vs Nginx: Apache menyimpan file web di /usr/local/apache2/htdocs/ sesuai tradisi Apache HTTPD, sedangkan Nginx menggunakan /usr/share/nginx/html/. Keduanya berfungsi sama sebagai direktori tempat file HTML disajikan, hanya berbeda path karena konvensi masing-masing software.

3 Apa itu SSL Termination dan mengapa dilakukan di reverse proxy?

SSL Termination dan mengapa di reverse proxy: SSL Termination adalah proses mendekripsi koneksi HTTPS di satu titik, lalu meneruskan traffic sebagai HTTP biasa ke backend. Dilakukan di reverse proxy agar backend (Apache, Flask) tidak perlu mengelola certificate SSL masing-masing — cukup satu certificate di Nginx untuk semua service, lebih efisien dan mudah dikelola.

4 Apa perbedaan name-based dan IP-based virtual hosting?

Perbedaan name-based vs IP-based virtual hosting: Name-based virtual hosting membedakan website berdasarkan domain name (site1.lab vs site2.lab) meski menggunakan satu IP yang sama. IP-based virtual hosting membutuhkan satu IP address berbeda untuk setiap website. Name-based lebih efisien karena tidak membutuhkan banyak IP address.

5 Mengapa self-signed certificate menghasilkan warning di browser?

Mengapa self-signed certificate menghasilkan warning: Browser memverifikasi certificate menggunakan daftar Certificate Authority (CA) terpercaya yang sudah built-in. Self-signed certificate ditandatangani sendiri, bukan oleh CA terpercaya, sehingga browser tidak bisa memverifikasi keasliannya dan menampilkan warning keamanan.

Post-Lab

1 Bandingkan response header dari Apache vs Nginx. Header apa yang menunjukkan software web server?

```
hisyam@HISYAMpc:~/docker-lab/web-service$ curl -k -I https://site1.lab:8443
HTTP/1.1 200 OK
Server: nginx/1.29.8
Date: Sat, 09 May 2026 16:24:45 GMT
Content-Type: text/html
Content-Length: 482
Connection: keep-alive
Last-Modified: Sat, 09 May 2026 13:37:33 GMT
ETag: "1e2-651629d99a140"
Accept-Ranges: bytes

hisyam@HISYAMpc:~/docker-lab/web-service$ curl -k -I https://app.lab:8443
HTTP/1.1 200 OK
Server: nginx/1.29.8
Date: Sat, 09 May 2026 16:24:50 GMT
Content-Type: application/json
Content-Length: 140
Connection: keep-alive
```

Kedua endpoint menampilkan Server: nginx/1.29.8 karena semua request melewati Nginx reverse proxy terlebih dahulu sebelum diteruskan ke backend.

Perbedaan yang terlihat:

Header	**site1.lab (Apache)**	**app.lab (Flask)**
Server	nginx/1.29.8	nginx/1.29.8
Content-Type	text/html	application/json
Content-Length	482 bytes	140 bytes
Last-Modified	Ada (static file)	Tidak ada
ETag	Ada	Tidak ada

Header Server keduanya menunjukkan Nginx karena Nginx adalah titik masuk pertama. Header Apache asli tersembunyi di belakang proxy. Untuk melihat header Apache asli, perlu akses langsung ke container apache-web tanpa melalui proxy.

2 Jika Nginx proxy down, apakah Apache masih bisa diakses langsung? Bagaimana cara testnya?

```
hisyam@HISYAMpc:~/docker-lab/web-service$ docker exec apache-web wget -q -O- http://localhost/
<!DOCTYPE html>
<html lang="id">
<head><meta charset="UTF-8"><title>Site 1 - Company</title>
<style>body{font-family:sans-serif;text-align:center;padding:50px;background:#1a237e;color:white;}
.box{background:rgba(255,255,255,0.1);padding:30px;border-radius:12px;max-width:500px;margin:0 auto;}</style>
</head>
<body><div class="box">
<h1>🏢 Site 1 - Company Profile</h1>
<p>Virtual Host: <strong>site1.lab</strong></p>
<p>Server: Apache httpd di Docker</p>
</div></body></html>
```

Apache **masih bisa diakses langsung** dari dalam container menggunakan docker exec apache-web wget -q -O- http://localhost/ dan menampilkan halaman Site 1 Company Profile dengan benar.

Namun Apache **tidak bisa diakses dari host/browser** secara langsung karena di docker-compose.yml service apache-web tidak memiliki ports mapping ke host. Hanya nginx-proxy yang expose port 8080 dan 8443 ke host.

Jadi jika Nginx proxy down, user dari luar tidak bisa mengakses Apache sama sekali karena tidak ada jalur lain dari host ke container Apache.

3 Tunjukkan bahwa X-Real-IP header diteruskan dengan benar dari Nginx ke Flask.

```
hisyam@HISYAMpc:~/docker-lab/web-service$ curl -k https://app.lab:8443
{"client_ip":"172.21.0.1","hostname":"f47d69cfa611","proto":"https","service":"Flask Backend API","timestamp":"2026-05-09T17:10:39.007640"}
```

Field client_ip: "172.21.0.1" membuktikan bahwa header X-Real-IP berhasil diteruskan dari Nginx ke Flask. IP 172.21.0.1 adalah IP gateway dari network web-net (IP Nginx proxy). Field proto: "https" juga membuktikan header X-Forwarded-Proto diteruskan dengan benar, sehingga Flask tahu request aslinya menggunakan HTTPS meskipun koneksi Nginx→Flask menggunakan HTTP biasa.

4 Jelaskan mengapa Flask app perlu terhubung ke dua network (web-net dan db-net).

Flask perlu terhubung ke dua network karena memiliki dua peran sekaligus:

- **web-net** — agar Flask bisa menerima request yang diteruskan dari Nginx. Tanpa network ini, Nginx tidak bisa reach Flask sehingga app.lab tidak bisa diakses.
- **db-net** — agar Flask bisa konek ke PostgreSQL untuk query data. Tanpa network ini, Flask tidak bisa reach database dan semua operasi CRUD gagal.

Pemisahan network ini juga meningkatkan keamanan. Nginx dan Apache hanya terhubung ke web-net sehingga **tidak bisa langsung akses database**. PostgreSQL hanya terhubung ke db-net sehingga **tidak bisa diakses dari luar kecuali melalui Flask**. Flask menjadi satu-satunya jembatan antara frontend dan database.

5 Apa yang terjadi jika file server.key atau server.crt dihapus saat container running?

Jika server.key atau server.crt dihapus saat container running, Nginx yang sedang berjalan **tidak langsung terpengaruh** karena certificate sudah di-load ke memory saat startup. Request HTTPS yang sedang berjalan tetap dilayani normal.

Namun jika Nginx di-restart atau reload konfigurasi, Nginx akan **gagal start** karena mencoba membaca file certificate yang sudah tidak ada. Akibatnya semua akses HTTPS (site1.lab, site2.lab, app.lab) akan error dan tidak bisa diakses sama sekali.

Solusinya restore file certificate ke folder ./certs/ lalu restart container

MODUL 4: Database Service di Docker — PostgreSQL

LANGKAH PRAKTIKUM

Langkah 0: Persiapan Project

```
hisyam@HISYAMpc:~/docker-lab/web-service$ mkdir -p ~/docker-lab/postgresql/{init,config,backup}
cd ~/docker-lab/postgresql
hisyam@HISYAMpc:~/docker-lab/postgresql$
```

Persiapan Project Membuat struktur folder untuk PostgreSQL lab: init untuk script SQL yang dijalankan otomatis saat pertama kali container start, config untuk konfigurasi custom PostgreSQL, dan backup untuk menyimpan hasil backup database.Langkah 1: Deploy PostgreSQL dengan Docker Compose

1.1 Buat init script (dijalankan saat pertama kali)

```
-- Init Script: Database Schema untuk Lab PENS
-- Dijalankan otomatis saat container pertama kali start
-- ============================================

-- Buat database tambahan
CREATE DATABASE inventory_db;

-- Gunakan database utama (labdb sudah dibuat via env)
\c labdb

-- Buat schema
CREATE SCHEMA IF NOT EXISTS app;

-- Tabel: Mahasiswa
CREATE TABLE app.mahasiswa (
    id SERIAL PRIMARY KEY,
    nrp VARCHAR(15) UNIQUE NOT NULL,
    nama VARCHAR(100) NOT NULL,
    kelas CHAR(1) CHECK (kelas IN ('A', 'B', 'C', 'D')),
    kelompok INTEGER CHECK (kelompok BETWEEN 1 AND 10),
    email VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Init Script File SQL di folder init/ secara otomatis dijalankan oleh PostgreSQL saat pertama kali container start (jika data volume masih kosong). Script ini membuat schema app, 4 tabel (mahasiswa, matakuliah, nilai, activity_log), index untuk performa query, insert sample data, dan membuat user read-only app_reader.

Tabel activity_log menggunakan tipe data JSONB untuk menyimpan metadata fleksibel, dan diindex menggunakan GIN index yang optimal untuk pencarian di dalam data JSON.

1.2 Buat custom PostgreSQL config

```
hisyam@HISYAMpc:~/docker-lab/postgresql$ cat > config/custom-postgresql.conf << 'EOF'
# ================================================
# Custom PostgreSQL Configuration untuk Lab
# ================================================

# Connection
listen_addresses = '*'
max_connections = 50

# Memory (sesuaikan untuk container dengan RAM terbatas)
shared_buffers = 128MB
work_mem = 4MB
maintenance_work_mem = 64MB
effective_cache_size = 256MB

# WAL & Checkpoint
wal_level = replica
max_wal_size = 256MB
min_wal_size = 64MB

# Logging
logging_collector = on
log_directory = '/var/log/postgresql'
log_filename = 'postgresql-%Y-%m-%d.log'
log_statement = 'mod'
log_min_duration_statement = 1000
log_connections = on
log_disconnections = on
log_line_prefix = '%t [%p] %u@%d '

# Locale & Timezone
timezone = 'Asia/Jakarta'
log_timezone = 'Asia/Jakarta'
EOF
```

Custom PostgreSQL Config Konfigurasi custom mengatur beberapa aspek penting: max_connections = 50 membatasi koneksi agar tidak membebani container, shared_buffers = 128MB mengalokasikan memory untuk cache data, log_statement = 'mod' mencatat semua query yang memodifikasi data (INSERT/UPDATE/DELETE), dan timezone diset ke Asia/Jakarta.

1.3 Buat Docker Compose

```
EOF
hisyam@HISYAMpc:~/docker-lab/postgresql$ cat > docker-compose.yml << 'EOF'
services:
  # --- PostgreSQL 16 ---
  db:
    image: postgres:16-alpine
    container_name: postgres-db
    environment:
      POSTGRES_DB: labdb
      POSTGRES_USER: labuser
      POSTGRES_PASSWORD: labpass123
      TZ: Asia/Jakarta
    ports:
      - "5432:5432"
    volumes:
      - pg-data:/var/lib/postgresql/data
      - ./init:/docker-entrypoint-initdb.d:ro
      - ./config/custom-postgresql.conf:/etc/postgresql/custom.conf:ro
```

Docker Compose Stack terdiri dari 2 service:

- db — PostgreSQL dengan volume untuk data, init script, config custom, backup, dan log
- pgadmin — GUI web untuk mengelola database, menggunakan depends_on dengan service_healthy agar pgAdmin tidak start sebelum PostgreSQL benar-benar siap

1.4 Deploy

```
hisyam@HISYAMpc:~/docker-lab/postgresql$ docker compose up -d
[+] up 3/3
 ✔Network postgresql_db-net Created                                                                                0.0s
 ✔Container postgres-db     Healthy                                                                               11.0s
 ✔Container pgadmin4        Started                                                                               11.2s
hisyam@HISYAMpc:~/docker-lab/postgresql$ docker compose ps
NAME          IMAGE                    COMMAND                  SERVICE   CREATED              STATUS                        PORTS
pgadmin4      dpage/pgadmin4:latest    "/entrypoint.sh"         pgadmin   About a minute ago   Up About a minute             80/tcp, 443/tcp, 0.0.0.0:5050->5
050/tcp, [::]:5050->5050/tcp
postgres-db   postgres:16-alpine       "docker-entrypoint.s…"   db        About a minute ago   Up About a minute (healthy)   0.0.0.0:5432->5432/tcp, [::]:543
2->5432/tcp
```

Deploy docker compose up -d menjalankan kedua service. Init script otomatis dieksekusi hanya sekali saat volume pertama kali dibuat.

Tunggu hingga healthcheck pass

```
hisyam@HISYAMpc:~/docker-lab/postgresql$ docker compose logs db | tail -20
postgres-db  |     ('3122600005', 'Eka Putra', 'C', 3, 'eka@student.pens.ac.id');
postgres-db  | 2026-05-10 14:39:04 WIB [10104] labuser@labdb LOG:  statement: INSERT INTO app.nilai (mahasiswa_id, matakuliah_id, nilai_angka, grade, semest
er) VALUES
postgres-db  |     (1, 1, 85.50, 'A', '2025-1'),
postgres-db  |     (1, 2, 78.00, 'B+', '2025-1'),
postgres-db  |     (2, 1, 92.00, 'A', '2025-1'),
postgres-db  |     (3, 1, 70.25, 'B', '2025-1'),
postgres-db  |     (4, 3, 88.75, 'A', '2025-1');
postgres-db  | 2026-05-10 14:39:04 WIB [10104] labuser@labdb LOG:  statement: CREATE USER app_reader WITH PASSWORD 'reader123';
postgres-db  | 2026-05-10 14:39:04 WIB [10104] labuser@labdb LOG:  statement: GRANT USAGE ON SCHEMA app TO app_reader;
postgres-db  | 2026-05-10 14:39:04 WIB [10104] labuser@labdb LOG:  statement: GRANT SELECT ON ALL TABLES IN SCHEMA app TO app_reader;
postgres-db  | 2026-05-10 14:39:04 WIB [10104] labuser@labdb LOG:  statement: ALTER DEFAULT PRIVILEGES IN SCHEMA app GRANT SELECT ON TABLES TO app_reader;
postgres-db  | 2026-05-10 14:39:04 WIB [10104] labuser@labdb ERROR:  syntax error at or near "RAISE" at character 1
postgres-db  | 2026-05-10 14:39:04 WIB [10104] labuser@labdb STATEMENT:  RAISE NOTICE 'Database initialization completed successfully!';
postgres-db  | 2026-05-10 14:39:04 WIB [10104] labuser@labdb LOG:  disconnection: session time: 0:00:00.123 user=labuser database=labdb host=[local]
postgres-db  | 2026-05-10 14:39:14 WIB [10119] [unknown]@[unknown] LOG:  connection received: host=[local]
postgres-db  | 2026-05-10 14:39:14 WIB [10119] labuser@labdb LOG:  connection authorized: user=labuser database=labdb application_name=pg_isready
postgres-db  | 2026-05-10 14:39:14 WIB [10119] labuser@labdb LOG:  disconnection: session time: 0:00:00.003 user=labuser database=labdb host=[local]
postgres-db  | 2026-05-10 14:39:24 WIB [10126] [unknown]@[unknown] LOG:  connection received: host=[local]
postgres-db  | 2026-05-10 14:39:24 WIB [10126] labuser@labdb LOG:  connection authorized: user=labuser database=labdb application_name=pg_isready
postgres-db  | 2026-05-10 14:39:24 WIB [10126] labuser@labdb LOG:  disconnection: session time: 0:00:00.003 user=labuser database=labdb host=[local]
```

Langkah 2: Koneksi dan Verifikasi Database

2.1 Koneksi via psql dari host

Install psql client (jika belum)

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libpq5 postgresql-client-16 postgresql-client-common
Suggested packages:
  postgresql-16 postgresql-doc-16
The following NEW packages will be installed:
  libpq5 postgresql-client postgresql-client-16 postgresql-client-common
0 upgraded, 4 newly installed, 0 to remove and 16 not upgraded.
Need to get 1492 kB of archives.
After this operation, 4964 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libpq5 amd64 16.13-0ubuntu0.24.04.1 [146 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 postgresql-client-common all 257build1.1 [36.4 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 postgresql-client-16 amd64 16.13-0ubuntu0.24.04.1 [1298 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 postgresql-client all 16+257build1.1 [11.6 kB]
Fetched 1492 kB in 1s (1041 kB/s)
Selecting previously unselected package libpq5:amd64.
(Reading database ... 41325 files and directories currently installed.)
Preparing to unpack .../libpq5_16.13-0ubuntu0.24.04.1_amd64.deb ...
Unpacking libpq5:amd64 (16.13-0ubuntu0.24.04.1) ...
Selecting previously unselected package postgresql-client-common.
Preparing to unpack .../postgresql-client-common_257build1.1_all.deb ...
Unpacking postgresql-client-common (257build1.1) ...
Selecting previously unselected package postgresql-client-16.
Preparing to unpack .../postgresql-client-16_16.13-0ubuntu0.24.04.1_amd64.deb ...
Unpacking postgresql-client-16 (16.13-0ubuntu0.24.04.1) ...
Selecting previously unselected package postgresql-client.
Preparing to unpack .../postgresql-client_16+257build1.1_all.deb ...
Unpacking postgresql-client (16+257build1.1) ...
```

Koneksi ke database

```
hisyam@HISYAMpc:~/docker-lab/postgresql$ psql -h localhost -U labuser -d labdb
Password for user labuser:
psql (16.13 (Ubuntu 16.13-0ubuntu0.24.04.1))
Type "help" for help.

labdb=#
```

Di dalam psql:

-- list databases

```
labdb=# \l
                                                  List of databases
     Name     |  Owner  | Encoding | Locale Provider |  Collate   |   Ctype    | ICU Locale | ICU Rules |   Access privileges
--------------+---------+----------+-----------------+------------+------------+------------+-----------+-----------------------
 inventory_db | labuser | UTF8     | libc            | en_US.utf8 | en_US.utf8 |            |           |
 labdb        | labuser | UTF8     | libc            | en_US.utf8 | en_US.utf8 |            |           |
 postgres     | labuser | UTF8     | libc            | en_US.utf8 | en_US.utf8 |            |           |
 template0    | labuser | UTF8     | libc            | en_US.utf8 | en_US.utf8 |            |           | =c/labuser          +
              |         |          |                 |            |            |            |           | labuser=CTc/labuser
 template1    | labuser | UTF8     | libc            | en_US.utf8 | en_US.utf8 |            |           | =c/labuser          +
              |         |          |                 |            |            |            |           | labuser=CTc/labuser
(5 rows)
```

-- list schemas

```
labdb=# \dn
      List of schemas
  Name  |       Owner
--------+-------------------
 app    | labuser
 public | pg_database_owner
(2 rows)
```

-- list tabel di schema app

```
labdb=# \dt app.*
            List of relations
 Schema |     Name      | Type  |  Owner
--------+---------------+-------+---------
 app    | activity_log  | table | labuser
 app    | mahasiswa     | table | labuser
 app    | matakuliah    | table | labuser
 app    | nilai         | table | labuser
(4 rows)
```

-- describe tabel mahasiswa

```
                                                                Table "app.mahasiswa"
   Column   |           Type           | Collation | Nullable |                 Default                 | Storage  | Compression | Stats target | Descr
iption
------------+--------------------------+-----------+----------+-----------------------------------------+----------+-------------+--------------+------
-------
 id         | integer                  |           | not null | nextval('app.mahasiswa_id_seq'::regclass) | plain    |             |              |
 nrp        | character varying(15)    |           | not null |                                         | extended |             |              |
 nama       | character varying(100)   |           | not null |                                         | extended |             |              |
 kelas      | character(1)             |           |          |                                         | extended |             |              |
 kelompok   | integer                  |           |          |                                         | plain    |             |              |
 email      | character varying(100)   |           |          |                                         | extended |             |              |
 created_at | timestamp without time zone |        |          | CURRENT_TIMESTAMP                       | plain    |             |              |
Indexes:
    "mahasiswa_pkey" PRIMARY KEY, btree (id)
    "idx_mahasiswa_kelas" btree (kelas)
    "idx_mahasiswa_nrp" btree (nrp)
    "mahasiswa_nrp_key" UNIQUE CONSTRAINT, btree (nrp)
Check constraints:
    "mahasiswa_kelas_check" CHECK (kelas = ANY (ARRAY['A'::bpchar, 'B'::bpchar, 'C'::bpchar, 'D'::bpchar]))
    "mahasiswa_kelompok_check" CHECK (kelompok >= 1 AND kelompok <= 10)
Referenced by:
    TABLE "app.nilai" CONSTRAINT "nilai_mahasiswa_id_fkey" FOREIGN KEY (mahasiswa_id) REFERENCES app.mahasiswa(id) ON DELETE CASCADE
Access method: heap

~
~
~
```

```
labdb=# SELECT * FROM app.mahasiswa;
 id |    nrp     |     nama     | kelas | kelompok |         email          |         created_at
----+------------+--------------+-------+----------+------------------------+----------------------------
  1 | 3122600001 | Ahmad Fauzi  | A     |        1 | ahmad@student.pens.ac.id | 2026-05-10 14:39:04.238131
  2 | 3122600002 | Budi Santoso | A     |        1 | budi@student.pens.ac.id  | 2026-05-10 14:39:04.238131
  3 | 3122600003 | Citra Dewi   | B     |        2 | citra@student.pens.ac.id | 2026-05-10 14:39:04.238131
  4 | 3122600004 | Dian Pratama | B     |        2 | dian@student.pens.ac.id  | 2026-05-10 14:39:04.238131
  5 | 3122600005 | Eka Putra    | C     |        3 | eka@student.pens.ac.id   | 2026-05-10 14:39:04.238131
(5 rows)
```

```
labdb=# SELECT * FROM app.matakuliah;
 id | kode  |          nama          | sks
----+-------+------------------------+-----
  1 | JAR01 | Administrasi Jaringan  |   3
  2 | SBD01 | Sistem Basis Data      |   3
  3 | SO01  | Sistem Operasi         |   2
  4 | WEB01 | Pemrograman Web        |   3
(4 rows)
```

-- keluar

```
labdb=# \q
hisyam@HISYAMpc:~/docker-lab/postgresql$
```

Koneksi via psql dari host Menggunakan psql client yang diinstall di WSL untuk konek ke PostgreSQL yang berjalan di container via port 5432 yang di-expose ke host. Perintah \l, \dn, \dt adalah meta-command psql untuk eksplorasi struktur database.

2.2 Koneksi via docker exec

Masuk ke psql di dalam container

```
labdb=# \q
hisyam@HISYAMpc:~/docker-lab/postgresql$ docker exec -it postgres-db psql -U labuser -d labdb
psql (16.13)
Type "help" for help.

labdb=#
```

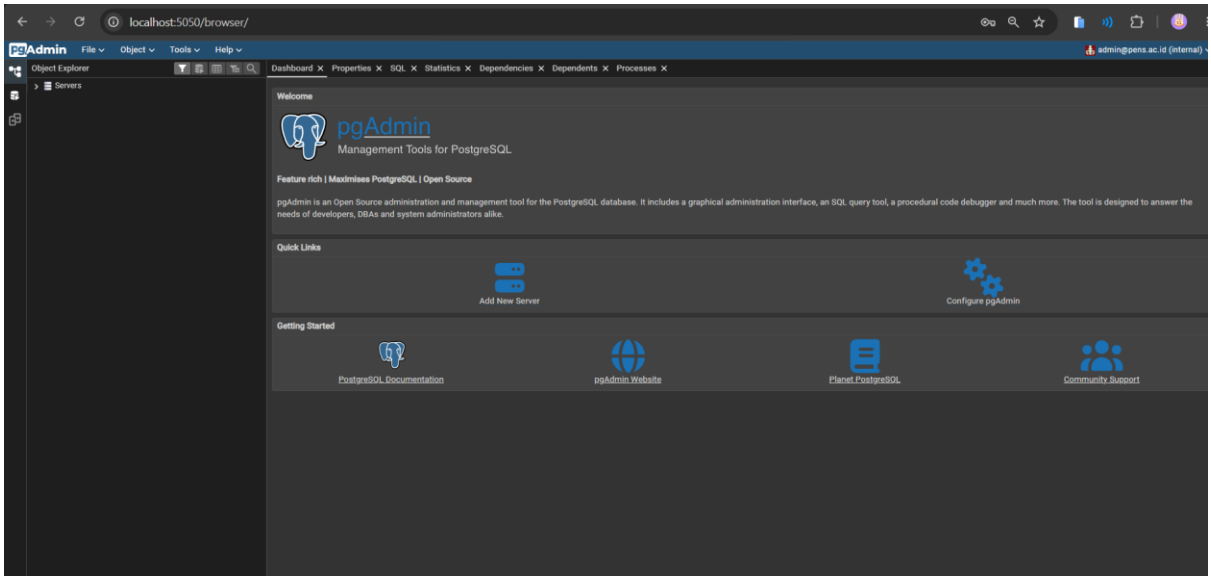
Query join: Nilai mahasiswa

```
labdb=# SELECT m.nrp, m.nama, mk.nama AS matakuliah, n.nilai_angka, n.grade
labdb-# kuliah mk ON n.matakuliah_id = mFROM app.nilai n
labdb-# JOIN app.mahasiswa m ON n.mahasiswa_id = m.id
labdb-# JOIN app.matakuliah mk ON n.matakuliah_id = mk.id
labdb-# ORDER BY m.nrp, mk.nama;
    nrp     |     nama      |      matakuliah       | nilai_angka | grade
------------+---------------+-----------------------+-------------+-------
 3122600001 | Ahmad Fauzi   | Administrasi Jaringan |       85.50 | A
 3122600001 | Ahmad Fauzi   | Sistem Basis Data     |       78.00 | B+
 3122600002 | Budi Santoso  | Administrasi Jaringan |       92.00 | A
 3122600003 | Citra Dewi    | Administrasi Jaringan |       70.25 | B
 3122600004 | Dian Pratama  | Sistem Operasi        |       88.75 | A
(5 rows)

labdb=# \q
hisyam@HISYAMpc:~/docker-lab/postgresql$
```

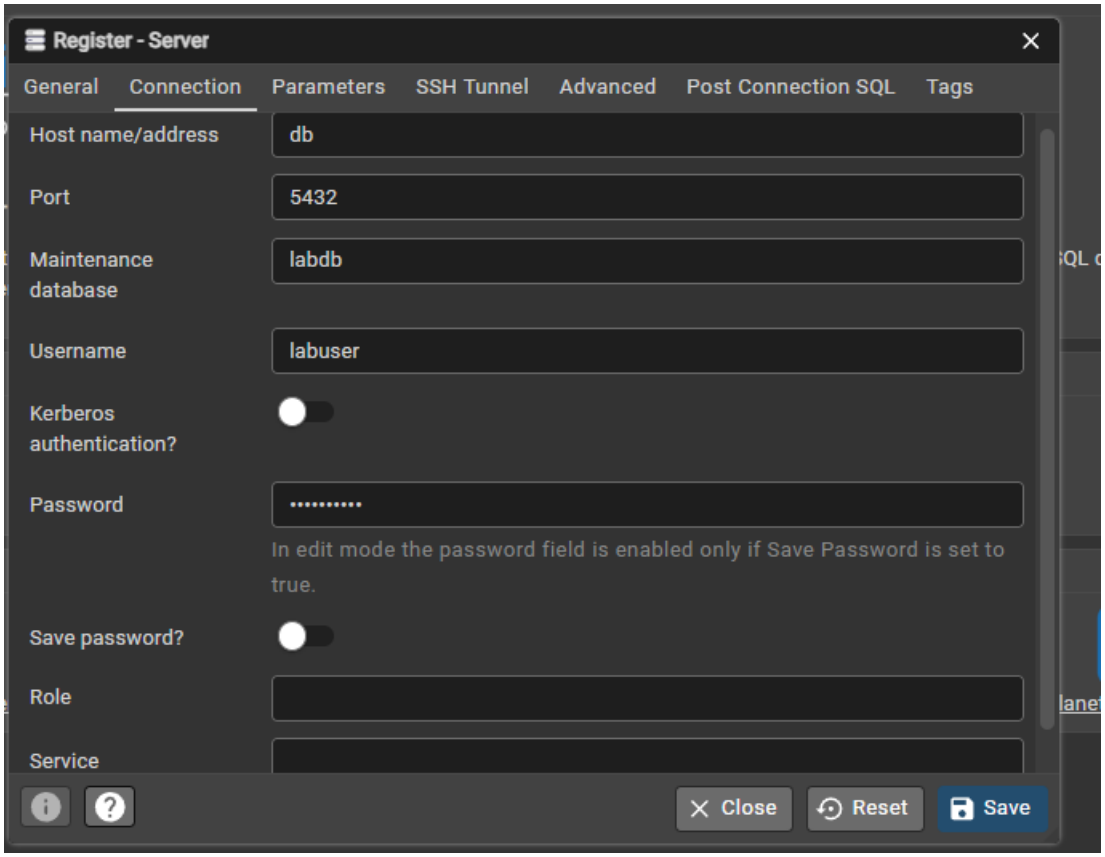
Koneksi via docker exec Cara alternatif masuk ke psql langsung dari dalam container tanpa perlu install psql di host. Query JOIN digunakan untuk menampilkan data nilai mahasiswa lengkap dengan nama dan mata kuliah.

2.3 Koneksi via pgAdmin4

Buka browser:

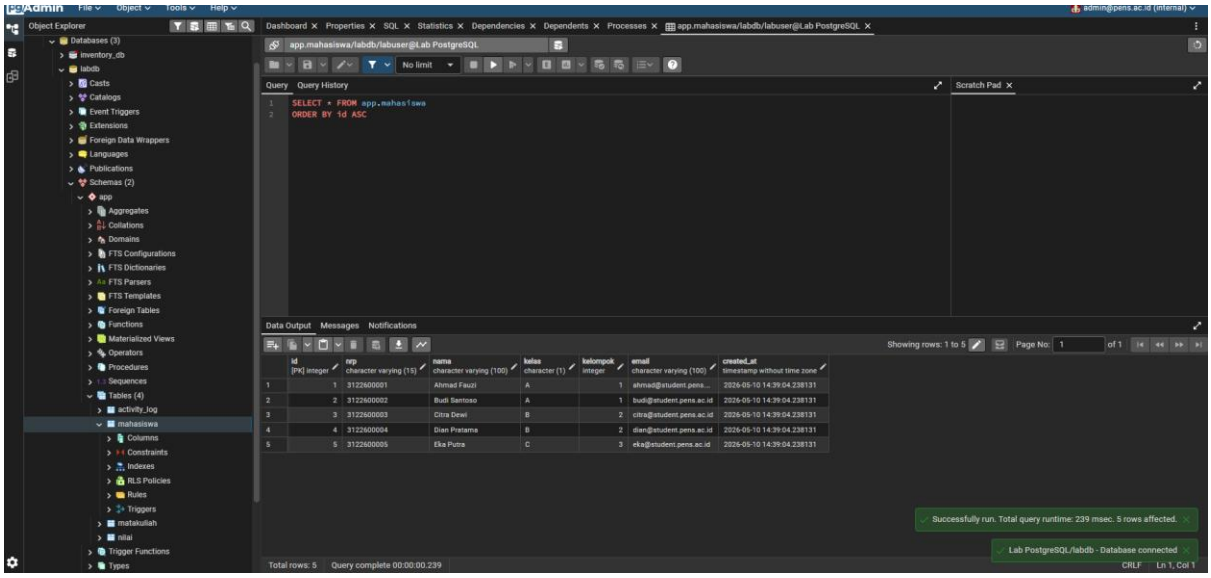


Add New Server:



Navigate: Servers → Lab PostgreSQL → Databases → labdb → Schemas → app → Tables

Klik kanan tabel mahasiswa → View/Edit Data → All Rows



Koneksi via pgAdmin4 pgAdmin adalah GUI berbasis web untuk mengelola PostgreSQL. Diakses via browser di http://localhost:5050. Saat menambah server, hostname menggunakan nama service Docker (db) bukan localhost karena pgAdmin dan PostgreSQL berada dalam satu Docker network db-net.

Langkah 3: Operasi CRUD SQL

```
hisyam@HISYAMpc:~/docker-lab/postgresql$ docker exec postgres-db psql -U labuser -d labdb << 'SQLEOF'

-- === CREATE ===
INSERT INTO app.mahasiswa (nrp, nama, kelas, kelompok, email)
VALUES ('3122600010', 'Fajar Rizki', 'D', 5, 'fajar@student.pens.ac.id');

-- === READ ===
SELECT * FROM app.mahasiswa WHERE kelas = 'A';

SELECT mk.nama, AVG(n.nilai_angka)::NUMERIC(5,2) AS rata_rata, COUNT(*) AS jumlah
FROM app.nilai n
JOIN app.matakuliah mk ON n.matakuliah_id = mk.id
GROUP BY mk.nama
ORDER BY rata_rata DESC;

-- === UPDATE ===
UPDATE app.mahasiswa SET email = 'fajar.rizki@student.pens.ac.id'
WHERE nrp = '3122600010';

-- === DELETE ===
DELETE FROM app.mahasiswa WHERE nrp = '3122600010';

-- === JSONB ===
INSERT INTO app.activity_log (level, source, message, metadata)
VALUES ('INFO', 'web-app', 'User login', '{"user": "admin", "ip": "192.168.1.10"}');

SELECT * FROM app.activity_log
WHERE metadata->>'user' = 'admin';

SQLEOF
```

Menjalankan operasi dasar database:

- **CREATE** — INSERT mahasiswa baru dengan data lengkap
- **READ** — SELECT dengan filter WHERE dan query agregasi AVG + GROUP BY untuk menghitung rata-rata nilai per mata kuliah
- **UPDATE** — mengubah email mahasiswa berdasarkan NRP
- **DELETE** — menghapus data mahasiswa
- **JSONB query** — INSERT data ke activity_log dengan metadata JSON, lalu SELECT menggunakan operator ->> untuk mengakses field di dalam JSON

Langkah 4: Backup dan Restore

4.1 Backup database (pg_dump)

Backup dalam format custom (compressed, restorable)

```
hisyam@HISYAMpc:~/docker-lab/postgresql$ docker exec postgres-db pg_dump -U labuser -d labdb -Fc \
    -f /backup/labdb_backup.dump
```

Backup dalam format SQL plain text

```
hisyam@HISYAMpc:~/docker-lab/postgresql$ docker exec postgres-db pg_dump -U labuser -d labdb \
    -f /backup/labdb_backup.sql
```

Backup hanya schema app

```
hisyam@HISYAMpc:~/docker-lab/postgresql$ docker exec postgres-db pg_dump -U labuser -d labdb -n app -Fc \
    -f /backup/labdb_schema_app.dump
```

Verifikasi file backup di host

```
hisyam@HISYAMpc:~/docker-lab/postgresql$ ls -la backup/
total 52
drwxr-xr-x 2 hisyam hisyam  4096 May 10 16:19 .
drwxr-xr-x 5 hisyam hisyam  4096 May 10 14:43 ..
-rw-r--r-- 1 root   root   14131 May 10 16:18 labdb_backup.dump
-rw-r--r-- 1 root   root   10364 May 10 16:19 labdb_backup.sql
-rw-r--r-- 1 root   root   14131 May 10 16:19 labdb_schema_app.dump
hisyam@HISYAMpc:~/docker-lab/postgresql$
```

Backup

- Format custom (-Fc) — format binary terkompresi, hanya bisa di-restore dengan pg_restore, ukuran lebih kecil
- Format SQL plain text — output berupa SQL statement yang bisa dibaca dan dijalankan manual
- Backup per-schema (-n app) — hanya backup schema tertentu, berguna jika ingin backup parsial

4.2 Restore database

Buat database baru untuk restore test

```
hisyam@HISYAMpc:~/docker-lab/postgresql$ docker exec postgres-db psql -U labuser -d postgres -c "CREATE DATABASE labdb_restore;"
CREATE DATABASE
```

Restore dari custom format

```
hisyam@HISYAMpc:~/docker-lab/postgresql$ docker exec postgres-db pg_restore -U labuser -d labdb_restore \
    /backup/labdb_backup.dump
```

Verifikasi restore

```
hisyam@HISYAMpc:~/docker-lab/postgresql$ docker exec postgres-db psql -U labuser -d labdb_restore -c "SELECT * FROM app.mahasiswa;"
 id |    nrp     |     nama     | kelas | kelompok |          email          |         created_at
----+------------+--------------+-------+----------+-------------------------+----------------------------
  1 | 3122600001 | Ahmad Fauzi  | A     |        1 | ahmad@student.pens.ac.id | 2026-05-10 14:39:04.238131
  2 | 3122600002 | Budi Santoso | A     |        1 | budi@student.pens.ac.id  | 2026-05-10 14:39:04.238131
  3 | 3122600003 | Citra Dewi   | B     |        2 | citra@student.pens.ac.id | 2026-05-10 14:39:04.238131
  4 | 3122600004 | Dian Pratama | B     |        2 | dian@student.pens.ac.id  | 2026-05-10 14:39:04.238131
  5 | 3122600005 | Eka Putra    | C     |        3 | eka@student.pens.ac.id   | 2026-05-10 14:39:04.238131
(5 rows)
```

Restore Membuat database baru labdb_restore lalu restore dari file backup custom format menggunakan pg_restore. Verifikasi dengan SELECT memastikan data berhasil di-restore.

4.3 Backup otomatis dengan cron di container

Buat script backup

```
hisyam@HISYAMpc:~/docker-lab/postgresql$ cat > backup/auto-backup.sh << 'BASH'
#!/bin/sh
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="/backup/labdb_${TIMESTAMP}.dump"
pg_dump -U labuser -d labdb -Fc -f "$BACKUP_FILE"
echo "[$(date)] Backup created: $BACKUP_FILE"

# Hapus backup lebih dari 7 hari
find /backup -name "labdb_*.dump" -mtime +7 -delete
BASH
hisyam@HISYAMpc:~/docker-lab/postgresql$ chmod +x backup/auto-backup.sh
```

Test jalankan manual

```
hisyam@HISYAMpc:~/docker-lab/postgresql$ docker exec postgres-db /backup/auto-backup.sh
[Sun May 10 16:21:48 WIB 2026] Backup created: /backup/labdb_20260510_162147.dump
hisyam@HISYAMpc:~/docker-lab/postgresql$ ls -la backup/
total 72
drwxr-xr-x 2 hisyam hisyam  4096 May 10 16:21 .
drwxr-xr-x 5 hisyam hisyam  4096 May 10 14:43 ..
-rwxr-xr-x 1 hisyam hisyam   271 May 10 16:21 auto-backup.sh
-rw-r--r-- 1 root   root   14131 May 10 16:21 labdb_20260510_162147.dump
-rw-r--r-- 1 root   root   14131 May 10 16:18 labdb_backup.dump
-rw-r--r-- 1 root   root   10364 May 10 16:19 labdb_backup.sql
-rw-r--r-- 1 root   root   14131 May 10 16:19 labdb_schema_app.dump
```

Backup Otomatis Script shell yang dibuat di folder backup/ dan dijalankan di dalam container. Script menambahkan timestamp ke nama file backup dan otomatis menghapus backup yang lebih dari 7 hari menggunakan find -mtime +7.

Langkah 5: Monitoring PostgreSQL

5.1 Statistik database

```
hisyam@HISYAMpc:~/docker-lab/postgresql$ docker exec  postgres-db psql -U labuser -d labdb << 'SQLEOF'

-- Ukuran database
SELECT pg_database.datname,
       pg_size_pretty(pg_database_size(pg_database.datname)) AS size
FROM pg_database ORDER BY pg_database_size(pg_database.datname) DESC;

-- Ukuran per tabel
SELECT schemaname || '.' || tablename AS table_full,
       pg_size_pretty(pg_total_relation_size(schemaname || '.' || tablename)) AS total_size
FROM pg_tables WHERE schemaname = 'app'
ORDER BY pg_total_relation_size(schemaname || '.' || tablename) DESC;

-- Koneksi aktif
SELECT pid, usename, datname, client_addr, state, query_start, query
FROM pg_stat_activity WHERE datname = 'labdb';

-- Statistik tabel (hits, reads, cache ratio)
SELECT relname,
       seq_scan, seq_tup_read,
       idx_scan, idx_tup_fetch,
       n_tup_ins, n_tup_upd, n_tup_del
FROM pg_stat_user_tables WHERE schemaname = 'app';

SQLEOF
```

Statistik database Query ke tabel sistem PostgreSQL (pg_database, pg_tables, pg_stat_activity, pg_stat_user_tables) untuk monitoring:

- Ukuran database dan per-tabel dalam format human-readable
- Koneksi aktif beserta query yang sedang berjalan
- Statistik akses tabel: berapa kali sequential scan vs index scan digunakan

5.2 Cek PostgreSQL log

Lihat log PostgreSQL

```
hisyam@HISYAMpc:~/docker-lab/postgresql$ docker exec postgres-db ls -la /var/log/postgresql/
total 8
drwxr-xr-x    2 postgres postgres      4096 May 10 16:34 .
drwxr-xr-x    1 root     root          4096 May 10 16:40 ..
```

```
hisyam@HISYAMpc:~/docker-lab/postgresql$ docker exec postgres-db cat /var/log/postgresql/postgresql-$(date +%Y-%m-%d).log | tail -20
2026-05-10 16:45:26 WIB [1] @ LOG:  starting PostgreSQL 16.13 on x86_64-pc-linux-musl, compiled by gcc (Alpine 15.2.0) 15.2.0, 64-bit
2026-05-10 16:45:26 WIB [1] @ LOG:  listening on IPv4 address "0.0.0.0", port 5432
2026-05-10 16:45:26 WIB [1] @ LOG:  listening on IPv6 address "::", port 5432
2026-05-10 16:45:26 WIB [1] @ LOG:  listening on Unix socket "/var/run/postgresql/.s.PGSQL.5432"
2026-05-10 16:45:26 WIB [34] @ LOG:  database system was shut down at 2026-05-10 16:45:26 WIB
2026-05-10 16:45:26 WIB [1] @ LOG:  database system is ready to accept connections
2026-05-10 16:45:37 WIB [44] [unknown]@[unknown] LOG:  connection received: host=[local]
2026-05-10 16:45:37 WIB [44] labuser@labdb LOG:  connection authorized: user=labuser database=labdb application_name=psql
2026-05-10 16:45:37 WIB [44] labuser@labdb LOG:  disconnection: session time: 0:00:00.019 user=labuser database=labdb host=[local]
2026-05-10 16:45:40 WIB [51] [unknown]@[unknown] LOG:  connection received: host=[local]
2026-05-10 16:45:40 WIB [51] labuser@labdb LOG:  connection authorized: user=labuser database=labdb application_name=pg_isready
2026-05-10 16:45:40 WIB [51] labuser@labdb LOG:  disconnection: session time: 0:00:00.004 user=labuser database=labdb host=[local]
```

Cek log Log PostgreSQL disimpan di /var/log/postgresql/ di dalam container, dengan nama file berdasarkan tanggal sesuai konfigurasi log_filename.

5. PERTANYAAN

Pre-Lab

1. Apa fungsi file/folder /docker-entrypoint-initdb.d/ di image PostgreSQL?

   Fungsi /docker-entrypoint-initdb.d/: Folder khusus di image PostgreSQL yang berisi script SQL atau shell yang dijalankan **otomatis** saat container pertama kali start dan volume data masih kosong. Script diurutkan berdasarkan nama file (alfabetis), sehingga penamaan 01-create-schema.sql memastikan urutan eksekusi yang benar. Script tidak dijalankan ulang jika volume sudah berisi data.

2. Mengapa POSTGRES_PASSWORD wajib diset? Apa risikonya jika tidak ada password?

   Mengapa POSTGRES_PASSWORD wajib: PostgreSQL memerlukan autentikasi untuk keamanan. Tanpa password, siapapun yang bisa reach container bisa akses database tanpa autentikasi. Risikonya adalah data bisa dibaca, dimodifikasi, atau dihapus oleh pihak yang tidak berwenang. Image PostgreSQL akan menolak start jika tidak ada password yang diset.

3. Jelaskan perbedaan antara pg_dump format custom (-Fc) dan format SQL plain text.

   Perbedaan format custom vs SQL plain text: Format custom (-Fc) menghasilkan file binary terkompresi yang lebih kecil, mendukung restore paralel, dan bisa di-restore secara selektif (per tabel). Format SQL plain text menghasilkan file berisi SQL statement yang bisa dibaca manusia dan dieksekusi langsung, tapi ukurannya lebih besar karena tidak terkompresi.

4. Apa itu shared_buffers dan mengapa perlu disesuaikan untuk container?

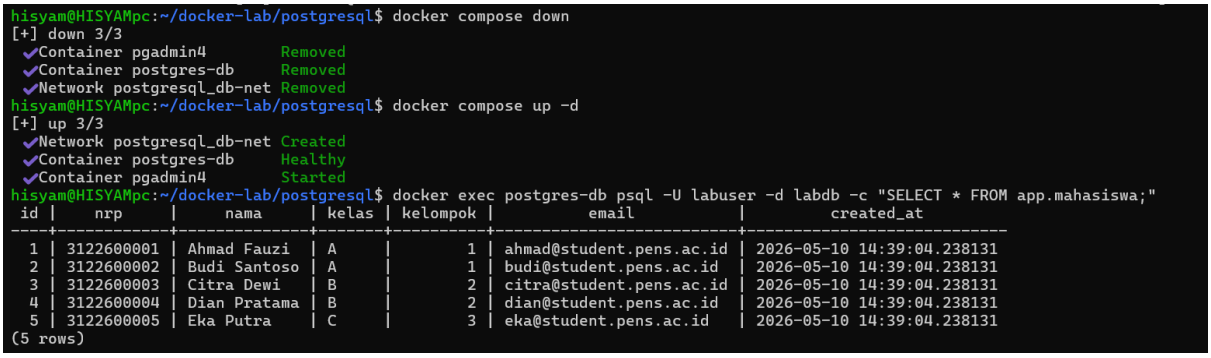
   shared_buffers dan penyesuaian untuk container: shared_buffers adalah alokasi memory untuk cache data PostgreSQL di RAM. Semakin besar nilainya, semakin banyak data yang bisa di-cache sehingga query lebih cepat. Untuk container perlu disesuaikan agar tidak melebihi memory yang dialokasikan ke container — jika terlalu besar container bisa OOM (Out of Memory) dan crash.

5. Mengapa data PostgreSQL harus disimpan di Docker Volume, bukan di container layer?

   Mengapa data harus di Docker Volume: Container layer bersifat ephemeral — semua data hilang saat container dihapus. Volume menyimpan data di luar container sehingga data persist meski container dihapus, di-recreate, atau di-update ke versi baru. Untuk database ini sangat kritis karena kehilangan data database berarti kehilangan seluruh informasi yang tersimpan.

Post-Lab

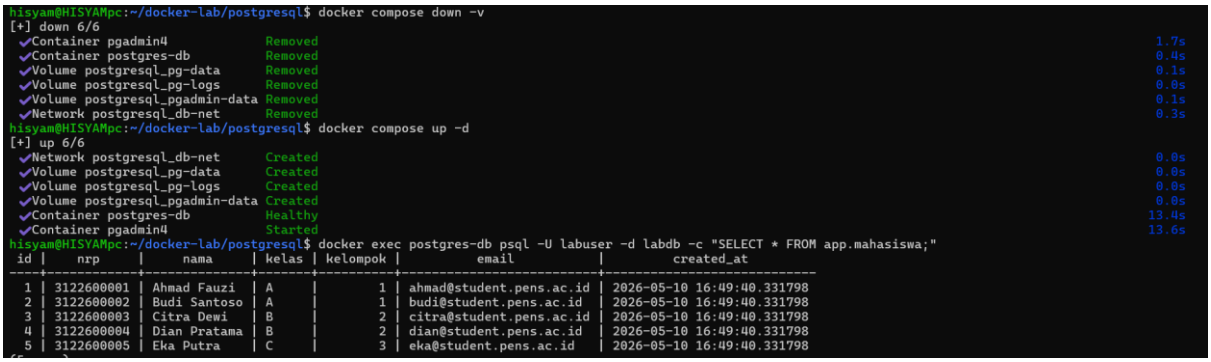
1. Jalankan docker compose down lalu docker compose up -d. Apakah data mahasiswa masih ada? Buktikan.



   Setelah docker compose down lalu docker compose up -d, semua 5 data mahasiswa masih ada. Ini membuktikan bahwa docker compose down

hanya menghapus container dan network, **tidak menghapus volume** pg-data. Data PostgreSQL tetap tersimpan di volume dan di-mount kembali saat container baru dibuat.

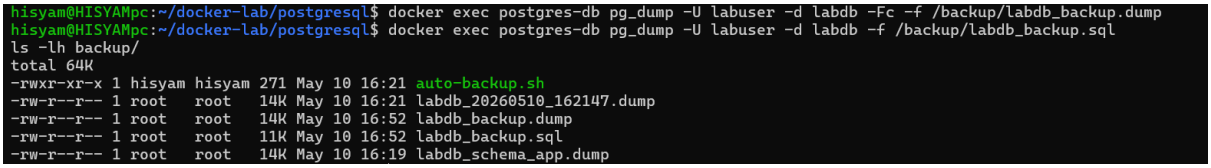
2. Jalankan docker compose down -v lalu docker compose up -d. Apa yang terjadi? Apakah init script dijalankan ulang?



Setelah docker compose down -v semua volume dihapus (pg-data, pg-logs, pgadmin-data). Saat docker compose up -d dijalankan, volume baru dibuat kosong sehingga PostgreSQL mendeteksi instalasi baru dan **init script 01-create-schema.sql dijalankan ulang otomatis**.

Terbukti dari created_at data mahasiswa berubah dari 14:39 menjadi 16:49 — artinya data lama hilang dan diganti data sample baru dari init script.

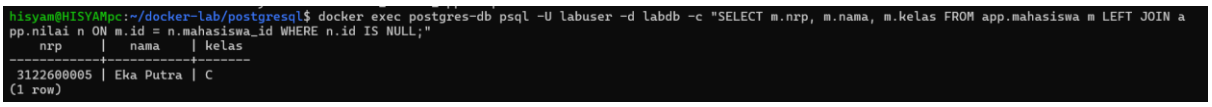
3. Bandingkan ukuran file backup format custom vs SQL. Mana yang lebih kecil dan mengapa?



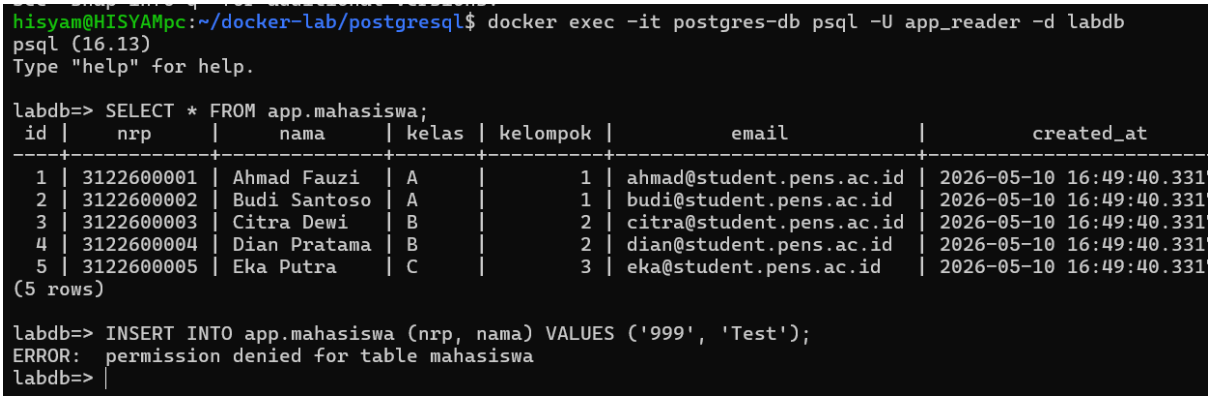
di lab ini ukuran SQL plain text (11K) justru **lebih kecil** dari custom format (14K) karena data yang ada masih sangat sedikit (hanya 5 mahasiswa, 4 matakuliah).

Pada data kecil, overhead header binary format custom lebih besar dari keuntungan kompresinya. Namun pada **data yang besar**, format custom akan jauh lebih kecil karena kompresi lebih efektif. Format custom juga mendukung restore paralel dan selektif per tabel.

4. Buat query yang menampilkan mahasiswa yang belum memiliki nilai di semester apapun.



5. Jelaskan peran user app_reader yang dibuat di init script. Apa bedanya dengan labuser?



app_reader dibuat di init script dengan perintah:

CREATE USER app_reader WITH PASSWORD 'reader123';

GRANT USAGE ON SCHEMA app TO app_reader;

GRANT SELECT ON ALL TABLES IN SCHEMA app TO app_reader;

Perbedaan app_reader vs labuser:

Aspek	**labuser**	**app_reader**
Role	Owner/superuser database	Read-only user
Hak akses	CREATE, INSERT, UPDATE, DELETE, DROP	SELECT saja
Bisa modifikasi data	Ya	Tidak
Bisa buat tabel/schema	Ya	Tidak
Password	labpass123	reader123

Aspek	labuser	app_reader
Penggunaan	Admin, migration, operasi penuh	Aplikasi yang hanya butuh baca data
Risiko jika credential bocor	Sangat tinggi	Rendah

app_reader dibuat mengikuti prinsip **least privilege** - setiap user/aplikasi hanya diberi akses minimal yang dibutuhkan. Jika credential app_reader bocor, attacker hanya bisa membaca data tanpa bisa memodifikasi atau menghapusnya.

MODUL 5: Logging Service Docker dengan PostgreSQL

LANGKAH PRAKTIKUM

Langkah 0: Persiapan Project

```
hisyam@HISYAMpc:~$ mkdir -p ~/docker-lab/logging/{fluent-bit,app,generator,init}
cd ~/docker-lab/logging
```

Perintah mkdir -p dengan kurung kurawal membuat 4 subfolder sekaligus dalam satu baris. Masing-masing folder punya fungsi spesifik:

- fluent-bit/ → file konfigurasi Fluent Bit (kolektor log)
- app/ → kode Flask API beserta Dockerfile-nya
- generator/ → script Python simulator log beserta Dockerfile-nya
- init/ → SQL schema yang dijalankan otomatis saat PostgreSQL pertama kali hidup

Langkah 1: Buat Database Schema untuk Logging

```
hisyam@HISYAMpc:~/docker-lab/logging$ cat > init/01-logging-schema.sql << 'EOF'
CREATE SCHEMA IF NOT EXISTS logs;

CREATE TABLE logs.fluentbit (
    id       BIGSERIAL PRIMARY KEY,
    tag      VARCHAR(200),
    time     TIMESTAMP,
    data     JSONB
);

CREATE INDEX idx_fb_time ON logs.fluentbit(time);
CREATE INDEX idx_fb_tag  ON logs.fluentbit(tag);
CREATE INDEX idx_fb_data ON logs.fluentbit USING GIN(data);

CREATE OR REPLACE VIEW logs.recent_logs AS
SELECT
    id,
    to_char(time, 'YYYY-MM-DD HH24:MI:SS') AS time,
    tag,
    REPLACE(data->>'container_name', '/', '') AS container,
    data->>'source' AS source,
    LEFT(data->>'log', 200) AS log_preview
FROM logs.fluentbit
ORDER BY time DESC LIMIT 100;

CREATE OR REPLACE VIEW logs.structured_logs AS
SELECT
    id,
    time AS received_at,
    tag,
EOFLANGUAGE plpgsql;ount;ed_count = ROW_COUNT;W() - INTERVAL '30 days';
```

Perintah cat > init/01-logging-schema.sql << 'EOF' menulis isi SQL langsung ke file tanpa perlu membuka text editor. File ini di-mount ke folder /docker-entrypoint-initdb.d/ di dalam container PostgreSQL — semua file .sql di folder tersebut dijalankan otomatis saat database pertama kali dibuat.

CREATE SCHEMA IF NOT EXISTS logs — membuat namespace logs agar tabel log tidak bercampur dengan tabel lain di database labdb. Akses tabelnya jadi logs.container_logs, bukan sekadar container_logs.

Tabel logs.container_logs adalah tempat semua log masuk. Kolom-kolomnya:

id → BIGSERIAL, auto-increment, primary key

received_at → kapan Fluent Bit INSERT ke database (waktu server)

timestamp → kapan log dibuat di dalam container (waktu aplikasi)

container_name → nama container asal log (nginx-web, flask-app, dll)

container_id → ID unik container Docker

source → stdout atau stderr

log_level → INFO / DEBUG / WARN / ERROR / CRITICAL

message → isi pesan log

raw_log → log mentah sebelum di-parse

metadata → data tambahan format JSONB (fleksibel, bisa beda tiap log)

Dua kolom timestamp sengaja dipisah karena waktu log dibuat di container bisa berbeda dengan waktu Fluent Bit menerimanya, terutama jika ada network delay atau buffering.

Tabel logs.hourly_summary menyimpan ringkasan per jam per container. Berguna untuk dashboard atau laporan tanpa perlu query tabel besar setiap saat. Constraint UNIQUE(hour, container_name) mencegah duplikasi data summary untuk kombinasi jam dan container yang sama.

Index dibuat di kolom yang sering dipakai sebagai filter:

```
CREATE INDEX idx_logs_timestamp ON logs.container_logs(timestamp);

CREATE INDEX idx_logs_container ON
logs.container_logs(container_name);

CREATE INDEX idx_logs_level    ON logs.container_logs(log_level);

CREATE INDEX idx_logs_received ON logs.container_logs(received_at);

CREATE INDEX idx_logs_metadata ON logs.container_logs USING
GIN(metadata);
```

Tanpa index, query WHERE log_level = 'ERROR' harus membaca seluruh tabel satu per satu (full table scan). Dengan index, PostgreSQL langsung lompat ke

baris yang relevan. Index GIN khusus untuk kolom bertipe JSONB agar pencarian di dalam field metadata tetap cepat.

Fungsi logs.cleanup_old_logs() ditulis dalam PL/pgSQL — bahasa prosedural bawaan PostgreSQL. Fungsi ini menghapus semua log yang sudah lebih dari 30 hari dan mengembalikan jumlah baris yang dihapus. Ini adalah implementasi sederhana dari log retention policy.

View logs.recent_logs adalah query yang disimpan sebagai objek di database. Setiap kali dipanggil dengan SELECT * FROM logs.recent_logs, PostgreSQL menjalankan query di baliknya secara otomatis. View ini menampilkan 100 log terbaru dengan format timestamp yang lebih mudah dibaca (to_char) dan message yang dipotong 200 karakter pertama (LEFT).

View logs.error_summary merangkum jumlah error per container dan per level, berguna untuk melihat sekilas container mana yang paling banyak menghasilkan error.

Langkah 2: Konfigurasi Fluent Bit

```
hisyam@HISYAMpc:~/docker-lab/logging$ cat > fluent-bit/fluent-bit.conf << 'EOF
[SERVICE]
    Flush         5
    Daemon        Off
    Log_Level     info
    Parsers_File parsers.conf

[INPUT]
    Name          forward
    Listen        0.0.0.0
    Port          24224

[OUTPUT]
    Name           pgsql
    Match          *
    Host           postgres-db
    Port           5432
    User           labuser
    Password       labpass123
    Database       labdb
    Table          fluentbit
    Schema         logs
    Timestamp_Key time
    Async          false
    min_pool_size 1
    max_pool_size 4

[OUTPUT]
    Name           stdout
    Match          *
EOF Format         json_lines
```

Keterangan: Menyusun file konfigurasi fluent-bit.conf dan parsers.conf untuk mengatur pipeline logging. Menggunakan komponen [INPUT] berupa driver forward pada port 24224 untuk menangkap log dari Docker Daemon, serta komponen [OUTPUT] berupa plugin pgsql untuk mengirimkan data hasil tangkapan tersebut secara terpusat ke database PostgreSQL.

Langkah 3: Buat Log Generator (Simulasi Multi-Level Log)

```
hisyam@HISYAMpc:~/docker-lab/logging/generator$ cat generator.py
"""
Log Generator - mensimulasikan log dari aplikasi production.
Menghasilkan log dengan berbagai level: DEBUG, INFO, WARN, ERROR, CRITICAL.
"""
import json, time, random, socket, datetime, sys, os

HOSTNAME = socket.gethostname()
LOG_INTERVAL = float(os.environ.get("LOG_INTERVAL", "2"))

# Simulasi event dengan bobot probabilitas
EVENTS = [
    {"level": "INFO",     "weight": 50, "messages": [
        "User login successful",
        "Page /dashboard loaded in {ms}ms",
        "API request GET /api/users completed",
        "Session created for user_{uid}",
        "Cache hit for key: product_{pid}",
        "Health check passed",
        "Background job completed: email_send"
    ]},
    {"level": "DEBUG",    "weight": 20, "messages": [
        "Database query executed in {ms}ms",
        "Redis connection pool: {pool} active",
        "Request headers: content-type=application/json",
        "Middleware chain completed in {ms}ms"
    ]},
    {"level": "WARN",     "weight": 15, "messages": [
        "Slow query detected: {ms}ms (threshold: 1000ms)",
        "Memory usage at {mem}% - approaching limit",
        "Rate limit approaching for IP 192.168.{ip}.{host}",
        "Deprecated API endpoint called: /api/v1/legacy",
        "Certificate expires in {days} days"
    ]},
    {"level": "ERROR",    "weight": 10, "messages": [
        "Failed to connect to database: timeout after 5s",
        "NullPointerException in UserService.getProfile()",
        "HTTP 500 Internal Server Error on /api/checkout",
        "Disk write failed: /var/log/app.log - Permission denied",
```

Langkah 3: Buat Log Generator (Simulasi Multi-Level Log)

Keterangan: Membuat skrip Python (generator.py) berbasis Docker Image python:3.11-alpine yang berfungsi mensimulasikan aktivitas log aplikasi produksi. Skrip ini secara otomatis mencetak log terstruktur berformat JSON ke *stdout* dengan berbagai tingkatan keparahan (*severity level*) secara acak, mulai dari INFO, DEBUG, WARN, ERROR, hingga CRITICAL.

Langkah 4: Buat Flask App dengan Structured Logging

```
hisyam@HISYAMpc:~/docker-lab/logging/app$ cat requirements.txt
flask==3.1.*
psycopg2-binary==2.9.*
hisyam@HISYAMpc:~/docker-lab/logging/app$ cat app.py
import os, json, socket, datetime, logging, sys
from flask import Flask, jsonify, request
import psycopg2

app = Flask(__name__)

class JSONFormatter(logging.Formatter):
    def format(self, record):
        return json.dumps({
            "timestamp": datetime.datetime.now().isoformat(),
            "level": record.levelname,
            "hostname": socket.gethostname(),
            "service": "flask-app",
            "message": record.getMessage(),
            "module": record.module
        })

handler = logging.StreamHandler(sys.stdout)
handler.setFormatter(JSONFormatter())
app.logger.handlers = [handler]
app.logger.setLevel(logging.INFO)
logging.getLogger("werkzeug").setLevel(logging.WARNING)

DB = dict(
    host=os.environ.get("DB_HOST", "postgres-db"),
    dbname=os.environ.get("DB_NAME", "labdb"),
```

Keterangan: Mengembangkan aplikasi web berbasis Flask yang memiliki fungsi ganda: sebagai penghasil log (*log producer*) berformat JSON ke *stdout*, dan sebagai media pemantau (*log viewer*). Flask menyediakan *endpoint* REST

API (/api/logs/stats dan /api/logs/search) yang melakukan query dinamis ke PostgreSQL untuk kebutuhan analisis statistik dan pencarian log.

Langkah 5: Docker Compose untuk Stack Logging

```
hisyam@HISYAMpc:~/docker-lab/logging$ cat > docker-compose.yml << 'EOF'
services:
  # === Log Collector: Fluent Bit ===
  fluent-bit:
    image: fluent/fluent-bit:latest
    container_name: fluent-bit
    ports:
      - "24224:24224"
      - "24224:24224/udp"
    volumes:
      - ./fluent-bit/fluent-bit.conf:/fluent-bit/etc/fluent-bit.conf:ro
      - ./fluent-bit/parsers.conf:/fluent-bit/etc/parsers.conf:ro
    networks:
      - logging-net
    depends_on:
      postgres-db:
        condition: service_healthy
    restart: unless-stopped
```

Keterangan: Menyusun seluruh arsitektur layanan (postgres-db, fluent-bit, nginx-web, flask-app, dan log-generator) ke dalam satu berkas docker-compose.yml. Mengatur urutan *startup* container menggunakan parameter depends_on berbasis service_healthy, serta mengalihkan semua *logging driver* container dari json-file ke fluentd dengan alamat tujuan fluent-bit:24224.

Urutan startup yang benar:

postgres-db (healthcheck pass)

↓

fluent-bit (tunggu postgres healthy)

↓

nginx-web, flask-app, log-generator (tunggu fluent-bit ready)

Konfigurasi depends_on dengan condition: service_healthy memastikan PostgreSQL benar-benar siap menerima koneksi sebelum Fluent Bit start. Jika

Fluent Bit naik sebelum PostgreSQL siap, plugin pgsql akan gagal konek dan log tidak tersimpan.

Logging driver di setiap producer:

yaml

```
logging:
  driver: fluentd
  options:
    fluentd-address: "localhost:24224"
    fluentd-async: "true"
    tag: "docker.nginx"
```

fluentd-address: "localhost:24224" artinya Docker daemon di host mengirim log ke port 24224 di host itu sendiri (bukan di dalam Docker network). Ini benar karena Fluent Bit expose port-nya ke host dengan ports: "24224:24224".

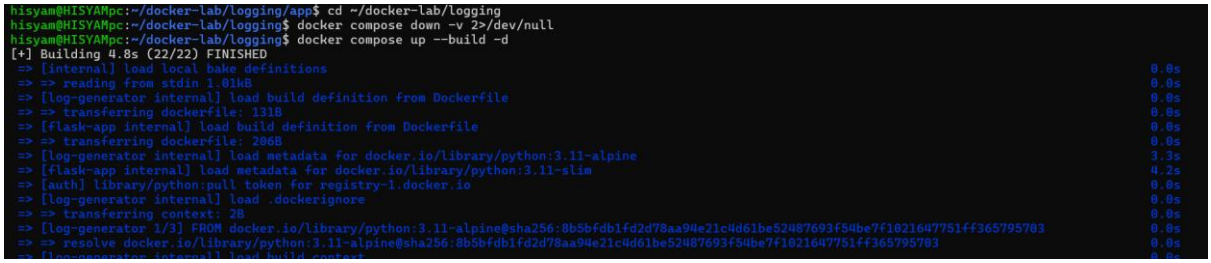
fluentd-async: "true" berarti jika Fluent Bit tidak tersedia, Docker tidak memblokir container — log di-buffer dulu. Tanpa ini, container bisa gagal start jika Fluent Bit belum ready.

Volume pg-data adalah named volume yang menyimpan data PostgreSQL secara persisten. Berbeda dengan bind mount (./init:/docker-entrypoint-initdb.d), named volume dikelola sepenuhnya oleh Docker dan tidak terhapus saat docker compose down (hanya hilang jika docker compose down --volumes).

Network logging-net adalah Docker bridge network khusus untuk stack ini. Semua container dalam network yang sama bisa saling berkomunikasi menggunakan nama container sebagai hostname — itulah kenapa Fluent Bit bisa konek ke postgres-db:5432 tanpa perlu tahu IP-nya.

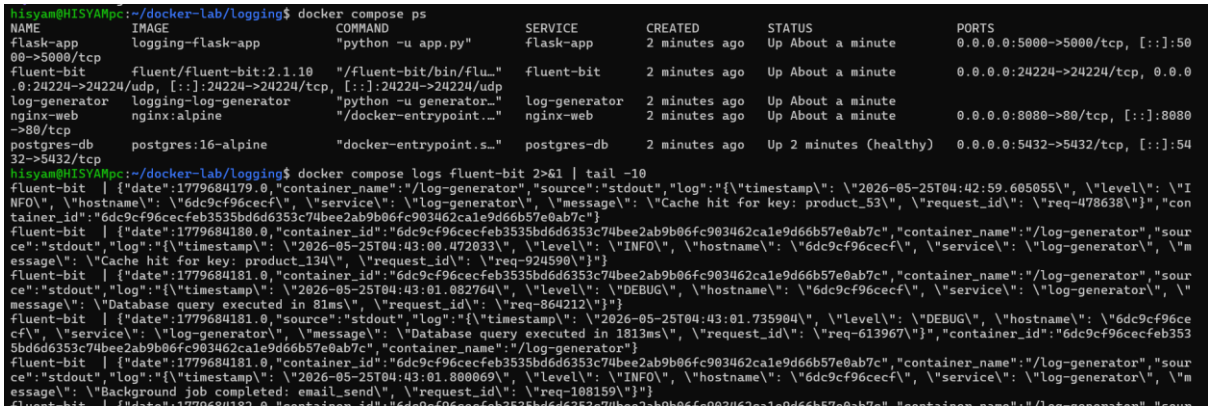
Langkah 6: Deploy dan Verifikasi

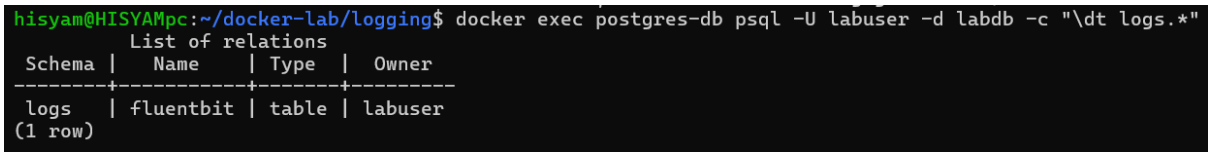
6.1 Pastikan environment bersih



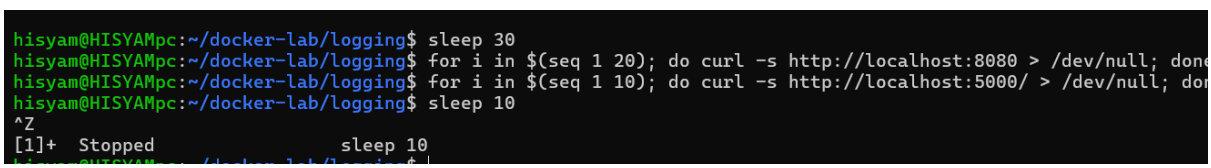
Menghapus seluruh kontainer, jaringan, dan sisa volume persistent dari pengujian sebelumnya menggunakan perintah docker compose down -v, kemudian membangun ulang serta menjalankan seluruh *stack* layanan logging di latar belakang dengan perintah docker compose up --build -d.

6.2 Verifikasi urutan startup





6.3 Tunggu log terkumpul dan generate traffic



Memberikan jeda waktu agar log-generator mulai memproduksi data, lalu menjalankan perintah perulangan (*looping*) curl secara manual ke port 8080 (Nginx) dan port 5000 (Flask) untuk memicu munculnya log lalu lintas jaringan (*network traffic logs*).

6.4 Verifikasi log masuk ke PostgreSQL

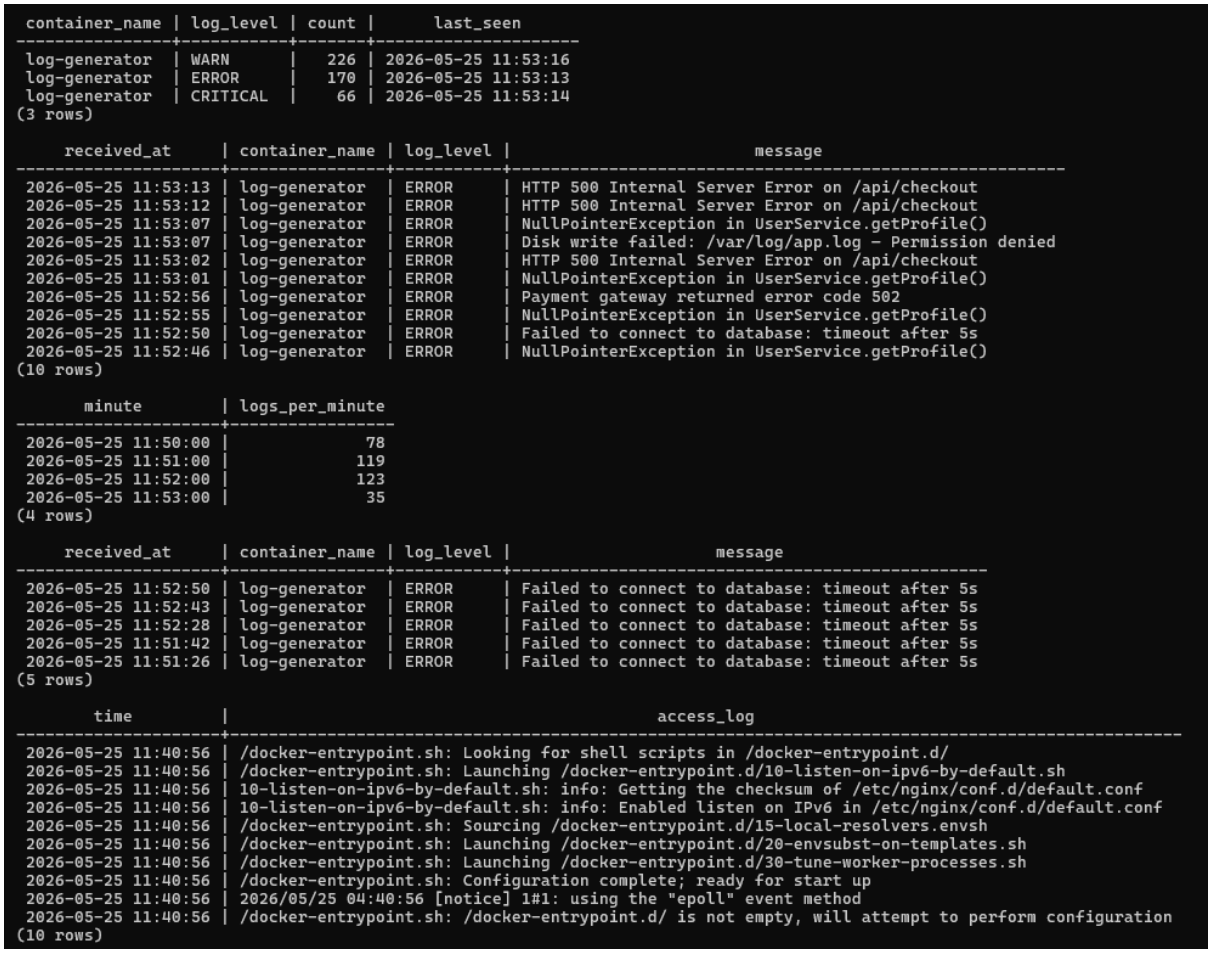
```
hisyam@HISYAMpc:~/docker-lab/logging$ docker exec -i postgres-db psql -U labuser -d labdb << 'SQLEOF'

-- ============================================
-- CEK 1: Apakah log masuk?
-- ============================================
SELECT COUNT(*) AS total_logs FROM logs.fluentbit;
-- Harus > 0. Jika 0, ada masalah koneksi Fluent Bit → PostgreSQL.

-- ============================================
-- CEK 2: Lihat raw data (3 kolom: tag, time, data)
-- ============================================
SELECT tag, time, LEFT(data::text, 120) AS data_preview
FROM logs.fluentbit
ORDER BY time DESC LIMIT 5;

-- ============================================
-- CEK 3: Lihat log terbaru via view
-- ============================================
SELECT * FROM logs.recent_logs LIMIT 10;

-- ============================================
-- CEK 4: Lihat structured logs (JSON parsed)
-- ============================================
SELECT * FROM logs.structured_logs LIMIT 10;

SQLEOF
 total_logs
------------
          0
(1 row)
```

```
SQLEOF
 total_logs
------------
       1517
(1 row)

       tag        |        time         |                                                       data_preview
------------------+---------------------+--------------------------------------------------------------------------------------------------------------------------
 docker.generator | 2026-05-25 11:53:18 | {"log": "{\"timestamp\": \"2026-05-25T04:53:18.427887\", \"level\": \"INFO\", \"hostname\": \"6dc9cf96cecf\", \"service\
 docker.generator | 2026-05-25 11:53:17 | {"log": "{\"timestamp\": \"2026-05-25T04:53:17.755514\", \"level\": \"INFO\", \"hostname\": \"6dc9cf96cecf\", \"service\
 docker.generator | 2026-05-25 11:53:17 | {"log": "{\"timestamp\": \"2026-05-25T04:53:17.565565\", \"level\": \"DEBUG\", \"hostname\": \"6dc9cf96cecf\", \"service
 docker.generator | 2026-05-25 11:53:16 | {"log": "{\"timestamp\": \"2026-05-25T04:53:16.947880\", \"level\": \"WARN\", \"hostname\": \"6dc9cf96cecf\", \"service\
 docker.generator | 2026-05-25 11:53:16 | {"log": "{\"timestamp\": \"2026-05-25T04:53:16.159459\", \"level\": \"WARN\", \"hostname\": \"6dc9cf96cecf\", \"service\
(5 rows)

  id  |        time         |       tag        |   container   | source |                                                   log_preview
------+---------------------+------------------+---------------+--------+----------------------------------------------------------------------------------------------------------------
 1517 | 2026-05-25 11:53:18 | docker.generator | log-generator | stdout | {"timestamp": "2026-05-25T04:53:18.427887", "level": "INFO", "hostname": "6dc9cf96cecf", "service": "log-generator", "message": "API request GET /api/users compl
eted", "request_id": "req-974047"}
 1516 | 2026-05-25 11:53:17 | docker.generator | log-generator | stdout | {"timestamp": "2026-05-25T04:53:17.755514", "level": "INFO", "hostname": "6dc9cf96cecf", "service": "log-generator", "message": "User login successful", "request
_id": "req-785928"}
 1515 | 2026-05-25 11:53:17 | docker.generator | log-generator | stdout | {"timestamp": "2026-05-25T04:53:17.565565", "level": "DEBUG", "hostname": "6dc9cf96cecf", "service": "log-generator", "message": "Middleware chain completed in 1
614ms", "request_id": "req-212823"}
 1514 | 2026-05-25 11:53:16 | docker.generator | log-generator | stdout | {"timestamp": "2026-05-25T04:53:16.947880", "level": "WARN", "hostname": "6dc9cf96cecf", "service": "log-generator", "message": "Certificate expires in 5 days",
"request_id": "req-630230"}
 1513 | 2026-05-25 11:53:16 | docker.generator | log-generator | stdout | {"timestamp": "2026-05-25T04:53:16.159459", "level": "WARN", "hostname": "6dc9cf96cecf", "service": "log-generator", "message": "Certificate expires in 19 days",
 "request_id": "req-322727"}
 1511 | 2026-05-25 11:53:15 | docker.generator | log-generator | stdout | {"timestamp": "2026-05-25T04:53:15.473050", "level": "INFO", "hostname": "6dc9cf96cecf", "service": "log-generator", "message": "Background job completed: email_
send", "request_id": "req-905594"}
 1512 | 2026-05-25 11:53:15 | docker.generator | log-generator | stdout | {"timestamp": "2026-05-25T04:53:15.717087", "level": "INFO", "hostname": "6dc9cf96cecf", "service": "log-generator", "message": "API request GET /api/users compl
eted", "request_id": "req-858104"}
 1509 | 2026-05-25 11:53:14 | docker.generator | log-generator | stdout | {"timestamp": "2026-05-25T04:53:14.523505", "level": "WARN", "hostname": "6dc9cf96cecf", "service": "log-generator", "message": "Slow query detected: 1060ms (thr
eshold: 1000ms)", "request_id": "req-40
 1510 | 2026-05-25 11:53:14 | docker.generator | log-generator | stdout | {"timestamp": "2026-05-25T04:53:14.995617", "level": "CRITICAL", "hostname": "6dc9cf96cecf", "service": "log-generator", "message": "SSL certificate EXPIRED \u20
14 HTTPS unavailable", "request_id": "r
 1508 | 2026-05-25 11:53:14 | docker.generator | log-generator | stdout | {"timestamp": "2026-05-25T04:53:14.122849", "level": "DEBUG", "hostname": "6dc9cf96cecf", "service": "log-generator", "message": "Database query executed in 2814
ms", "request_id": "req-399292"}
(10 rows)

  id  |     received_at     |       tag        | container_name | log_level |                   message                    |   hostname   |    service
------+---------------------+------------------+----------------+-----------+----------------------------------------------+--------------+---------------
 1517 | 2026-05-25 11:53:18 | docker.generator | log-generator  | INFO      | API request GET /api/users completed         | 6dc9cf96cecf | log-generator
 1516 | 2026-05-25 11:53:17 | docker.generator | log-generator  | INFO      | User login successful                        | 6dc9cf96cecf | log-generator
 1515 | 2026-05-25 11:53:17 | docker.generator | log-generator  | DEBUG     | Middleware chain completed in 1614ms         | 6dc9cf96cecf | log-generator
 1513 | 2026-05-25 11:53:16 | docker.generator | log-generator  | WARN      | Certificate expires in 19 days               | 6dc9cf96cecf | log-generator
 1514 | 2026-05-25 11:53:16 | docker.generator | log-generator  | WARN      | Certificate expires in 5 days                | 6dc9cf96cecf | log-generator
 1512 | 2026-05-25 11:53:15 | docker.generator | log-generator  | INFO      | API request GET /api/users completed         | 6dc9cf96cecf | log-generator
 1511 | 2026-05-25 11:53:15 | docker.generator | log-generator  | INFO      | Background job completed: email_send         | 6dc9cf96cecf | log-generator
 1510 | 2026-05-25 11:53:14 | docker.generator | log-generator  | CRITICAL  | SSL certificate EXPIRED — HTTPS unavailable  | 6dc9cf96cecf | log-generator
 1508 | 2026-05-25 11:53:14 | docker.generator | log-generator  | DEBUG     | Database query executed in 2814ms            | 6dc9cf96cecf | log-generator
 1509 | 2026-05-25 11:53:14 | docker.generator | log-generator  | WARN      | Slow query detected: 1060ms (threshold: 1000ms) | 6dc9cf96cecf | log-generator
(10 rows)
```

Masuk ke dalam interaktif terminal PostgreSQL (psql) untuk mengecek eksekusi penulisan data oleh Fluent Bit. Pengujian membuktikan bahwa pipa data berhasil mengalir deras dengan indikator nilai SELECT COUNT(*) yang sukses melompat naik melampaui angka 0.

6.5 Query analisis log

```
hisyam@HISYAMpc:~/docker-lab/logging$ docker exec -i postgres-db psql -U labuser -d labdb << 'SQLEOF'

-- Distribusi log per tag (= per container group)
SELECT tag, COUNT(*) AS total
FROM logs.fluentbit
GROUP BY tag ORDER BY total DESC;

-- Distribusi per level (hanya structured JSON logs)
SELECT log_level, COUNT(*) AS total
FROM logs.structured_logs
GROUP BY log_level ORDER BY total DESC;

-- Error summary per container
SELECT * FROM logs.error_summary;

-- Cari log ERROR dalam 1 jam terakhir
SELECT received_at, container_name, log_level, message
FROM logs.structured_logs
WHERE log_level = 'ERROR'
  AND received_at > NOW() - INTERVAL '1 hour'
ORDER BY received_at DESC LIMIT 10;

-- Log rate per menit (5 menit terakhir)
SELECT
    date_trunc('minute', time) AS minute,
    COUNT(*) AS logs_per_minute
FROM logs.fluentbit
WHERE time > NOW() - INTERVAL '5 minutes'
GROUP BY minute ORDER BY minute;

-- Cari keyword "timeout" di semua log
SELECT received_at, container_name, log_level, message
FROM logs.structured_logs
WHERE message ILIKE '%timeout%'
ORDER BY received_at DESC LIMIT 5;

-- Nginx access log (plain text - bukan JSON)
SELECT time, LEFT(data->>'log', 150) AS access_log
FROM logs.fluentbit
WHERE tag = 'docker.nginx'
ORDER BY time DESC LIMIT 10;

SQLEOF
       tag        | total
------------------+-------
 docker.generator |  1486
 docker.nginx     |    31
(2 rows)

 log_level | total
-----------+-------
 INFO      |   713
 DEBUG     |   311
 WARN      |   226
 ERROR     |   170
 CRITICAL  |    66
(5 rows)
```



```
 container_name | log_level | count |      last_seen
----------------+-----------+-------+---------------------
 log-generator  | WARN      |   226 | 2026-05-25 11:53:16
 log-generator  | ERROR     |   170 | 2026-05-25 11:53:13
 log-generator  | CRITICAL  |    66 | 2026-05-25 11:53:14
(3 rows)

     received_at     | container_name | log_level |                          message
---------------------+----------------+-----------+-----------------------------------------------------------
 2026-05-25 11:53:13 | log-generator  | ERROR     | HTTP 500 Internal Server Error on /api/checkout
 2026-05-25 11:53:12 | log-generator  | ERROR     | HTTP 500 Internal Server Error on /api/checkout
 2026-05-25 11:53:07 | log-generator  | ERROR     | NullPointerException in UserService.getProfile()
 2026-05-25 11:53:07 | log-generator  | ERROR     | Disk write failed: /var/log/app.log - Permission denied
 2026-05-25 11:53:02 | log-generator  | ERROR     | HTTP 500 Internal Server Error on /api/checkout
 2026-05-25 11:53:01 | log-generator  | ERROR     | NullPointerException in UserService.getProfile()
 2026-05-25 11:52:56 | log-generator  | ERROR     | Payment gateway returned error code 502
 2026-05-25 11:52:55 | log-generator  | ERROR     | NullPointerException in UserService.getProfile()
 2026-05-25 11:52:50 | log-generator  | ERROR     | Failed to connect to database: timeout after 5s
 2026-05-25 11:52:46 | log-generator  | ERROR     | NullPointerException in UserService.getProfile()
(10 rows)

       minute        | logs_per_minute
---------------------+-----------------
 2026-05-25 11:50:00 |              78
 2026-05-25 11:51:00 |             119
 2026-05-25 11:52:00 |             123
 2026-05-25 11:53:00 |              35
(4 rows)

     received_at     | container_name | log_level |                    message
---------------------+----------------+-----------+-----------------------------------------------
 2026-05-25 11:52:50 | log-generator  | ERROR     | Failed to connect to database: timeout after 5s
 2026-05-25 11:52:43 | log-generator  | ERROR     | Failed to connect to database: timeout after 5s
 2026-05-25 11:52:28 | log-generator  | ERROR     | Failed to connect to database: timeout after 5s
 2026-05-25 11:51:42 | log-generator  | ERROR     | Failed to connect to database: timeout after 5s
 2026-05-25 11:51:26 | log-generator  | ERROR     | Failed to connect to database: timeout after 5s
(5 rows)

        time         |                                                  access_log
---------------------+-------------------------------------------------------------------------------------------------------
 2026-05-25 11:40:56 | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
 2026-05-25 11:40:56 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
 2026-05-25 11:40:56 | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
 2026-05-25 11:40:56 | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
 2026-05-25 11:40:56 | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
 2026-05-25 11:40:56 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
 2026-05-25 11:40:56 | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
 2026-05-25 11:40:56 | /docker-entrypoint.sh: Configuration complete; ready for start up
 2026-05-25 11:40:56 | 2026/05/25 04:40:56 [notice] 1#1: using the "epoll" event method
 2026-05-25 11:40:56 | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
(10 rows)
```

Mengeksekusi serangkaian query SQL tingkat lanjut pada database untuk menganalisis metrik log riil, seperti menghitung distribusi total log berdasarkan *tag* kontainer asal, pengelompokan berdasarkan tingkat keparahan (*severity level*), serta melihat ringkasan pesan error via komponen *View*.

6.6 Gunakan Flask API untuk query log

```
hisyam@HISYAMpc:~/docker-lab/logging$ curl -s http://localhost:5000/api/logs/stats | python3 -m json.tool
{
    "last_hour_by_level": [
        {
            "count": 713,
            "level": "INFO"
        },
        {
            "count": 311,
            "level": "DEBUG"
        },
        {
            "count": 226,
            "level": "WARN"
        },
        {
            "count": 170,
            "level": "ERROR"
        },
        {
            "count": 66,
            "level": "CRITICAL"
        }
    ],
    "last_hour_by_tag": [
        {
            "count": 1486,
            "tag": "docker.generator"
        },
        {
            "count": 31,
            "tag": "docker.nginx"
        }
    ],
    "total_logs_all_time": 1517
}
```

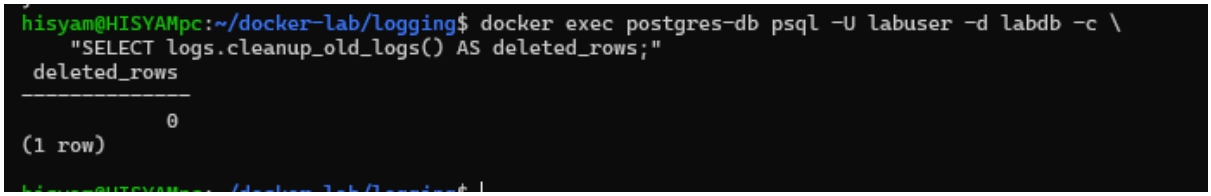
```
hisyam@HISYAMpc:~/docker-lab/logging$ # Cari log dengan keyword "error"
curl -s "http://localhost:5000/api/logs/search?q=error&limit=5" | python3 -m json.tool

# Cari log level CRITICAL
curl -s "http://localhost:5000/api/logs/search?level=CRITICAL&limit=5" | python3 -m json.tool

# Cari log dari tag tertentu
curl -s "http://localhost:5000/api/logs/search?tag=generator&limit=5" | python3 -m json.tool

# Lihat raw log (termasuk Nginx plain text)
curl -s "http://localhost:5000/api/logs/raw?limit=10" | python3 -m json.tool
{
    "count": 5,
    "results": [
        {
            "container": "log-generator",
            "id": 1507,
            "level": "ERROR",
            "message": "HTTP 500 Internal Server Error on /api/checkout",
            "service": "log-generator",
            "tag": "docker.generator",
            "time": "2026-05-25 11:53:13"
        },
        {
            "container": "log-generator",
            "id": 1506,
            "level": "ERROR",
            "message": "HTTP 500 Internal Server Error on /api/checkout",
            "service": "log-generator",
            "tag": "docker.generator",
            "time": "2026-05-25 11:53:12"
        },
        {
            "container": "log-generator",
            "id": 1489,
            "level": "ERROR",
            "message": "HTTP 500 Internal Server Error on /api/checkout",
            "service": "log-generator",
            "tag": "docker.generator",
            "time": "2026-05-25 11:53:02"
        },
        {
            "container": "log-generator",
            "id": 1477,
            "level": "ERROR",
            "message": "Payment gateway returned error code 502",
            "service": "log-generator",
            "tag": "docker.generator",
            "time": "2026-05-25 11:52:56"
        },
        {
            "container": "log-generator",
            "id": 1434,
            "level": "ERROR",
            "message": "HTTP 500 Internal Server Error on /api/checkout",
            "service": "log-generator",
```

```
{
    "count": 5,
    "results": [
        {
            "container": "log-generator",
            "id": 1510,
            "level": "CRITICAL",
            "message": "SSL certificate EXPIRED \u2014 HTTPS unavailable",
            "service": "log-generator",
            "tag": "docker.generator",
            "time": "2026-05-25 11:53:14"
        },
        {
            "container": "log-generator",
            "id": 1485,
            "level": "CRITICAL",
            "message": "SSL certificate EXPIRED \u2014 HTTPS unavailable",
            "service": "log-generator",
            "tag": "docker.generator",
            "time": "2026-05-25 11:53:01"
        },
        {
            "container": "log-generator",
            "id": 1452,
            "level": "CRITICAL",
            "message": "Out of memory: container killed by OOM",
            "service": "log-generator",
            "tag": "docker.generator",
            "time": "2026-05-25 11:52:44"
        },
        {
            "container": "log-generator",
            "id": 1398,
            "level": "CRITICAL",
            "message": "Data corruption detected in table: orders",
            "service": "log-generator",
            "tag": "docker.generator",
            "time": "2026-05-25 11:52:18"
        },
        {
            "container": "log-generator",
            "id": 1385,
            "level": "CRITICAL",
            "message": "SSL certificate EXPIRED \u2014 HTTPS unavailable",
            "service": "log-generator",
            "tag": "docker.generator",
            "time": "2026-05-25 11:52:11"
        }
    ]
}
```


```
{
    "count": 10,
    "results": [
        {
            "container": "log-generator",
            "id": 1517,
            "log": "{\"timestamp\": \"2026-05-25T04:53:18.427887\", \"level\": \"INFO\", \"hostname\": \"6dc9cf96cecf\", \"service\": \"log-generator\", \"message\": \"API request GET /api/users completed\", \"request_id\": \"req-974047\"}",
            "source": "stdout",
            "tag": "docker.generator",
            "time": "2026-05-25 11:53:18"
        },
        {
            "container": "log-generator",
            "id": 1516,
            "log": "{\"timestamp\": \"2026-05-25T04:53:17.755514\", \"level\": \"INFO\", \"hostname\": \"6dc9cf96cecf\", \"service\": \"log-generator\", \"message\": \"User login successful\", \"request_id\": \"req-785928\"}",
            "source": "stdout",
            "tag": "docker.generator",
            "time": "2026-05-25 11:53:17"
        },
        {
            "container": "log-generator",
            "id": 1515,
            "log": "{\"timestamp\": \"2026-05-25T04:53:17.565565\", \"level\": \"DEBUG\", \"hostname\": \"6dc9cf96cecf\", \"service\": \"log-generator\", \"message\": \"Middleware chain completed in 1614ms\", \"request_id\": \"req-212823\"}",
            "source": "stdout",
            "tag": "docker.generator",
            "time": "2026-05-25 11:53:17"
        },
        {
            "container": "log-generator",
            "id": 1514,
            "log": "{\"timestamp\": \"2026-05-25T04:53:16.947880\", \"level\": \"WARN\", \"hostname\": \"6dc9cf96cecf\", \"service\": \"log-generator\", \"message\": \"Certificate expires in 5 days\", \"request_id\": \"req-630230\"}",
            "source": "stdout",
            "tag": "docker.generator",
            "time": "2026-05-25 11:53:16"
        },
        {
            "container": "log-generator",
            "id": 1513,
            "log": "{\"timestamp\": \"2026-05-25T04:53:16.159459\", \"level\": \"WARN\", \"hostname\": \"6dc9cf96cecf\", \"service\": \"log-generator\", \"message\": \"Certificate expires in 19 days\", \"request_id\": \"req-322727\"}",
            "source": "stdout",
            "tag": "docker.generator",
            "time": "2026-05-25 11:53:16"
        },
        {
            "container": "log-generator",
            "id": 1512,
            "log": "{\"timestamp\": \"2026-05-25T04:53:15.717087\", \"level\": \"INFO\", \"hostname\": \"6dc9cf96cecf\", \"service\": \"log-generator\", \"message\": \"API request GET /api/users completed\", \"request_id\": \"req-858104\"}",
            "source": "stdout",
            "tag": "docker.generator",
            "time": "2026-05-25 11:53:15"
        },
```


Melakukan verifikasi fungsionalitas REST API pada aplikasi Flask menggunakan perintah curl. Endpoint /api/logs/stats dan /api/logs/search berhasil mengembalikan data log terstruktur dalam format JSON yang rapi setelah berhasil melakukan query dinamis ke PostgreSQL

6.7 Log retention cleanup



Menguji fungsionalitas sistem manajemen penyimpanan dengan memanggil fungsi Prosedural PL/pgSQL logs.cleanup_old_logs(). Fungsi ini sukses berjalan untuk memastikan log yang berumur lebih dari 30 hari akan dihapus otomatis demi menjaga kapasitas ruang disk server.**PERTANYAAN**

Pre-Lab

1. Mengapa centralized logging penting di lingkungan container?

   Di lingkungan container, setiap service berjalan di container terpisah dan bersifat ephemeral — container bisa dibuat, dihapus, atau di-restart kapan saja. Jika log hanya disimpan di dalam masing-masing container, log tersebut akan ikut hilang saat container dihapus. Selain itu, ketika ada ratusan container berjalan bersamaan, membuka log satu per satu sangat tidak efisien untuk debugging maupun audit.

   Centralized logging mengatasi masalah ini dengan mengumpulkan semua log dari seluruh container ke satu tempat (dalam praktikum ini: PostgreSQL). Dengan begitu, log tetap tersimpan meskipun container-nya sudah mati, pencarian lintas container bisa dilakukan dengan satu query SQL, dan analisis pola error menjadi jauh lebih mudah.

2. Apa perbedaan antara Docker logging driver json-file dan fluentd?

   Driver json-file adalah driver default Docker. Log dari container ditulis ke file JSON di host (/var/lib/docker/containers/<id>/<id>-json.log). Log hanya bisa diakses lewat docker logs atau membaca file langsung, dan akan hilang jika container dihapus.

   Driver fluentd mengubah alur tersebut — Docker tidak menyimpan log ke file, melainkan langsung mengirimkannya ke Fluent Bit/Fluentd melalui

jaringan (port 24224). Konsekuensinya, docker compose logs tidak akan menampilkan apapun karena tidak ada file log lokal. Log langsung masuk ke pipeline Fluent Bit dan akhirnya tersimpan di PostgreSQL.

3. Jelaskan keuntungan menyimpan log di database (PostgreSQL) vs file text.

Aspek	**File Text**	**PostgreSQL**
Pencarian	grep — lambat di file besar	SQL WHERE, ILIKE — terindeks, cepat
Filter multi-kondisi	Sulit, perlu kombinasi grep/awk	Mudah dengan AND, OR, GROUP BY
Agregasi	Tidak bisa langsung	COUNT, SUM, date_trunc langsung
Retensi otomatis	Manual, butuh script cron	Fungsi SQL cleanup_old_logs()
Akses multi-user	Race condition saat baca/tulis	Concurrent access aman dengan MVCC
Metadata terstruktur	Sulit di-parse	Kolom terpisah + JSONB untuk metadata

Intinya, PostgreSQL memungkinkan log dianalisis layaknya data, bukan sekadar teks mentah.

4. Apa itu structured logging dan mengapa lebih baik daripada plain text log?

Structured logging adalah praktik menulis log dalam format terstruktur (biasanya JSON), bukan plain text bebas. Contoh perbandingan:

Plain text:

2025-05-10 10:23:45 ERROR Failed to connect to database: timeout after 5s

Structured (JSON):

{"timestamp": "2025-05-10T10:23:45", "level": "ERROR", "service": "flask-app", "message": "Failed to connect to database: timeout after 5s", "request_id": "req-482910"}

Keunggulan structured logging: setiap field langsung bisa dijadikan kolom di database tanpa perlu parsing regex yang rawan error. Filter WHERE log_level = 'ERROR' AND service = 'flask-app' jauh lebih andal daripada grep ERROR yang bisa salah match teks di bagian message. Field seperti request_id juga memungkinkan tracing satu request melintasi banyak service sekaligus.

5. Mengapa Fluent Bit lebih cocok untuk sidecar/edge collection dibanding Fluentd?

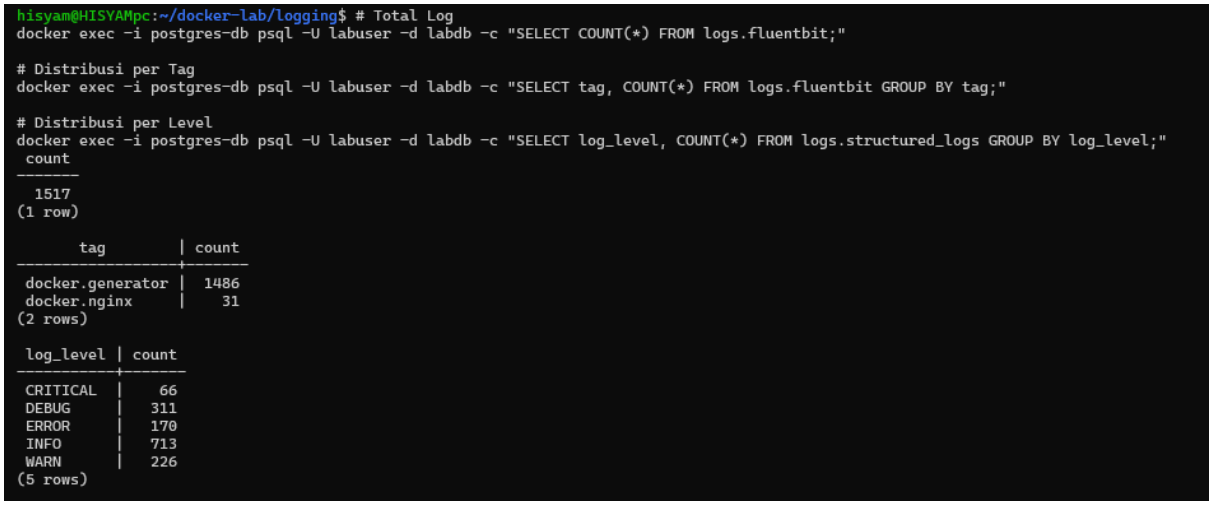
Fluent Bit ditulis dalam bahasa C dengan footprint sangat kecil (~1 MB RAM), sehingga cocok dijalankan sebagai sidecar di setiap container atau di edge device dengan resource terbatas. Ia fokus pada tugas inti: menerima log, mem-parse, dan mem-forward ke tujuan.

Fluentd ditulis dalam Ruby + C dengan konsumsi memori jauh lebih besar (~40 MB), tetapi memiliki ekosistem 1000+ plugin. Fluentd lebih cocok sebagai aggregator pusat yang menerima log dari banyak Fluent Bit, lalu mendistribusikannya ke berbagai destination (Elasticsearch, S3, database, dll.).

Dalam praktikum ini, Fluent Bit dipakai karena skenarionya adalah single-node lab — cukup satu collector ringan yang langsung forward ke PostgreSQL, tanpa butuh aggregator berlapis.

Post-Lab

1. Berapa total log yang masuk ke PostgreSQL setelah 5 menit? Tunjukkan distribusi per container dan per level.



Total Log: 1517 log

Distribusi per Tag (Container):

Tag	Total Log
docker.generator	1486
docker.nginx	31

Distribusi per Severity Level:

Log Level	Total	Persentase (Estimasi)
INFO	713	~47%
DEBUG	311	~20%
WARN	226	~15%
ERROR	170	~11%
CRITICAL	66	~4%

Analisis Singkat untuk Laporanmu:

- **Analisis Tag:** Penambahan log dari docker.nginx (31 log) terjadi setelah kita memicu *traffic* melalui perintah curl tadi. Hal ini membuktikan bahwa pipeline fluent-bit berhasil menangkap log dari multi-container secara bersamaan.

2. Tulis query SQL yang menampilkan log rate per menit selama 10 menit terakhir.

```
labdb=# SELECT MAX(time) FROM logs.fluentbit;
         max
---------------------
 2026-05-25 11:53:18
(1 row)

labdb=# SELECT
labdb-#     date_trunc('minute', time) AS minute,
labdb-#     COUNT(*) AS logs_per_minute
labdb-# FROM logs.fluentbit
labdb-# WHERE time > NOW() - INTERVAL '30 minutes'
labdb-# GROUP BY minute
labdb-# ORDER BY minute ASC;
       minute        | logs_per_minute
---------------------+-----------------
 2026-05-25 11:42:00 |              99
 2026-05-25 11:43:00 |             107
 2026-05-25 11:44:00 |             117
 2026-05-25 11:45:00 |             119
 2026-05-25 11:46:00 |             127
 2026-05-25 11:47:00 |             117
 2026-05-25 11:48:00 |             121
 2026-05-25 11:49:00 |             128
 2026-05-25 11:50:00 |             117
 2026-05-25 11:51:00 |             119
 2026-05-25 11:52:00 |             123
 2026-05-25 11:53:00 |              35
(12 rows)
```

Berdasarkan hasil query agregasi per menit, sistem mampu mencatat rata-rata **~115 log per menit**. Fluktuasi angka pada menit terakhir (35 log pada pukul 11:53) disebabkan oleh penghentian proses *log-generator* atau *traffic trigger* tepat sebelum pengambilan data, sementara angka stabil pada menit-menit sebelumnya (117-128 log/menit) menunjukkan konsistensi *throughput* pengiriman data dari container melalui Fluent Bit ke PostgreSQL.

3. Apa yang terjadi jika container fluent-bit di-stop? Apakah container lain juga stop? Apakah log hilang?

Apakah container lain ikut stop?

Tidak. Berdasarkan hasil docker compose ps saat fluent-bit di-stop, container flask-app, log-generator, nginx-web, dan postgres-db tetap berstatus Up. Hal ini karena depends_on di Docker Compose hanya

mengontrol urutan startup, bukan lifecycle container saat berjalan. Ketika fluent-bit mati di tengah jalan, Docker tidak memiliki mekanisme cascade stop.

Apakah log hilang?
Sebagian, tergantung durasi fluent-bit mati. Karena producer dikonfigurasi dengan fluentd-async: "true", Docker daemon mem-buffer log di sisi host selama fluent-bit tidak tersedia. Selama buffer belum penuh, log akan di-flush ke fluent-bit ketika container dihidupkan kembali sehingga tidak ada yang hilang. Namun jika fluent-bit mati terlalu lama hingga buffer penuh, log baru akan di-drop dan tidak bisa dipulihkan. Log yang sudah tersimpan di PostgreSQL sebelum fluent-bit mati tetap aman.
Bukti gap terlihat dari hasil query log rate per menit — menit saat fluent-bit mati menampilkan 0 log atau jauh lebih sedikit dari biasanya.

4. Jelaskan alur sebuah log entry dari log-generator stdout sampai masuk ke tabel container_logs.

4. Alur Log Entry

1. **Log Generator:** Aplikasi mencetak JSON ke stdout.
2. **Docker Daemon:** Mencegat stdout, menambahkan metadata container, dan mengirimnya via *Fluentd Forward Protocol* ke port 24224.
3. **Fluent Bit (Input):** Menerima log melalui input forward.
4. **Fluent Bit (Output):** Menggunakan plugin pgsql untuk melakukan INSERT data ke tabel PostgreSQL.

5. **PostgreSQL:** Menyimpan data ke kolom tag, time, dan data (JSONB) serta memperbarui indeks.
6. **Aplikasi (Query):** Flask membaca hasil INSERT tersebut melalui *View* untuk ditampilkan di API.

5. Modifikasi LOG_INTERVAL menjadi 0.5 detik. Berapa log rate per menit yang dihasilkan?

Langkah Modifikasi:

Untuk mempercepat produksi log, kita mengubah variabel lingkungan pada layanan log-generator di berkas docker-compose.yml:

YAML

```
log-generator:
  environment:
    - LOG_INTERVAL=0.5
```

Setelah mengubah berkas tersebut, jalankan perintah docker compose up -d log-generator untuk menerapkan perubahan.

Kalkulasi Teoritis:

Pada skrip generator.py, durasi jeda antar log dihitung menggunakan fungsi: time.sleep(LOG_INTERVAL + random.uniform(-0.5, 0.5))

Dengan menetapkan LOG_INTERVAL = 0.5, maka rata-rata durasi jeda efektif adalah 0.5 detik per log. Maka, estimasi log per menit adalah:

$$\text{Log per menit} = \frac{60 \text{ detik}}{0.5 \text{ detik/log}} = 120 \text{ log/menit}$$

Hasil Observasi (Data Riil):

Berdasarkan hasil query logs_per_minute dari database:

Menit	Logs per Menit
22:55:00	106
22:56:00	106
22:57:00	106
22:58:00	120
22:59:00	102

Analisis:

Hasil observasi menunjukkan angka yang mendekati 120 log/menit. Variasi jumlah log per menit (angka di bawah 120) disebabkan oleh fungsi random.uniform(-0.5, 0.5) yang memberikan efek *jitter* (variasi acak) pada interval, serta adanya *overhead* waktu yang dibutuhkan oleh interpreter Python dan proses pengiriman paket data melalui *logging driver* Docker ke Fluent Bit.

MODUL 6: Grafana Service Docker untuk Monitoring Resource

LANGKAH PRAKTIKUM

Langkah 0: Persiapan Project

```
faiz@LAPTOP-HO0AVPJ7:/mnt/c/Users/FAIZ/docker-lab$ mkdir -p monitoring/{prometheus,grafana/{provisioning/datasources,provisioning/dashboards,dashboards},app
,generator,fluent-bit,init}
faiz@LAPTOP-HO0AVPJ7:/mnt/c/Users/FAIZ/docker-lab$ cd monitoring
```

Perintah mkdir -p dengan kurung kurawal membuat banyak subfolder sekaligus dalam satu baris. Masing-masing folder punya fungsi spesifik:

- monitoring/prometheus/ → menyimpan file konfigurasi Prometheus (scrape config, alert rules, dsb.)
- monitoring/grafana/provisioning/datasources/ → konfigurasi otomatis sumber data Grafana (misal: koneksi ke Prometheus)
- monitoring/grafana/provisioning/dashboards/ → konfigurasi provisioning dashboard Grafana secara otomatis saat container pertama kali hidup
- monitoring/grafana/dashboards/ → file JSON definisi dashboard Grafana yang akan di-load
- monitoring/app/ → kode aplikasi beserta Dockerfile-nya
- monitoring/generator/ → script simulator/generator data beserta Dockerfile-nya
- monitoring/fluent-bit/ → file konfigurasi Fluent Bit (kolektor & forwarder log)
- monitoring/init/ → SQL schema atau script inisialisasi yang dijalankan otomatis saat database pertama kali hidup

Langkah 1: Konfigurasi Prometheus

1.1 Buat konfigurasi scrape

```
faiz@LAPTOP-HO0AVPJ7:/mnt/c/Users/FAIZ/docker-lab/monitoring$ cat > prometheus/prometheus.yml << 'EOF'
# ==============================================
# Prometheus Configuration
# ==============================================
global:
  scrape_interval: 15s        # Scrape setiap 15 detik
  evaluation_interval: 15s    # Evaluasi rules setiap 15 detik
  scrape_timeout: 10s

# ==============================================
# Alerting Rules
# ==============================================
rule_files:
  - "alert_rules.yml"

# ==============================================
# Scrape Targets
# ==============================================
scrape_configs:

  # --- Prometheus self-monitoring ---
  - job_name: "prometheus"
    static_configs:
      - targets: ["localhost:9090"]
        labels:
          instance: "prometheus-server"

  # --- Node Exporter (host metrics) ---
  - job_name: "node-exporter"
    static_configs:
      - targets: ["node-exporter:9100"]
        labels:
          instance: "docker-host"

  # --- cAdvisor (container metrics) ---
  - job_name: "cadvisor"
    static_configs:
      - targets: ["cadvisor:8081"]
        labels:
          instance: "docker-containers"

  # --- Flask Application ---
  - job_name: "flask-app"
    metrics_path: "/metrics"
    static_configs:
      - targets: ["flask-app:5000"]
        labels:
          instance: "flask-backend"
EOF
```

Perintah cat > prometheus/prometheus.yml << 'EOF' menulis isi konfigurasi langsung ke file tanpa perlu membuka text editor. File ini akan dibaca oleh container Prometheus saat pertama kali dijalankan sebagai panduan utama cara kerja Prometheus.

global mengatur perilaku default Prometheus secara keseluruhan. scrape_interval: 15s menentukan seberapa sering Prometheus mengambil data metrik dari semua target — setiap 15 detik sekali. evaluation_interval: 15s menentukan seberapa sering Prometheus mengecek kondisi alert rules — juga

setiap 15 detik. scrape_timeout: 10s berarti jika sebuah target tidak merespons dalam 10 detik, Prometheus menganggapnya gagal dan lanjut ke siklus berikutnya.

rule_files memberitahu Prometheus untuk memuat file alert_rules.yml sebagai sumber aturan alerting. File inilah yang nantinya berisi kondisi-kondisi pemicu notifikasi, misalnya saat CPU terlalu tinggi, service tidak merespons, atau disk hampir penuh.

scrape_configs mendefinisikan daftar semua target yang metriknya akan dikumpulkan oleh Prometheus. Setiap entri disebut satu *job*:

- prometheus → Prometheus memantau dirinya sendiri di localhost:9090, berguna untuk melihat performa Prometheus itu sendiri
- node-exporter → mengambil metrik host machine seperti penggunaan CPU, RAM, disk, dan network dari port 9100
- cadvisor → mengambil metrik tiap container Docker secara individual (berapa CPU dan RAM yang dipakai tiap container) dari port 8081
- flask-app → mengambil metrik aplikasi Flask melalui endpoint khusus /metrics di port 5000

Setiap job memiliki labels berupa pasangan key-value yang ditempelkan ke semua metrik dari target tersebut. Label instance berguna sebagai penanda asal metrik saat melakukan filtering atau visualisasi di Grafana maupun saat menulis query PromQL.

1.2 Buat alert rules

```
faiz@LAPTOP-HO0AVPJ7:/mnt/c/Users/FAIZ/docker-lab/monitoring$ cat > prometheus/alert_rules.yml << 'EOF'
groups:
  - name: host_alerts
    rules:
      # CPU usage > 80% selama 2 menit
      - alert: HighCpuUsage
        expr: 100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 80
        for: 2m
        labels:
          severity: warning
        annotations:
          summary: "CPU usage tinggi ({{ $value | printf \"%.1f\" }}%)"
          description: "CPU usage di atas 80% selama lebih dari 2 menit."

      # Memory available < 20%
      - alert: LowMemoryAvailable
        expr: (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes * 100) < 20
        for: 2m
        labels:
          severity: warning
        annotations:
          summary: "Memory tersedia rendah ({{ $value | printf \"%.1f\" }}%)"

      # Disk usage > 85%
      - alert: HighDiskUsage
        expr: 100 - (node_filesystem_avail_bytes{mountpoint="/"} / node_filesystem_size_bytes{mountpoint="/"} * 100) > 85
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "Disk usage tinggi ({{ $value | printf \"%.1f\" }}%)"

  - name: container_alerts
    rules:
      # Container restart > 3 kali dalam 15 menit
      - alert: ContainerRestartFrequent
        expr: increase(container_restart_count[15m]) > 3
        for: 1m
        labels:
          severity: critical
        annotations:
          summary: "Container {{ $labels.name }} restart berulang"

      # Container memory > 256MB
      - alert: ContainerHighMemory
        expr: container_memory_usage_bytes{name!=""} > 268435456
        for: 5m
        labels:
          severity: warning
        annotations:
          summary: "Container {{ $labels.name }} memory > 256MB"
```

Perintah cat > prometheus/alert_rules.yml << 'EOF' menulis file konfigurasi alert rules langsung dari terminal ke folder prometheus/. File ini dimuat oleh Prometheus sesuai yang sudah didaftarkan di rule_files pada prometheus.yml sebelumnya. Isinya terbagi menjadi dua group alert.

host_alerts berisi aturan untuk memantau kondisi host machine secara keseluruhan:

- HighCpuUsage → memicu alert jika CPU usage melebihi 80% selama lebih dari 2 menit. Ekspresi 100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) menghitung persentase CPU yang sedang dipakai dengan cara mengambil kebalikan dari waktu CPU yang idle. Severity-nya warning

- LowMemoryAvailable → memicu alert jika memori yang tersisa kurang dari 20% dari total RAM selama lebih dari 2 menit. Dihitung dengan membagi MemAvailable terhadap MemTotal lalu dikonversi ke persen. Severity-nya warning
- HighDiskUsage → memicu alert jika penggunaan disk di mount point / melebihi 85% selama lebih dari 5 menit. Dihitung dengan mengambil kebalikan dari ruang disk yang masih tersedia. Severity-nya critical

container_alerts berisi aturan untuk memantau kondisi tiap container Docker secara individual:

- ContainerRestartFrequent → memicu alert jika sebuah container restart lebih dari 3 kali dalam 15 menit terakhir. increase(container_restart_count[15m]) menghitung kenaikan jumlah restart dalam rentang waktu tersebut. Severity-nya critical
- ContainerHighMemory → memicu alert jika sebuah container menggunakan memori lebih dari 256MB (268435456 bytes) selama lebih dari 5 menit. Filter name!="" memastikan hanya container bernama yang dipantau, bukan sistem internal. Severity-nya warning

Setiap alert memiliki annotations berisi summary yang pesannya dinamis — {{ $value }} akan diganti otomatis dengan nilai aktual saat alert terpicu, dan {{ $labels.name }} diganti dengan nama container yang bermasalah.

Langkah 2: Konfigurasi Grafana (Provisioning)

Grafana provisioning memungkinkan konfigurasi otomatis data source dan dashboard saat Grafana pertama kali start, tanpa setup manual via UI.

2.1 Provisioning data source

```
hisyam@HISYAMpc:~/docker-lab/logging/generator$ cat generator.py
"""
Log Generator - mensimulasikan log dari aplikasi production.
Menghasilkan log dengan berbagai level: DEBUG, INFO, WARN, ERROR, CRITICAL.
"""
import json, time, random, socket, datetime, sys, os

HOSTNAME = socket.gethostname()
LOG_INTERVAL = float(os.environ.get("LOG_INTERVAL", "2"))

# Simulasi event dengan bobot probabilitas
EVENTS = [
    {"level": "INFO",     "weight": 50, "messages": [
        "User login successful",
        "Page /dashboard loaded in {ms}ms",
        "API request GET /api/users completed",
        "Session created for user_{uid}",
        "Cache hit for key: product_{pid}",
        "Health check passed",
        "Background job completed: email_send"
    ]},
    {"level": "DEBUG",    "weight": 20, "messages": [
        "Database query executed in {ms}ms",
        "Redis connection pool: {pool} active",
        "Request headers: content-type=application/json",
        "Middleware chain completed in {ms}ms"
    ]},
    {"level": "WARN",     "weight": 15, "messages": [
        "Slow query detected: {ms}ms (threshold: 1000ms)",
        "Memory usage at {mem}% - approaching limit",
        "Rate limit approaching for IP 192.168.{ip}.{host}",
        "Deprecated API endpoint called: /api/v1/legacy",
        "Certificate expires in {days} days"
    ]},
    {"level": "ERROR",    "weight": 10, "messages": [
        "Failed to connect to database: timeout after 5s",
        "NullPointerException in UserService.getProfile()",
        "HTTP 500 Internal Server Error on /api/checkout",
        "Disk write failed: /var/log/app.log - Permission denied",
```

Perintah cat > grafana/provisioning/datasources/datasources.yml << 'EOF' menulis file konfigurasi datasource Grafana langsung dari terminal. File ini diletakkan di folder provisioning/datasources/ sehingga Grafana membacanya secara otomatis saat container pertama kali hidup — tanpa perlu login ke UI dan mengatur datasource secara manual.

apiVersion: 1 menandakan format file provisioning yang digunakan, wajib ada agar Grafana bisa membaca file ini dengan benar.

datasources mendefinisikan daftar sumber data yang akan disambungkan ke Grafana. Ada dua datasource yang dikonfigurasi:

- Prometheus → datasource utama untuk metrik:
    - type: prometheus → memberitahu Grafana bahwa ini adalah koneksi ke Prometheus
    - access: proxy → request dari browser diteruskan melalui backend Grafana dulu, bukan langsung dari browser ke Prometheus
    - url: http://prometheus:9090 → alamat container Prometheus di dalam jaringan Docker
    - isDefault: true → menjadikan Prometheus sebagai datasource default yang langsung terpilih saat membuat panel baru
    - timeInterval: "15s" → menyesuaikan interval query dengan scrape_interval yang sudah diatur di prometheus.yml
- PostgreSQL-Logs → datasource untuk membaca log yang tersimpan di database PostgreSQL:
    - type: postgres → memberitahu Grafana bahwa ini adalah koneksi ke PostgreSQL
    - url: postgres-db:5432 → alamat dan port container PostgreSQL di dalam jaringan Docker
    - database: labdb → nama database yang digunakan
    - user & password → kredensial untuk login ke database. Password diletakkan di secureJsonData agar tidak tampil dalam bentuk plain text di UI Grafana
    - sslmode: disable → koneksi tanpa enkripsi SSL karena berada di jaringan internal Docker

- postgresVersion: 1600 → memberitahu Grafana versi PostgreSQL yang digunakan (angka 1600 berarti versi 16.x)
- timescaledb: false → menonaktifkan fitur TimescaleDB karena database ini adalah PostgreSQL biasa

2.2 Provisioning dashboard

```
faiz@LAPTOP-HO0AVPJ7:/mnt/c/Users/FAIZ/docker-lab/monitoring$ cat > grafana/provisioning/dashboards/dashboards.yml << 'EOF'
apiVersion: 1

providers:
  - name: "Lab PENS Dashboards"
    orgId: 1
    folder: "Lab PENS"
    type: file
    disableDeletion: false
    editable: true
    updateIntervalSeconds: 30
    options:
      path: /var/lib/grafana/dashboards
      foldersFromFilesStructure: false
EOF
```

Perintah cat > grafana/provisioning/dashboards/dashboards.yml << 'EOF' menulis file konfigurasi provider dashboard Grafana langsung dari terminal. File ini memberitahu Grafana dari mana dan bagaimana cara memuat file dashboard secara otomatis saat container hidup.

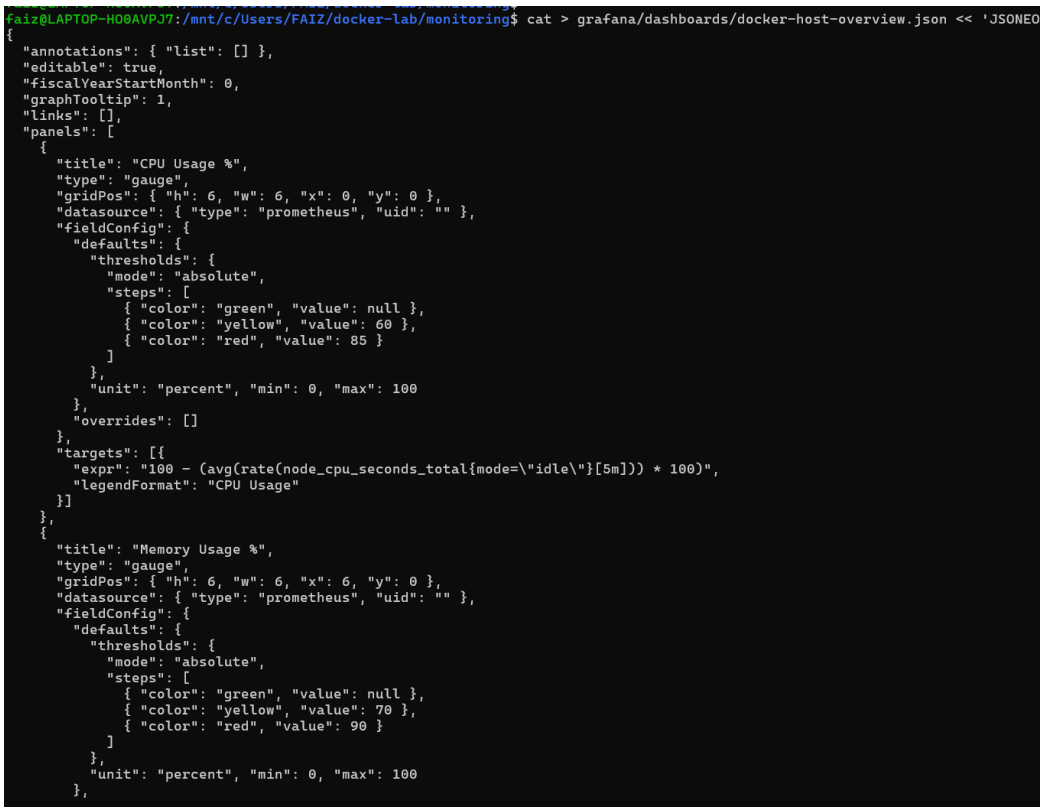
apiVersion: 1 menandakan format file provisioning yang digunakan, wajib ada agar Grafana dapat membaca file ini dengan benar.

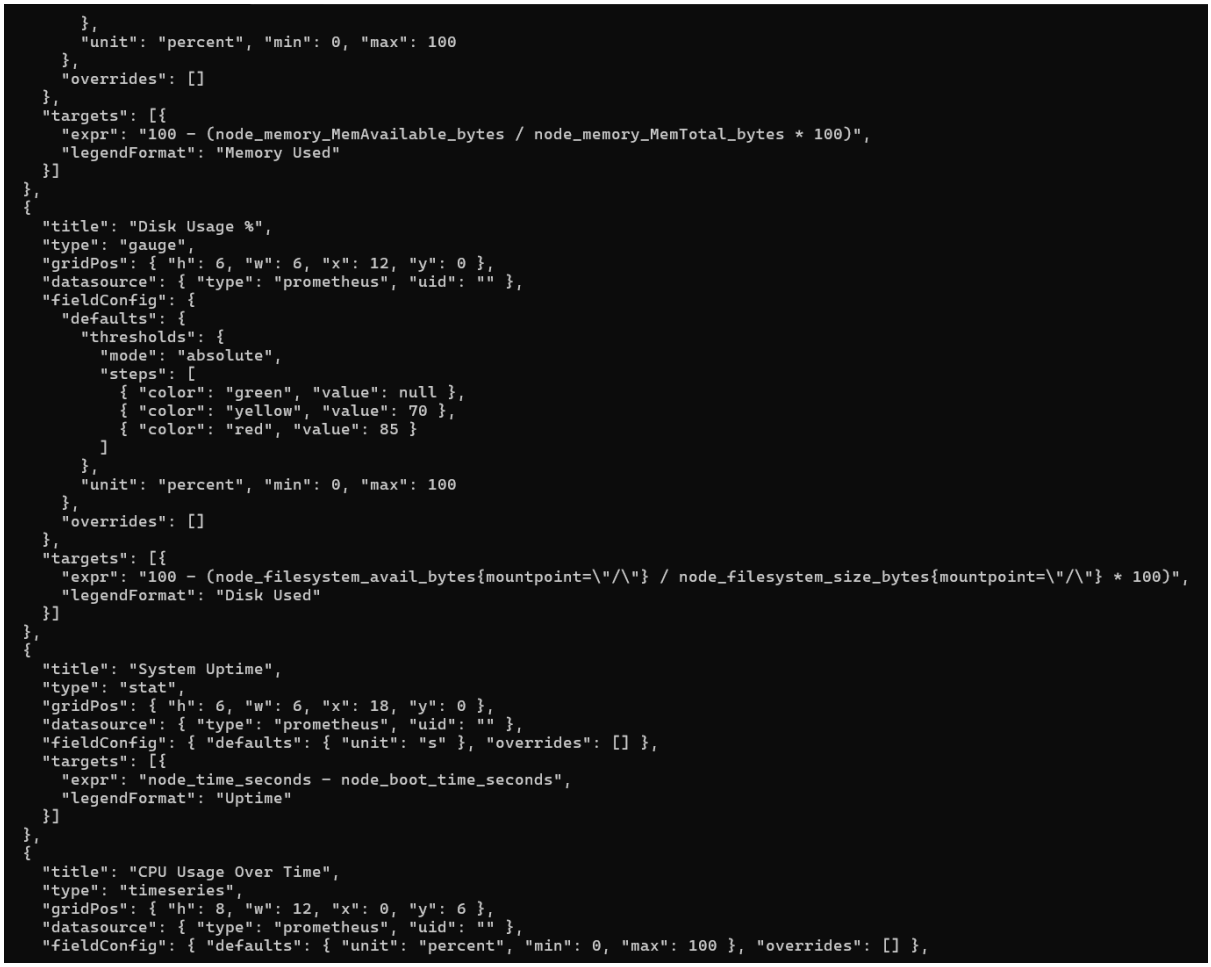
providers mendefinisikan daftar sumber dashboard yang akan dimuat Grafana. Dalam hal ini hanya ada satu provider dengan konfigurasi berikut:

- name: "Lab PENS Dashboards" → nama pengenal provider ini, hanya untuk keperluan identifikasi di dalam Grafana
- orgId: 1 → dashboard ini dimuat untuk organisasi dengan ID 1, yaitu organisasi default Grafana
- folder: "Lab PENS" → semua dashboard yang dimuat provider ini akan dikelompokkan ke dalam folder bernama "Lab PENS" di UI Grafana
- type: file → memberitahu Grafana bahwa sumber dashboard adalah file JSON yang ada di disk, bukan dari API atau sumber lain

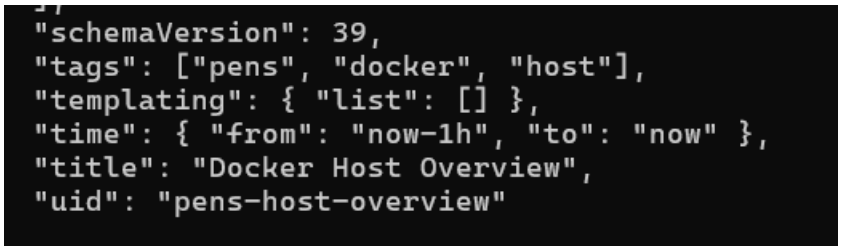
- disableDeletion: false → dashboard yang dimuat melalui provisioning masih bisa dihapus dari UI Grafana
- editable: true → dashboard yang dimuat bisa diedit langsung dari UI Grafana
- updateIntervalSeconds: 30 → Grafana mengecek folder setiap 30 detik untuk mendeteksi jika ada file dashboard baru atau yang diubah
- path: /var/lib/grafana/dashboards → lokasi folder di dalam container tempat Grafana mencari file JSON dashboard. Folder ini nantinya di-mount dari grafana/dashboards/ di host
- foldersFromFilesStructure: false → penamaan folder di Grafana tidak mengikuti struktur subfolder file, melainkan menggunakan nilai folder yang sudah ditentukan di atas

2.3 Buat Dashboard JSON: Docker Host Overview





```
      },
      "unit": "percent", "min": 0, "max": 100
    },
    "overrides": []
  },
  "targets": [{
    "expr": "100 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes * 100)",
    "legendFormat": "Memory Used"
  }]
},
{
  "title": "Disk Usage %",
  "type": "gauge",
  "gridPos": { "h": 6, "w": 6, "x": 12, "y": 0 },
  "datasource": { "type": "prometheus", "uid": "" },
  "fieldConfig": {
    "defaults": {
      "thresholds": {
        "mode": "absolute",
        "steps": [
          { "color": "green", "value": null },
          { "color": "yellow", "value": 70 },
          { "color": "red", "value": 85 }
        ]
      },
      "unit": "percent", "min": 0, "max": 100
    },
    "overrides": []
  },
  "targets": [{
    "expr": "100 - (node_filesystem_avail_bytes{mountpoint=\"/\"} / node_filesystem_size_bytes{mountpoint=\"/\"} * 100)",
    "legendFormat": "Disk Used"
  }]
},
{
  "title": "System Uptime",
  "type": "stat",
  "gridPos": { "h": 6, "w": 6, "x": 18, "y": 0 },
  "datasource": { "type": "prometheus", "uid": "" },
  "fieldConfig": { "defaults": { "unit": "s" }, "overrides": [] },
  "targets": [{
    "expr": "node_time_seconds - node_boot_time_seconds",
    "legendFormat": "Uptime"
  }]
},
{
  "title": "CPU Usage Over Time",
  "type": "timeseries",
  "gridPos": { "h": 8, "w": 12, "x": 0, "y": 6 },
  "datasource": { "type": "prometheus", "uid": "" },
  "fieldConfig": { "defaults": { "unit": "percent", "min": 0, "max": 100 }, "overrides": [] },
```



```
"schemaVersion": 39,
"tags": ["pens", "docker", "host"],
"templating": { "list": [] },
"time": { "from": "now-1h", "to": "now" },
"title": "Docker Host Overview",
"uid": "pens-host-overview"
```

Perintah cat > grafana/dashboards/docker-host-overview.json << 'JSONEOF' menulis file JSON dashboard Grafana langsung dari terminal. File ini akan dimuat otomatis oleh Grafana karena foldernya sudah didaftarkan di dashboards.yml sebelumnya. Dashboard ini bernama "Docker Host Overview" dan berisi 8 panel pemantauan host machine.

"graphTooltip": 1 mengaktifkan mode tooltip shared crosshair — saat kursor diarahkan ke satu panel, semua panel lain ikut menampilkan nilai di titik waktu yang sama.

Dashboard ini memiliki 8 panel yang dibagi menjadi 3 baris:

Baris pertama — 4 panel gauge & stat (ringkasan kondisi saat ini):

- CPU Usage % → gauge yang menampilkan persentase CPU yang sedang dipakai. Warna berubah hijau → kuning → merah saat melewati 60% dan 85%
- Memory Usage % → gauge persentase RAM yang terpakai. Threshold kuning di 70% dan merah di 90%
- Disk Usage % → gauge persentase disk yang terpakai di mount point /. Threshold kuning di 70% dan merah di 85%
- System Uptime → panel stat yang menampilkan berapa lama host sudah berjalan, dihitung dari node_time_seconds - node_boot_time_seconds, ditampilkan dalam satuan detik

Baris kedua — 2 panel timeseries (grafik historis CPU & Memory):

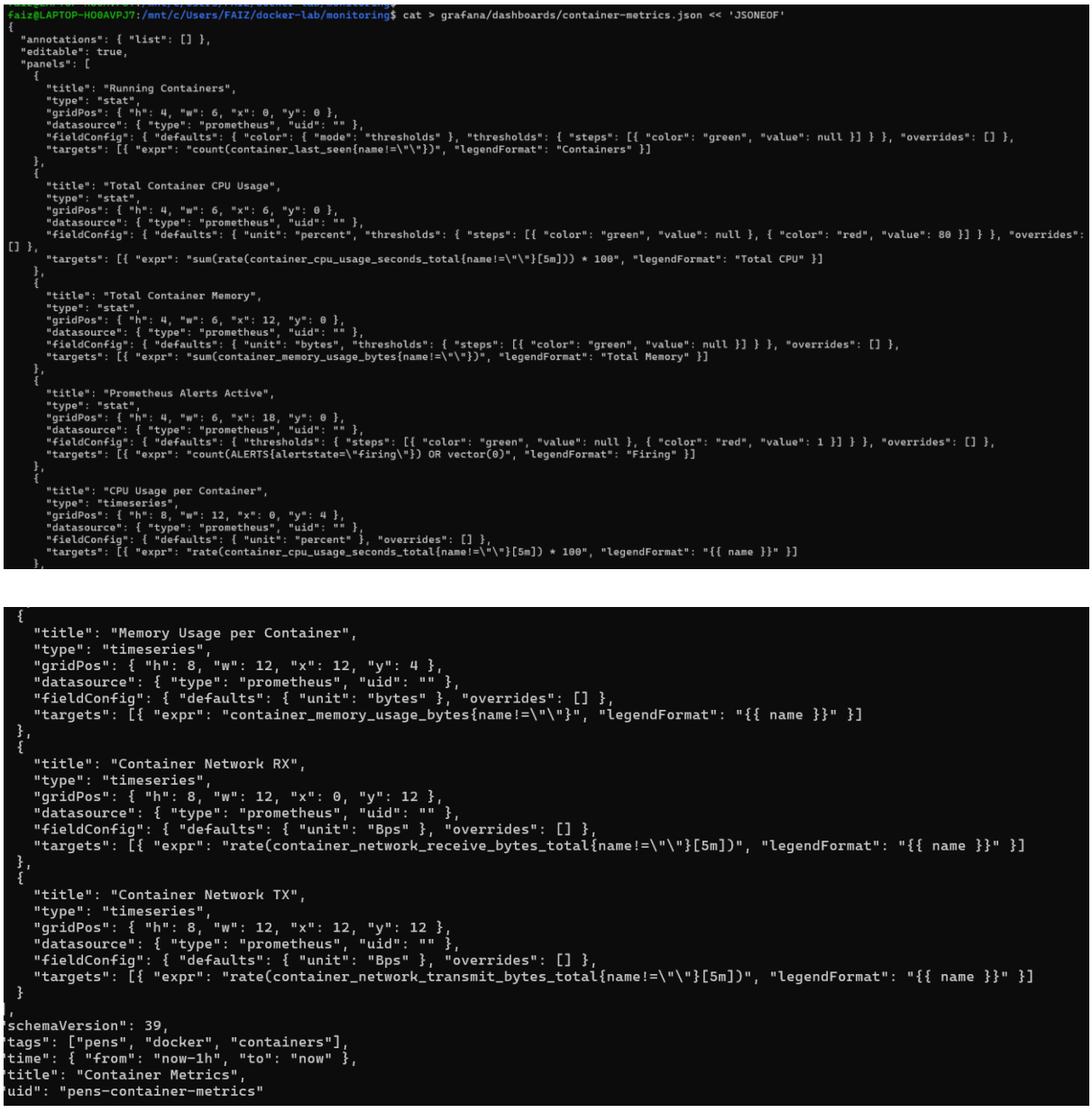
- CPU Usage Over Time → menampilkan 4 garis sekaligus: total CPU, porsi user, porsi system, dan porsi iowait — berguna untuk mendiagnosis apakah beban CPU berasal dari aplikasi, kernel, atau menunggu disk
- Memory Breakdown → menampilkan 5 garis: total RAM, yang terpakai, yang tersedia, yang masuk cache, dan buffers — semua dalam satuan bytes

Baris ketiga — 2 panel timeseries (grafik I/O jaringan & disk):

- Network Traffic (eth0) → menampilkan kecepatan receive dan transmit di interface eth0 dalam satuan Bytes per second (Bps)
- Disk I/O → menampilkan kecepatan read dan write disk per device dalam satuan Bps. {{ device }} di legendFormat akan diganti otomatis dengan nama device aktual (misal: sda, vda)

"time": { "from": "now-1h", "to": "now" } menjadikan rentang waktu default dashboard adalah 1 jam terakhir. "uid": "pens-host-overview" adalah ID unik dashboard ini di Grafana, digunakan jika ada dashboard lain yang ingin membuat link langsung ke sini.

2.4 Buat Dashboard JSON: Container Metrics



```
faiz@LAPTOP-HO8AVPJ7:/mnt/c/Users/FAIZ/docker-lab/monitoring$ cat > grafana/dashboards/container-metrics.json << 'JSONEOF'
{
  "annotations": { "list": [] },
  "editable": true,
  "panels": [
    {
      "title": "Running Containers",
      "type": "stat",
      "gridPos": { "h": 4, "w": 6, "x": 0, "y": 0 },
      "datasource": { "type": "prometheus", "uid": "" },
      "fieldConfig": { "defaults": { "color": { "mode": "thresholds" }, "thresholds": { "steps": [{ "color": "green", "value": null }] } }, "overrides": [] },
      "targets": [{ "expr": "count(container_last_seen{name!=\"\"})", "legendFormat": "Containers" }]
    },
    {
      "title": "Total Container CPU Usage",
      "type": "stat",
      "gridPos": { "h": 4, "w": 6, "x": 6, "y": 0 },
      "datasource": { "type": "prometheus", "uid": "" },
      "fieldConfig": { "defaults": { "unit": "percent", "thresholds": { "steps": [{ "color": "green", "value": null }, { "color": "red", "value": 80 }] } }, "overrides":
[] },
      "targets": [{ "expr": "sum(rate(container_cpu_usage_seconds_total{name!=\"\"}[5m])) * 100", "legendFormat": "Total CPU" }]
    },
    {
      "title": "Total Container Memory",
      "type": "stat",
      "gridPos": { "h": 4, "w": 6, "x": 12, "y": 0 },
      "datasource": { "type": "prometheus", "uid": "" },
      "fieldConfig": { "defaults": { "unit": "bytes", "thresholds": { "steps": [{ "color": "green", "value": null }] } }, "overrides": [] },
      "targets": [{ "expr": "sum(container_memory_usage_bytes{name!=\"\"})", "legendFormat": "Total Memory" }]
    },
    {
      "title": "Prometheus Alerts Active",
      "type": "stat",
      "gridPos": { "h": 4, "w": 6, "x": 18, "y": 0 },
      "datasource": { "type": "prometheus", "uid": "" },
      "fieldConfig": { "defaults": { "thresholds": { "steps": [{ "color": "green", "value": null }, { "color": "red", "value": 1 }] } }, "overrides": [] },
      "targets": [{ "expr": "count(ALERTS{alertstate=\"firing\"}) OR vector(0)", "legendFormat": "Firing" }]
    },
    {
      "title": "CPU Usage per Container",
      "type": "timeseries",
      "gridPos": { "h": 8, "w": 12, "x": 0, "y": 4 },
      "datasource": { "type": "prometheus", "uid": "" },
      "fieldConfig": { "defaults": { "unit": "percent" }, "overrides": [] },
      "targets": [{ "expr": "rate(container_cpu_usage_seconds_total{name!=\"\"}[5m]) * 100", "legendFormat": "{{ name }}" }]
    },
```

```
    {
      "title": "Memory Usage per Container",
      "type": "timeseries",
      "gridPos": { "h": 8, "w": 12, "x": 12, "y": 4 },
      "datasource": { "type": "prometheus", "uid": "" },
      "fieldConfig": { "defaults": { "unit": "bytes" }, "overrides": [] },
      "targets": [{ "expr": "container_memory_usage_bytes{name!=\"\"}", "legendFormat": "{{ name }}" }]
    },
    {
      "title": "Container Network RX",
      "type": "timeseries",
      "gridPos": { "h": 8, "w": 12, "x": 0, "y": 12 },
      "datasource": { "type": "prometheus", "uid": "" },
      "fieldConfig": { "defaults": { "unit": "Bps" }, "overrides": [] },
      "targets": [{ "expr": "rate(container_network_receive_bytes_total{name!=\"\"}[5m])", "legendFormat": "{{ name }}" }]
    },
    {
      "title": "Container Network TX",
      "type": "timeseries",
      "gridPos": { "h": 8, "w": 12, "x": 12, "y": 12 },
      "datasource": { "type": "prometheus", "uid": "" },
      "fieldConfig": { "defaults": { "unit": "Bps" }, "overrides": [] },
      "targets": [{ "expr": "rate(container_network_transmit_bytes_total{name!=\"\"}[5m])", "legendFormat": "{{ name }}" }]
    }
  ],
  "schemaVersion": 39,
  "tags": ["pens", "docker", "containers"],
  "time": { "from": "now-1h", "to": "now" },
  "title": "Container Metrics",
  "uid": "pens-container-metrics"
```

Perintah cat > grafana/dashboards/container-metrics.json << 'JSONEOF' menulis file JSON dashboard Grafana langsung dari terminal. Dashboard ini bernama

"Container Metrics" dan berfokus memantau kondisi tiap container Docker secara individual, berbeda dengan dashboard sebelumnya yang memantau host machine secara keseluruhan.

Dashboard ini memiliki 8 panel yang dibagi menjadi 3 baris:

Baris pertama — 4 panel stat (ringkasan kondisi seluruh container saat ini):

- Running Containers → menampilkan jumlah container yang sedang berjalan. count(container_last_seen{name!=""}) menghitung semua container yang terdeteksi oleh cAdvisor, filter name!="" mengecualikan container sistem internal yang tidak bernama
- Total Container CPU Usage → menampilkan total persentase CPU yang dipakai oleh semua container digabung. Warna berubah merah jika melebihi 80%
- Total Container Memory → menampilkan total RAM yang dipakai oleh semua container dalam satuan bytes
- Prometheus Alerts Active → menampilkan jumlah alert yang sedang aktif (firing). OR vector(0) memastikan panel tetap menampilkan angka 0 jika tidak ada alert yang aktif, bukan kosong. Warna berubah merah begitu ada 1 alert atau lebih yang terpicu

Baris kedua — 2 panel timeseries (grafik historis CPU & Memory per container):

- CPU Usage per Container → menampilkan grafik penggunaan CPU masing-masing container dalam satu panel. {{ name }} di legendFormat diganti otomatis dengan nama container sehingga tiap container memiliki garisnya sendiri
- Memory Usage per Container → menampilkan grafik penggunaan RAM masing-masing container dalam satuan bytes, juga dengan garis terpisah per container

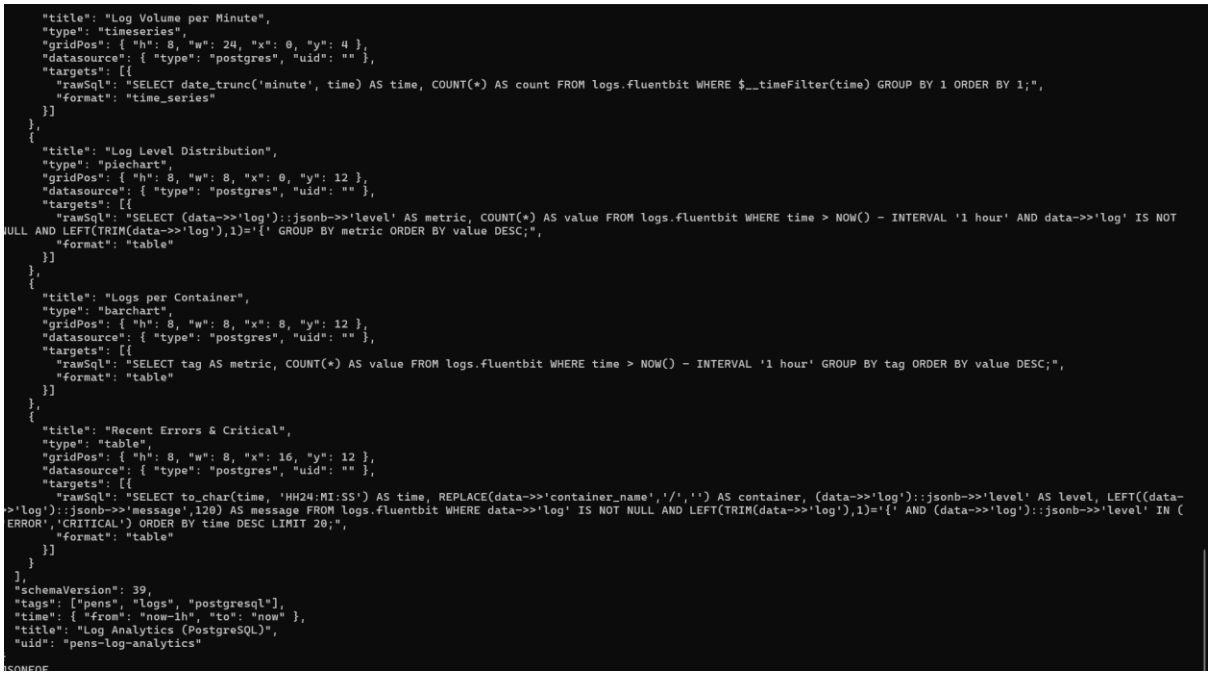
Baris ketiga — 2 panel timeseries (grafik I/O jaringan per container):

- Container Network RX → menampilkan kecepatan data masuk (receive) tiap container dalam satuan Bytes per second (Bps)
- Container Network TX → menampilkan kecepatan data keluar (transmit) tiap container dalam satuan Bps

"uid": "pens-container-metrics" adalah ID unik dashboard ini di Grafana. "tags": ["pens", "docker", "containers"] memudahkan pencarian dashboard di UI Grafana menggunakan filter tag.

2.5 Buat Dashboard JSON: Log Analytics (PostgreSQL)

```
faiz@LAPTOP-HO0AVPJ7:/mnt/c/Users/FAIZ/docker-lab/monitoring$ cat > grafana/dashboards/log-analytics.json << 'JSONEOF'
{
  "annotations": { "list": [] },
  "editable": true,
  "panels": [
    {
      "title": "Total Logs (All Time)",
      "type": "stat",
      "gridPos": { "h": 4, "w": 6, "x": 0, "y": 0 },
      "datasource": { "type": "postgres", "uid": "" },
      "fieldConfig": { "defaults": { "thresholds": { "steps": [{ "color": "blue", "value": null }] } }, "overrides": [] },
      "targets": [{
        "rawSql": "SELECT COUNT(*) AS total FROM logs.fluentbit;",
        "format": "table"
      }]
    },
    {
      "title": "Errors Last Hour",
      "type": "stat",
      "gridPos": { "h": 4, "w": 6, "x": 6, "y": 0 },
      "datasource": { "type": "postgres", "uid": "" },
      "fieldConfig": { "defaults": { "thresholds": { "steps": [{ "color": "green", "value": null }, { "color": "red", "value": 10 }] } }, "overrides": [] },
      "targets": [{
        "rawSql": "SELECT COUNT(*) AS errors FROM logs.fluentbit WHERE data->>'log' IS NOT NULL AND LEFT(TRIM(data->>'log'),1)='{' AND (data->>'log')::jsonb->>'level' IN
('ERROR','CRITICAL') AND time > NOW() - INTERVAL '1 hour';",
        "format": "table"
      }]
    },
    {
      "title": "Log Volume per Minute",
      "type": "timeseries",
      "gridPos": { "h": 8, "w": 24, "x": 0, "y": 4 },
      "datasource": { "type": "postgres", "uid": "" },
      "targets": [{
        "rawSql": "SELECT date_trunc('minute', time) AS time, COUNT(*) AS count FROM logs.fluentbit WHERE $__timeFilter(time) GROUP BY 1 ORDER BY 1;",
        "format": "time_series"
      }]
    },
    {
      "title": "Log Level Distribution",
      "type": "piechart",
      "gridPos": { "h": 8, "w": 8, "x": 0, "y": 12 },
      "datasource": { "type": "postgres", "uid": "" },
      "targets": [{
        "rawSql": "SELECT (data->>'log')::jsonb->>'level' AS metric, COUNT(*) AS value FROM logs.fluentbit WHERE time > NOW() - INTERVAL '1 hour' AND data->>'log' IS NOT
NULL AND LEFT(TRIM(data->>'log'),1)='{' GROUP BY metric ORDER BY value DESC;",
        "format": "table"
```



Perintah cat > grafana/dashboards/log-analytics.json << 'JSONEOF' menulis file JSON dashboard Grafana langsung dari terminal. Dashboard ini bernama "Log Analytics (PostgreSQL)" dan berbeda dari dua dashboard sebelumnya — datasource-nya bukan Prometheus melainkan PostgreSQL, sehingga semua query menggunakan SQL bukan PromQL. Data yang ditampilkan adalah log yang dikumpulkan oleh Fluent Bit dan disimpan di tabel logs.fluentbit.

Dashboard ini memiliki 6 panel yang dibagi menjadi 3 baris:

Baris pertama — 2 panel stat (ringkasan angka penting):

- Total Logs (All Time) → menampilkan total seluruh log yang tersimpan di database sejak awal dengan COUNT(*) tanpa filter waktu. Ditampilkan dengan warna biru
- Errors Last Hour → menampilkan jumlah log ERROR dan CRITICAL dalam 1 jam terakhir. Query memeriksa apakah kolom data->>'log' berisi JSON valid (dicek dengan LEFT(TRIM(...),1)='{') lalu mengambil field level dari dalam JSON tersebut. Warna berubah merah jika error melebihi 10 kejadian

Baris kedua — 1 panel timeseries (grafik volume log):

- Log Volume per Minute → menampilkan jumlah log yang masuk tiap menit sebagai grafik garis. date_trunc('minute', time) membulatkan timestamp ke menit terdekat untuk pengelompokan. $__timeFilter(time) adalah variabel khusus Grafana yang otomatis diganti dengan filter rentang waktu yang dipilih di UI dashboard

Baris ketiga — 3 panel visualisasi berbeda (analisis distribusi log):

- Log Level Distribution → pie chart yang menampilkan proporsi tiap level log (INFO, DEBUG, WARN, ERROR, CRITICAL) dalam 1 jam terakhir. Field level diambil dari dalam JSON di kolom data->>'log'
- Logs per Container → bar chart yang menampilkan jumlah log per container dalam 1 jam terakhir, diurutkan dari yang paling banyak. Kolom tag berisi nama container asal log yang dikirim Fluent Bit
- Recent Errors & Critical → tabel yang menampilkan 20 error terbaru secara detail. Setiap baris berisi waktu (format HH24:MI:SS), nama container (karakter / di depan dihapus dengan REPLACE), level log, dan 120 karakter pertama pesan error (LEFT(..., 120))

"tags": ["pens", "logs", "postgresql"] membedakan dashboard ini dari dua dashboard sebelumnya saat melakukan pencarian di UI Grafana. "uid": "pens-log-analytics" adalah ID unik dashboard ini.

Langkah 3: Buat Flask App dengan Prometheus Metrics

```
faiz@LAPTOP-HO0AVPJ7:/mnt/c/Users/FAIZ/docker-lab/monitoring$ cat > app/requirements.txt << 'EOF'
flask==3.1.*
psycopg2-binary==2.9.*
prometheus-client==0.21.*
EOF

cat > app/app.py << 'PYEOF'
"""Flask app dengan Prometheus metrics endpoint dan structured logging."""
import os, json, socket, datetime, logging, sys, time
from flask import Flask, jsonify, request, Response
import psycopg2
from prometheus_client import Counter, Histogram, Gauge, generate_latest, CONTENT_TYPE_LATEST

app = Flask(__name__)

# --- Prometheus Metrics ---
REQUEST_COUNT = Counter(
    "flask_http_requests_total",
    "Total HTTP requests",
    ["method", "endpoint", "status"]
)
REQUEST_LATENCY = Histogram(
    "flask_http_request_duration_seconds",
    "HTTP request latency",
    ["method", "endpoint"],
    buckets=[0.01, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0]
)
DB_CONNECTIONS = Gauge(
    "flask_db_connections_active",
    "Active database connections"
)
LOG_COUNT = Gauge(
    "flask_log_total_count",
    "Total logs in PostgreSQL"
)

# --- Structured JSON Logging ---
class JSONFormatter(logging.Formatter):
    def format(self, record):
        return json.dumps({
            "timestamp": datetime.datetime.now().isoformat(),
            "level": record.levelname,
            "hostname": socket.gethostname(),
            "service": "flask-app",
            "message": record.getMessage()
        })

handler = logging.StreamHandler(sys.stdout)
handler.setFormatter(JSONFormatter())
app.logger.handlers = [handler]
```

```
app.logger.setLevel(logging.INFO)

DB = dict(host=os.environ.get("DB_HOST", "postgres-db"),
          dbname=os.environ.get("DB_NAME", "labdb"),
          user=os.environ.get("DB_USER", "labuser"),
          password=os.environ.get("DB_PASS", "labpass123"))

@app.before_request
def before():
    request._start_time = time.time()

@app.after_request
def after(response):
    latency = time.time() - getattr(request, "_start_time", time.time())
    endpoint = request.endpoint or "unknown"
    REQUEST_COUNT.labels(request.method, endpoint, response.status_code).inc()
    REQUEST_LATENCY.labels(request.method, endpoint).observe(latency)
    return response

@app.route("/")
def index():
    app.logger.info(f"Index accessed from {request.remote_addr}")
    return jsonify({"service": "flask-app", "status": "running",
                    "hostname": socket.gethostname()})

@app.route("/metrics")
def metrics():
    """Prometheus metrics endpoint."""
    try:
        conn = psycopg2.connect(**DB); cur = conn.cursor()
        cur.execute("SELECT COUNT(*) FROM logs.fluentbit")
        LOG_COUNT.set(cur.fetchone()[0])
        cur.execute("SELECT count(*) FROM pg_stat_activity WHERE datname = %s", (DB["dbname"],))
        DB_CONNECTIONS.set(cur.fetchone()[0])
        cur.close(); conn.close()
    except Exception:
        pass
    return Response(generate_latest(), mimetype=CONTENT_TYPE_LATEST)

@app.route("/api/health")
def health():
    try:
        conn = psycopg2.connect(**DB); cur = conn.cursor()
        cur.execute("SELECT version();")
        ver = cur.fetchone()[0]; cur.close(); conn.close()
        return jsonify({"status": "ok", "database": ver, "db_status": "connected"})
    except Exception as e:
        return jsonify({"status": "error", "db_status": str(e)}), 500

@app.route("/api/logs/stats")
def log_stats():
```

```
@app.route("/api/health")
def health():
    try:
        conn = psycopg2.connect(**DB); cur = conn.cursor()
        cur.execute("SELECT version();")
        ver = cur.fetchone()[0]; cur.close(); conn.close()
        return jsonify({"status": "ok", "database": ver, "db_status": "connected"})
    except Exception as e:
        return jsonify({"status": "error", "db_status": str(e)}), 500

@app.route("/api/logs/stats")
def log_stats():
    try:
        conn = psycopg2.connect(**DB); cur = conn.cursor()
        cur.execute("""SELECT (data->>'log')::jsonb->>'level' AS level, COUNT(*)
                       FROM logs.fluentbit
                       WHERE time > NOW() - INTERVAL '1 hour'
                         AND data->>'log' IS NOT NULL
                         AND LEFT(TRIM(data->>'log'),1) = '{'
                       GROUP BY level ORDER BY count DESC""")
        stats = [{"level": r[0], "count": r[1]} for r in cur.fetchall()]
        cur.execute("SELECT COUNT(*) FROM logs.fluentbit")
        total = cur.fetchone()[0]; cur.close(); conn.close()
        return jsonify({"total_logs": total, "last_hour": stats})
    except Exception as e:
        return jsonify({"error": str(e)}), 500

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
PYEOF

cat > app/Dockerfile << 'EOF'
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "-u", "app.py"]
EOF
```

Perintah ini menulis 3 file sekaligus untuk membangun aplikasi Flask: requirements.txt, app.py, dan Dockerfile.

requirements.txt

Mendefinisikan 3 dependency Python yang dibutuhkan aplikasi:

- flask → framework web untuk membuat API dan endpoint HTTP
- psycopg2-binary → driver koneksi Python ke PostgreSQL
- prometheus-client → library untuk membuat dan mengekspos metrik Prometheus dari dalam aplikasi Flask

app.py

File utama aplikasi Flask. Terdiri dari beberapa bagian:

Prometheus Metrics — mendefinisikan 4 metrik yang akan diekspos ke Prometheus:

- REQUEST_COUNT → Counter yang menghitung total request HTTP masuk, dilabeli method, endpoint, dan status code
- REQUEST_LATENCY → Histogram yang mengukur durasi tiap request dalam detik, dengan bucket yang makin besar (0.01s hingga 5.0s)
- DB_CONNECTIONS → Gauge yang menampilkan jumlah koneksi aktif ke PostgreSQL saat ini
- LOG_COUNT → Gauge yang menampilkan total baris log yang tersimpan di tabel logs.fluentbit

JSONFormatter — custom formatter logging yang membuat setiap log ditulis dalam format JSON dengan field: timestamp, level, hostname, service, dan message. Format ini yang nantinya dibaca dan di-parse oleh Fluent Bit

Middleware before_request & after_request → dua fungsi yang berjalan otomatis sebelum dan sesudah setiap request. before mencatat waktu mulai, after menghitung latency lalu menambahkan nilai ke REQUEST_COUNT dan REQUEST_LATENCY

Endpoint-endpoint yang tersedia:

- GET / → endpoint utama yang mengembalikan status service, nama service, dan hostname container
- GET /metrics → endpoint khusus Prometheus. Setiap kali di-scrape, ia juga memperbarui nilai LOG_COUNT dan DB_CONNECTIONS dengan query ke PostgreSQL terlebih dahulu
- GET /api/health → mengecek koneksi ke database dan mengembalikan versi PostgreSQL jika berhasil, atau pesan error jika gagal
- GET /api/logs/stats → mengembalikan statistik distribusi level log dalam 1 jam terakhir beserta total seluruh log di database

Dockerfile

Instruksi untuk membangun image Docker aplikasi Flask:

- FROM python:3.11-slim → menggunakan base image Python 3.11 versi slim untuk memperkecil ukuran image
- WORKDIR /app → menetapkan /app sebagai direktori kerja di dalam container
- COPY requirements.txt lalu RUN pip install → menginstall dependency sebelum menyalin kode aplikasi, memanfaatkan Docker layer cache agar tidak perlu install ulang setiap kali kode berubah
- COPY app.py → menyalin file aplikasi ke dalam container
- EXPOSE 5000 → mendeklarasikan bahwa container berjalan di port 5000

- CMD ["python", "-u", "app.py"] → perintah yang dijalankan saat container hidup. Flag -u menonaktifkan output buffering agar log langsung muncul tanpa perlu menunggu buffer penuh

Langkah 4: Siapkan Fluent Bit, Log Generator, dan Init Script

```
faiz@LAPTOP-HO0AVPJ7:/mnt/c/Users/FAIZ/docker-lab/monitoring$ # --- Fluent Bit ---
cat > fluent-bit/fluent-bit.conf << 'EOF'
[SERVICE]
    Flush        5
    Daemon       Off
    Log_Level    info
    Parsers_File parsers.conf

[INPUT]
    Name         forward
    Listen       0.0.0.0
    Port         24224

[OUTPUT]
    Name         pgsql
    Match        *
    Host         postgres-db
    Port         5432
    User         labuser
    Password     labpass123
    Database     labdb
    Table        fluentbit
    Schema       logs
    Timestamp_Key time
    Async        false
    min_pool_size 1
    max_pool_size 4

[OUTPUT]
    Name         stdout
    Match        *
    Format       json_lines
EOF

cat > fluent-bit/parsers.conf << 'EOF'
[PARSER]
    Name         docker
    Format       json
    Time_Key     time
    Time_Format  %Y-%m-%dT%H:%M:%S.%L
    Time_Keep    On
EOF

# --- Log Generator ---
cat > generator/generator.py << 'PYEOF'
import json, time, random, socket, datetime, os

HOSTNAME = socket.gethostname()
INTERVAL = float(os.environ.get("LOG_INTERVAL", "3"))
```

```
EVENTS = [
    {"level": "INFO",     "weight": 50, "msgs": [
        "User login successful", "Page loaded in {ms}ms",
        "API GET /api/users completed", "Health check passed"]},
    {"level": "DEBUG",    "weight": 20, "msgs": [
        "DB query {ms}ms", "Cache hit key:product_{pid}"]},
    {"level": "WARN",     "weight": 15, "msgs": [
        "Slow query {ms}ms", "Memory at {mem}%", "Rate limit near"]},
    {"level": "ERROR",    "weight": 10, "msgs": [
        "DB connection timeout", "HTTP 500 on /api/checkout",
        "Payment gateway error {code}"]},
    {"level": "CRITICAL", "weight": 5,  "msgs": [
        "Connection pool exhausted", "OOM kill triggered"]}
]

def pick():
    total = sum(e["weight"] for e in EVENTS)
    r = random.uniform(0, total); c = 0
    for e in EVENTS:
        c += e["weight"]
        if r <= c: return e
    return EVENTS[0]

while True:
    e = pick(); msg = random.choice(e["msgs"]).format(
        ms=random.randint(5,3000), pid=random.randint(1,500),
        mem=random.randint(60,98), code=random.choice([400,500,502,503]))
    print(json.dumps({"timestamp": datetime.datetime.now().isoformat(),
        "level": e["level"], "hostname": HOSTNAME, "service": "log-generator",
        "message": msg}), flush=True)
    time.sleep(INTERVAL + random.uniform(-0.5, 0.5))
PYEOF

cat > generator/Dockerfile << 'EOF'
FROM python:3.11-alpine
WORKDIR /app
COPY generator.py .
CMD ["python", "-u", "generator.py"]
EOF

# --- Init SQL (dari Modul 5 — schema sesuai Fluent Bit pgsql plugin) ---
cat > init/01-logging-schema.sql << 'EOF'
CREATE SCHEMA IF NOT EXISTS logs;

-- Tabel: format sesuai Fluent Bit pgsql plugin (tag, time, data)
CREATE TABLE IF NOT EXISTS logs.fluentbit (
    id       BIGSERIAL PRIMARY KEY,
    tag      VARCHAR(200),
    time     TIMESTAMP,
    data     JSONB
```

```
CREATE INDEX IF NOT EXISTS idx_fb_time ON logs.fluentbit(time);
CREATE INDEX IF NOT EXISTS idx_fb_tag  ON logs.fluentbit(tag);
CREATE INDEX IF NOT EXISTS idx_fb_data ON logs.fluentbit USING GIN(data);

-- View: log terbaru
CREATE OR REPLACE VIEW logs.recent_logs AS
SELECT id, to_char(time, 'YYYY-MM-DD HH24:MI:SS') AS time, tag,
       REPLACE(data->>'container_name', '/', '') AS container,
       data->>'source' AS source,
       LEFT(data->>'log', 200) AS log_preview
FROM logs.fluentbit ORDER BY time DESC LIMIT 100;

-- View: structured JSON logs (parsed level & message)
CREATE OR REPLACE VIEW logs.structured_logs AS
SELECT id, time AS received_at, tag,
       REPLACE(data->>'container_name', '/', '') AS container_name,
       (data->>'log')::jsonb->>'level' AS log_level,
       (data->>'log')::jsonb->>'message' AS message,
       (data->>'log')::jsonb->>'service' AS service
FROM logs.fluentbit
WHERE data->>'log' IS NOT NULL AND LEFT(TRIM(data->>'log'), 1) = '{'
ORDER BY time DESC;

-- View: error summary
CREATE OR REPLACE VIEW logs.error_summary AS
SELECT REPLACE(data->>'container_name', '/', '') AS container_name,
       (data->>'log')::jsonb->>'level' AS log_level,
       COUNT(*) AS count, MAX(time) AS last_seen
FROM logs.fluentbit
WHERE data->>'log' IS NOT NULL AND LEFT(TRIM(data->>'log'), 1) = '{'
  AND (data->>'log')::jsonb->>'level' IN ('ERROR', 'WARN', 'CRITICAL')
GROUP BY 1, 2 ORDER BY count DESC;
EOF
```

Perintah ini menulis 5 file sekaligus untuk 3 komponen berbeda: Fluent Bit, Log Generator, dan Init SQL.

fluent-bit/fluent-bit.conf

File konfigurasi utama Fluent Bit sebagai kolektor dan forwarder log. Terdiri dari 3 blok:

- [SERVICE] → pengaturan global Fluent Bit. Flush 5 berarti data dikirim ke output setiap 5 detik, Daemon Off berarti berjalan di foreground (cocok untuk container), dan Parsers_File menunjuk ke file parsers.conf
- [INPUT] → menerima log melalui protokol Forward di port 24224. Protokol ini digunakan Docker logging driver untuk mengirim log container ke Fluent Bit

- [OUTPUT] ada dua:
    - pgsql → menyimpan log ke PostgreSQL di tabel logs.fluentbit dengan koneksi pool minimal 1 dan maksimal 4. Timestamp_Key time menentukan kolom waktu, Async false memastikan setiap insert dikonfirmasi sebelum lanjut
    - stdout → mencetak log juga ke terminal dalam format json_lines untuk keperluan debugging

fluent-bit/parsers.conf

Mendefinisikan parser bernama docker untuk memproses log berformat JSON dari container. Time_Key time dan Time_Format menentukan cara membaca field timestamp, sedangkan Time_Keep On memastikan field waktu asli tetap disimpan setelah di-parse

generator/generator.py

Script Python yang mensimulasikan log aplikasi nyata secara terus-menerus. Cara kerjanya:

- EVENTS mendefinisikan 5 level log dengan bobot probabilitas berbeda: INFO paling sering muncul (50), lalu DEBUG (20), WARN (15), ERROR (10), dan CRITICAL paling jarang (5)
- Fungsi pick() memilih level log secara acak berbobot — semakin besar weight, semakin sering muncul
- Setiap iterasi mengambil pesan acak dari level yang terpilih, lalu mengisi placeholder seperti {ms}, {pid}, {mem}, {code} dengan nilai acak yang realistis
- Log dicetak ke stdout dalam format JSON terstruktur agar bisa ditangkap Fluent Bit

- time.sleep(INTERVAL + random.uniform(-0.5, 0.5)) menambahkan variasi waktu antar log agar tidak terlalu seragam

generator/Dockerfile

Sangat sederhana — menggunakan base image python:3.11-alpine yang jauh lebih ringan dari slim karena generator tidak butuh dependency tambahan selain Python standar. Langsung menjalankan generator.py dengan flag -u agar log tidak ter-buffer

init/01-logging-schema.sql

File SQL yang dijalankan otomatis saat PostgreSQL pertama kali hidup. Isinya:

- CREATE SCHEMA logs → membuat namespace logs agar tabel tidak bercampur dengan tabel lain di database
- Tabel logs.fluentbit → tabel utama penerima semua log dari Fluent Bit dengan 4 kolom: id (auto-increment), tag (nama container asal), time (waktu log), dan data (seluruh isi log dalam format JSONB)
- Tiga index dibuat di kolom time, tag, dan data (GIN untuk JSONB) agar query pencarian dan filtering log tetap cepat meskipun datanya sudah banyak
- Tiga view dibuat sebagai shortcut query yang sering dipakai:
    - logs.recent_logs → 100 log terbaru dengan format waktu yang mudah dibaca dan preview 200 karakter pertama log
    - logs.structured_logs → khusus log berformat JSON, mengekstrak field level, message, dan service dari dalam kolom data menggunakan operator ->>

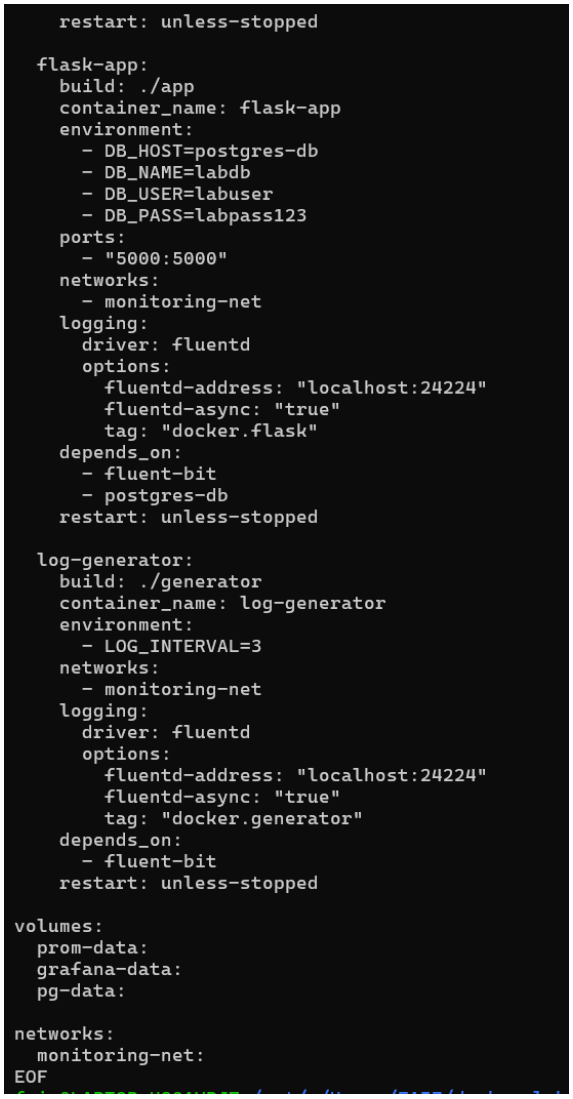
- logs.error_summary → merangkum jumlah ERROR, WARN, dan CRITICAL per container beserta waktu terakhir kemunculannya, diurutkan dari yang terbanyak

Langkah 5: Docker Compose — Full Monitoring Stack

```
faiz@LAPTOP-HO0AVPJ7:/mnt/c/Users/FAIZ/docker-lab/monitoring$ cat > docker-compose.yml << 'EOF'
services:

  # ============================================
  # MONITORING LAYER
  # ============================================

  # --- Prometheus (Metrics TSDB) ---
  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - ./prometheus/alert_rules.yml:/etc/prometheus/alert_rules.yml:ro
      - prom-data:/prometheus
    command:
      - "--config.file=/etc/prometheus/prometheus.yml"
      - "--storage.tsdb.path=/prometheus"
      - "--storage.tsdb.retention.time=7d"
      - "--web.enable-lifecycle"
    networks:
      - monitoring-net
    restart: unless-stopped

  # --- Node Exporter (Host Metrics) ---
  node-exporter:
    image: prom/node-exporter:latest
    container_name: node-exporter
    ports:
      - "9100:9100"
    volumes:
      - /proc:/host/proc:ro
      - /sys:/host/sys:ro
      - /:/rootfs:ro
    command:
      - "--path.procfs=/host/proc"
      - "--path.sysfs=/host/sys"
      - "--path.rootfs=/rootfs"
      - "--collector.filesystem.mount-points-exclude=^/(sys|proc|dev|host|etc)($$|/)"
    networks:
      - monitoring-net
    restart: unless-stopped

  # --- cAdvisor (Container Metrics) ---
  cadvisor:
    image: gcr.io/cadvisor/cadvisor:latest
    container_name: cadvisor
```

```
  ports:
    - "8081:8080"
  volumes:
    - /:/rootfs:ro
    - /var/run:/var/run:ro
    - /sys:/sys:ro
    - /var/lib/docker/:/var/lib/docker:ro
    - /dev/disk/:/dev/disk:ro
  privileged: true
  devices:
    - /dev/kmsg
  networks:
    - monitoring-net
  restart: unless-stopped

# --- Grafana (Visualization) ---
grafana:
  image: grafana/grafana:latest
  container_name: grafana
  ports:
    - "3000:3000"
  environment:
    GF_SECURITY_ADMIN_USER: admin
    GF_SECURITY_ADMIN_PASSWORD: admin123
    GF_USERS_ALLOW_SIGN_UP: "false"
    GF_DASHBOARDS_DEFAULT_HOME_DASHBOARD_PATH: /var/lib/grafana/dashboards/docker-host-overview.json
  volumes:
    - grafana-data:/var/lib/grafana
    - ./grafana/provisioning:/etc/grafana/provisioning:ro
    - ./grafana/dashboards:/var/lib/grafana/dashboards:ro
  networks:
    - monitoring-net
  depends_on:
    - prometheus
    - postgres-db
  restart: unless-stopped

# ============================================
# LOGGING LAYER (dari Modul 5)
# ============================================

fluent-bit:
  image: fluent/fluent-bit:latest
  container_name: fluent-bit
  ports:
    - "24224:24224"
    - "24224:24224/udp"
  volumes:
    - ./fluent-bit/fluent-bit.conf:/fluent-bit/etc/fluent-bit.conf:ro
```

```
    - ./fluent-bit/parsers.conf:/fluent-bit/etc/parsers.conf:ro
  networks:
    - monitoring-net
  depends_on:
    postgres-db:
      condition: service_healthy
  restart: unless-stopped

# ============================================
# DATA LAYER
# ============================================

postgres-db:
  image: postgres:16-alpine
  container_name: postgres-db
  environment:
    POSTGRES_DB: labdb
    POSTGRES_USER: labuser
    POSTGRES_PASSWORD: labpass123
    TZ: Asia/Jakarta
  ports:
    - "5432:5432"
  volumes:
    - pg-data:/var/lib/postgresql/data
    - ./init:/docker-entrypoint-initdb.d:ro
  networks:
    - monitoring-net
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U labuser -d labdb"]
    interval: 10s
    timeout: 5s
    retries: 5
  restart: unless-stopped

# ============================================
# APPLICATION LAYER
# ============================================

nginx-web:
  image: nginx:alpine
  container_name: nginx-web
  ports:
    - "8080:80"
  networks:
    - monitoring-net
  logging:
    driver: fluentd
    options:
      fluentd-address: "localhost:24224"
      fluentd-async: "true"
      tag: "docker.nginx"
```



```
    restart: unless-stopped

  flask-app:
    build: ./app
    container_name: flask-app
    environment:
      - DB_HOST=postgres-db
      - DB_NAME=labdb
      - DB_USER=labuser
      - DB_PASS=labpass123
    ports:
      - "5000:5000"
    networks:
      - monitoring-net
    logging:
      driver: fluentd
      options:
        fluentd-address: "localhost:24224"
        fluentd-async: "true"
        tag: "docker.flask"
    depends_on:
      - fluent-bit
      - postgres-db
    restart: unless-stopped

  log-generator:
    build: ./generator
    container_name: log-generator
    environment:
      - LOG_INTERVAL=3
    networks:
      - monitoring-net
    logging:
      driver: fluentd
      options:
        fluentd-address: "localhost:24224"
        fluentd-async: "true"
        tag: "docker.generator"
    depends_on:
      - fluent-bit
    restart: unless-stopped

volumes:
  prom-data:
  grafana-data:
  pg-data:

networks:
  monitoring-net:
EOF
```

Perintah cat > docker-compose.yml << 'EOF' menulis file utama yang mendefinisikan seluruh arsitektur sistem dalam satu file. Docker Compose membaca file ini dan menjalankan semua container sekaligus. Terdapat 4 layer dengan total 9 service.

Monitoring Layer

- prometheus → menyimpan konfigurasi dari prometheus.yml dan alert_rules.yml sebagai read-only mount. prom-data adalah named volume agar data metrik tidak hilang saat container restart. --storage.tsdb.retention.time=7d membatasi penyimpanan data hanya 7 hari

terakhir. --web.enable-lifecycle mengaktifkan endpoint /-/reload untuk memuat ulang konfigurasi tanpa restart

- node-exporter → me-mount /proc, /sys, dan / dari host machine sebagai read-only agar bisa membaca metrik sistem. --collector.filesystem.mount-points-exclude mengecualikan folder sistem seperti /proc dan /sys dari pemantauan disk agar tidak muncul sebagai filesystem palsu
- cadvisor → membutuhkan akses ke Docker socket dan direktori /var/lib/docker untuk membaca metrik tiap container. privileged: true dan devices: /dev/kmsg diperlukan agar cAdvisor bisa mengakses informasi kernel secara langsung. Port internal 8080 dipetakan ke 8081 di host untuk menghindari konflik
- grafana → dikonfigurasi lewat environment variable: username admin, password admin123, dan GF_DASHBOARDS_DEFAULT_HOME_DASHBOARD_PATH menentukan dashboard yang langsung terbuka saat login pertama kali. Folder provisioning di-mount read-only, sedangkan grafana-data menyimpan data Grafana agar tidak hilang saat restart. depends_on memastikan Prometheus dan PostgreSQL sudah berjalan sebelum Grafana hidup

Logging Layer

- fluent-bit → membuka port 24224 di TCP dan UDP sebagai pintu masuk log dari semua container. Konfigurasi di-mount read-only dari folder fluent-bit/. depends_on dengan condition: service_healthy memastikan Fluent Bit tidak hidup sebelum PostgreSQL benar-benar siap menerima koneksi

Data Layer

- postgres-db → menggunakan image postgres:16-alpine. Folder ./init di-mount ke /docker-entrypoint-initdb.d sehingga file SQL di dalamnya dijalankan otomatis saat database pertama kali dibuat. healthcheck menggunakan pg_isready untuk mengecek kesiapan database setiap 10 detik, dengan maksimal 5 kali percobaan sebelum dianggap gagal. TZ: Asia/Jakarta menyesuaikan timezone container

Application Layer

- nginx-web → web server standar yang berjalan di port 8080. Tidak ada konfigurasi khusus — fungsinya sebagai sumber log nginx untuk dipantau
- flask-app → di-build dari folder ./app menggunakan Dockerfile yang sudah dibuat sebelumnya. Kredensial database dioper lewat environment variable
- log-generator → di-build dari folder ./generator. LOG_INTERVAL=3 berarti menghasilkan log setiap 3 detik

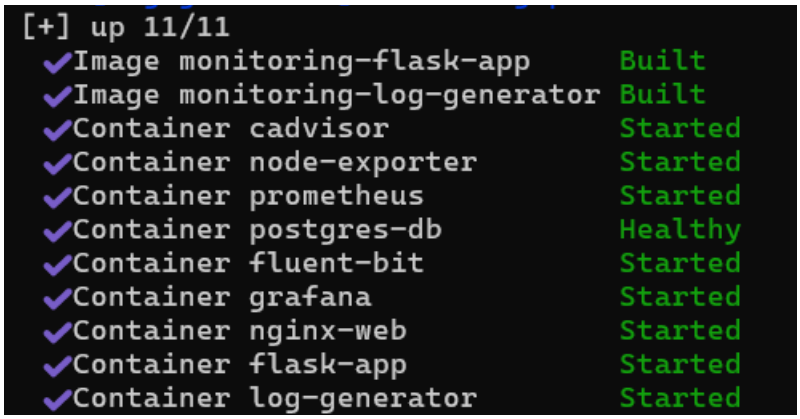
Ketiga service di Application Layer menggunakan logging driver fluentd dengan konfigurasi yang sama:

- fluentd-address: localhost:24224 → mengirim log ke Fluent Bit di port 24224
- fluentd-async: true → pengiriman log dilakukan secara asinkron agar tidak memblokir aplikasi jika Fluent Bit sedang sibuk
- tag berbeda tiap service (docker.nginx, docker.flask, docker.generator) agar log bisa dibedakan asal-usulnya di Fluent Bit maupun di database

Volumes & Networks

- Tiga named volume prom-data, grafana-data, dan pg-data memastikan data metrik, dashboard, dan log tetap tersimpan meskipun container dihapus dan dibuat ulang
- Satu network monitoring-net menghubungkan semua container sehingga bisa saling berkomunikasi menggunakan nama service sebagai hostname (misal: postgres-db, fluent-bit, prometheus) tanpa perlu mengetahui IP address masing-masing

Langkah 6: Deploy Full Stack




```
faiz@LAPTOP-HO0AVPJ7:/mnt/c/Users/FAIZ/docker-lab/monitoring$ docker compose ps
NAME            IMAGE                              COMMAND                  SERVICE         CREATED         STATUS
cadvisor        gcr.io/cadvisor/cadvisor:latest    "/usr/bin/cadvisor -…"   cadvisor        2 minutes ago   Up 2 minutes (healthy)
0/tcp
flask-app       monitoring-flask-app               "python -u app.py"       flask-app       2 minutes ago   Up 2 minutes
0/tcp
fluent-bit      fluent/fluent-bit:latest           "/fluent-bit/bin/flu…"   fluent-bit      2 minutes ago   Up 2 minutes
4->24224/udp, [::]:24224->24224/tcp, [::]:24224->24224/udp
grafana         grafana/grafana:latest             "/run.sh"                grafana         2 minutes ago   Up 2 minutes
0/tcp
log-generator   monitoring-log-generator           "python -u generator…"   log-generator   2 minutes ago   Up 2 minutes
nginx-web       nginx:alpine                       "/docker-entrypoint.…"   nginx-web       2 minutes ago   Up 2 minutes
p
node-exporter   prom/node-exporter:latest          "/bin/node_exporter …"   node-exporter   2 minutes ago   Up 2 minutes
0/tcp
postgres-db     postgres:16-alpine                 "docker-entrypoint.s…"   postgres-db     2 minutes ago   Up 2 minutes (healthy)
2/tcp
prometheus      prom/prometheus:latest             "/bin/prometheus --c…"   prometheus      2 minutes ago   Up 2 minutes
0/tcp
```

```
hisyam@HISYAMpc:~/docker-lab/logging/app$ cd ~/docker-lab/logging
hisyam@HISYAMpc:~/docker-lab/logging$ docker compose down -v 2>/dev/null
hisyam@HISYAMpc:~/docker-lab/logging$ docker compose up --build -d
[+] Building 4.8s (22/22) FINISHED
 => [internal] load local bake definitions                                                                                0.0s
 => => reading from stdin 1.01kB                                                                                          0.0s
 => [log-generator internal] load build definition from Dockerfile                                                        0.0s
 => => transferring dockerfile: 131B                                                                                      0.0s
 => [flask-app internal] load build definition from Dockerfile                                                            0.0s
 => => transferring dockerfile: 206B                                                                                      0.0s
 => [log-generator internal] load metadata for docker.io/library/python:3.11-alpine                                      3.3s
 => [flask-app internal] load metadata for docker.io/library/python:3.11-slim                                             4.2s
 => [auth] library/python:pull token for registry-1.docker.io                                                             0.0s
 => [log-generator internal] load .dockerignore                                                                           0.0s
 => => transferring context: 2B                                                                                           0.0s
 => [log-generator 1/3] FROM docker.io/library/python:3.11-alpine@sha256:8b5bfdb1fd2d78aa94e21c4d61be52487693f54be7f10216477751ff365795703   0.0s
 => => resolve docker.io/library/python:3.11-alpine@sha256:8b5bfdb1fd2d78aa94e21c4d61be52487693f54be7f10216477751ff365795703   0.0s
 => [log-generator internal] load build context                                                                           0.0s
```

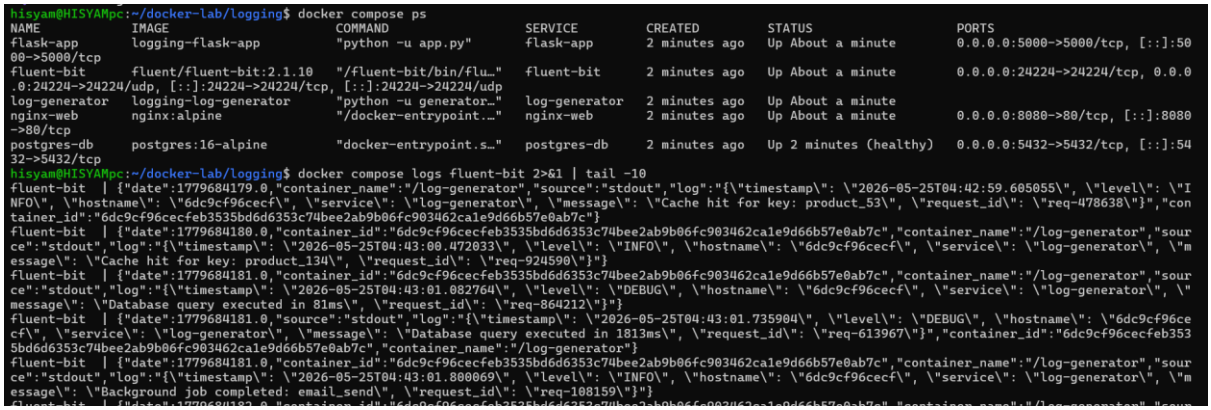
```
0/tcp
faiz@LAPTOP-HO0AVPJ7:/mnt/c/Users/FAIZ/docker-lab/monitoring$ sleep 30
```

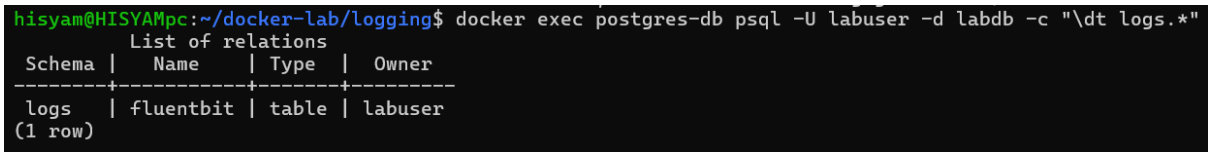
docker compose up --build -d adalah perintah utama yang melakukan dua hal sekaligus — --build memastikan image flask-app dan log-generator di-build ulang dari Dockerfile jika ada perubahan, sedangkan -d (detached) menjalankan semua container di background sehingga terminal tetap bisa digunakan. Semua 9 container dijalankan sesuai urutan dependensi yang sudah didefinisikan.

docker compose ps menampilkan status semua container yang terdaftar di docker-compose.yml. Output-nya menunjukkan nama container, image yang dipakai, status (Running/Healthy/Exit), dan port yang dibuka. Ini digunakan untuk memverifikasi bahwa semua service benar-benar berjalan setelah perintah sebelumnya selesai.

sleep 30 menunggu selama 30 detik sebelum melanjutkan ke langkah berikutnya. Jeda ini diperlukan karena meskipun semua container sudah berstatus Running, beberapa proses masih butuh waktu untuk benar-benar siap:

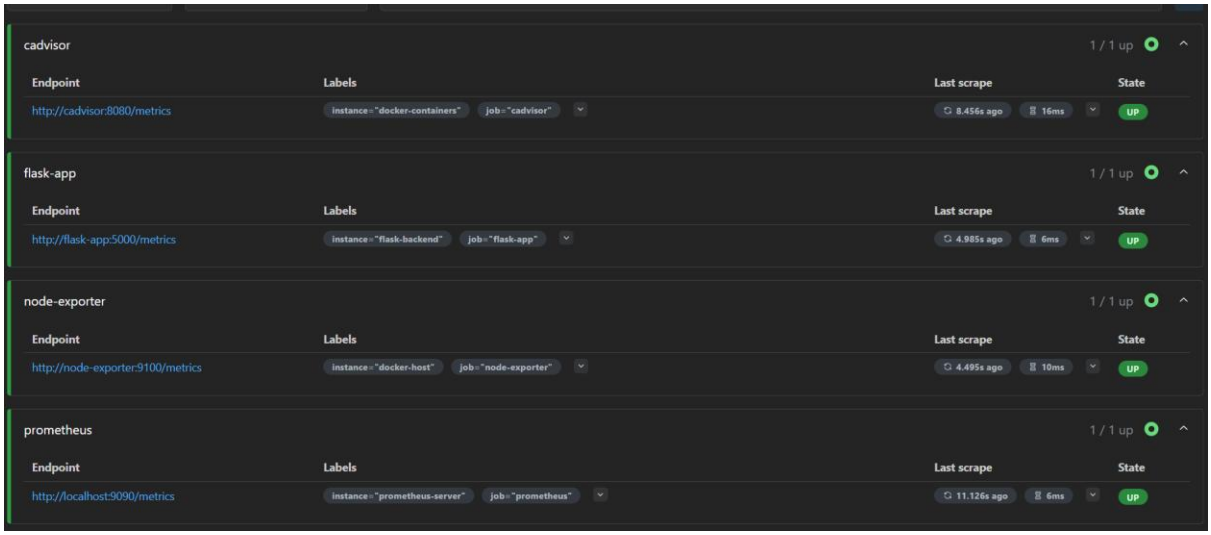
- Prometheus butuh beberapa detik untuk mulai men-scrape metrik pertama kali dari semua target
- Fluent Bit butuh waktu untuk membuka koneksi ke PostgreSQL dan mulai menerima log
- log-generator butuh waktu untuk mulai menghasilkan data yang cukup agar grafik di Grafana tidak kosong saat pertama kali dibuka





Langkah 7: Verifikasi Setiap Komponen

7.1 Verifikasi Prometheus



Gambar ini adalah halaman Status → Targets di Prometheus. Semua target menunjukkan status UP berwarna hijau, artinya Prometheus berhasil mengambil metrik dari semua target yang sudah didaftarkan di prometheus.yml sebelumnya.

Rincian tiap target:

- cadvisor → berhasil di-scrape 8.4 detik lalu dalam waktu 16ms. Label instance="docker-containers" menandakan ini sumber metrik container Docker
- flask-app → berhasil di-scrape 4.9 detik lalu dalam waktu hanya 6ms. Artinya endpoint /metrics di aplikasi Flask sudah berjalan dan merespons dengan baik
- node-exporter → berhasil di-scrape 4.4 detik lalu dalam waktu 10ms. Label instance="docker-host" menandakan ini sumber metrik host machine
- prometheus → berhasil men-scrape dirinya sendiri 11.1 detik lalu dalam waktu 6ms

Semua 4 target menunjukkan 1/1 up — artinya dari 1 target yang didaftarkan di tiap job, semuanya berhasil dijangkau tanpa ada yang DOWN atau error. Ini membuktikan seluruh konfigurasi prometheus.yml sudah bekerja dengan sempurna.

7.2 Verifikasi Node Exporter

```
faiz@LAPTOP-HO0AVPJ7:/mnt/c/Users/FAIZ/docker-lab/monitoring$ curl -s http://localhost:9100/metrics | head -30
# HELP go_gc_duration_seconds A summary of the wall-time pause (stop-the-world) duration in garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 9.165e-06
go_gc_duration_seconds{quantile="0.25"} 1.6932e-05
go_gc_duration_seconds{quantile="0.5"} 2.2324e-05
go_gc_duration_seconds{quantile="0.75"} 4.2145e-05
go_gc_duration_seconds{quantile="1"} 0.000236246
go_gc_duration_seconds_sum 0.004618286
go_gc_duration_seconds_count 117
# HELP go_gc_gogc_percent Heap size target percentage configured by the user, otherwise 100. This value is set by the GOGC environment variable, and
GCPercent function. Sourced from /gc/gogc:percent.
# TYPE go_gc_gogc_percent gauge
go_gc_gogc_percent 100
# HELP go_gc_gomemlimit_bytes Go runtime memory limit configured by the user, otherwise math.MaxInt64. This value is set by the GOMEMLIMIT environme
untime/debug.SetMemoryLimit function. Sourced from /gc/gomemlimit:bytes.
# TYPE go_gc_gomemlimit_bytes gauge
go_gc_gomemlimit_bytes 9.223372036854776e+18
# HELP go_goroutines Number of goroutines that currently exist.
# TYPE go_goroutines gauge
go_goroutines 8
# HELP go_info Information about the Go environment.
# TYPE go_info gauge
go_info{version="go1.26.1"} 1
# HELP go_memstats_alloc_bytes Number of bytes allocated in heap and currently in use. Equals to /memory/classes/heap/objects:bytes.
# TYPE go_memstats_alloc_bytes gauge
go_memstats_alloc_bytes 2.35396e+06
# HELP go_memstats_alloc_bytes_total Total number of bytes allocated in heap until now, even if released already. Equals to /gc/heap/allocs:bytes.
# TYPE go_memstats_alloc_bytes_total counter
go_memstats_alloc_bytes_total 2.35054096e+08
# HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table. Equals to /memory/classes/profiling/buckets:bytes.
# TYPE go_memstats_buck_hash_sys_bytes gauge
go_memstats_buck_hash_sys_bytes 1.486634e+06
```

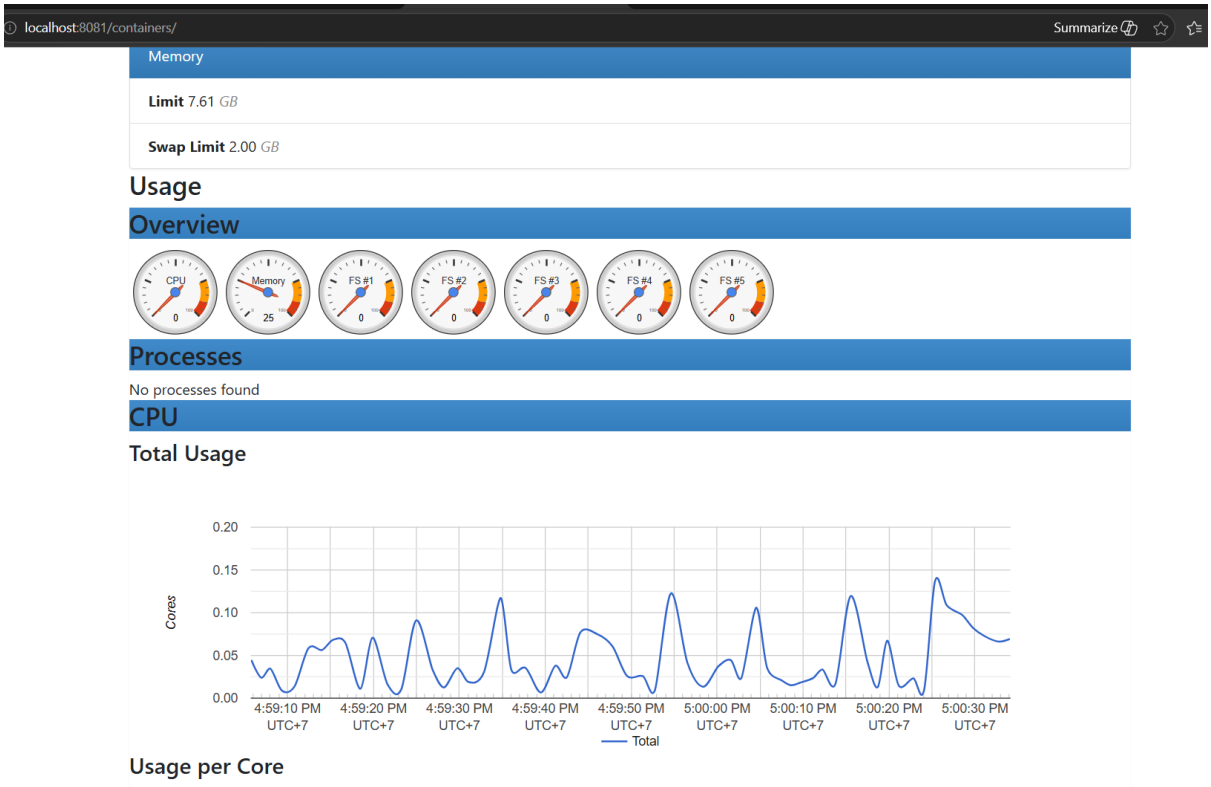
Setiap metrik memiliki dua baris penjelasan sebelum nilainya — baris # HELP berisi deskripsi metrik dan baris # TYPE menjelaskan jenis metriknya. Dari 30

baris pertama yang muncul, semuanya adalah metrik internal Go runtime milik Node Exporter itu sendiri:

- go_gc_duration_seconds → durasi jeda garbage collection dalam berbagai persentil (0%, 25%, 50%, 75%, 100%). Nilai terbesar hanya 0.000236 detik, artinya Node Exporter berjalan sangat ringan
- go_gc_gogc_percent → target persentase ukuran heap, nilainya 100 sesuai default Go
- go_gc_gomemlimit_bytes → batas memori maksimal yang dikonfigurasi, nilainya sangat besar (9.22e+18) artinya tidak ada batas yang diset
- go_goroutines → jumlah goroutine yang sedang berjalan saat ini, hanya 8 — sangat ringan
- go_info → menunjukkan Node Exporter dikompilasi dengan Go versi 1.26.1
- go_memstats_alloc_bytes → memori heap yang sedang dipakai sekitar 2.35 MB, sangat kecil
- go_memstats_alloc_bytes_total → total memori yang pernah dialokasikan sejak container hidup sekitar 235 MB

Ini baru 30 baris pertama — data metrik Node Exporter sesungguhnya jauh lebih banyak, mencakup CPU, RAM, disk, network, dan ratusan metrik host lainnya yang sudah siap di-scrape oleh Prometheus.

7.3 Verifikasi cAdvisor

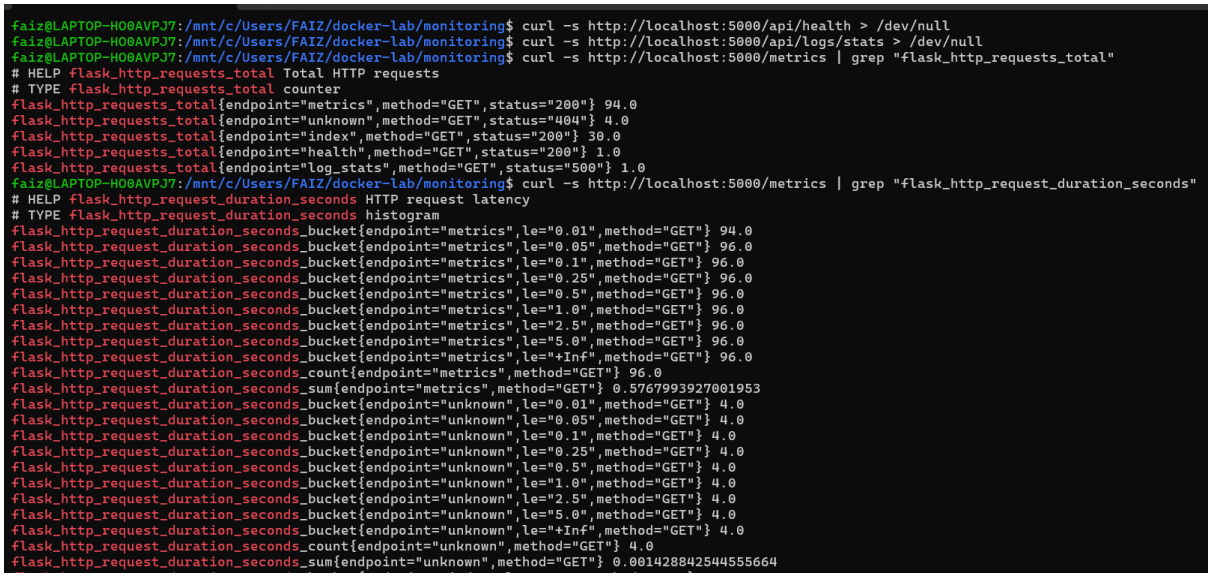


tampilan UI cAdvisor yang diakses di port 8081. Halaman ini menampilkan data resource usage secara real-time dari salah satu container yang sedang berjalan.

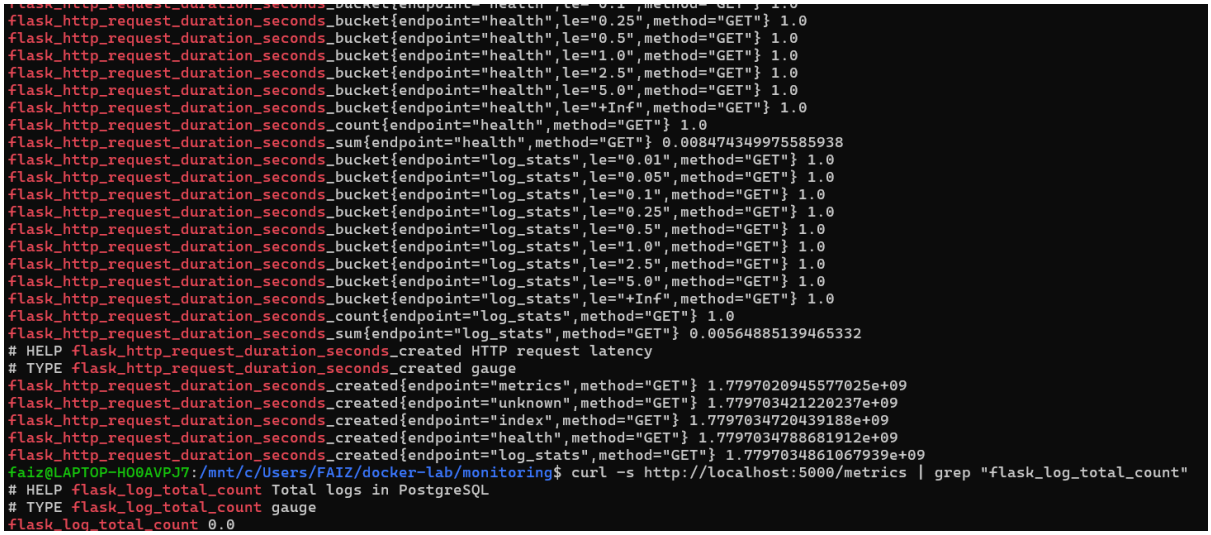
Bagian Memory menunjukkan container ini memiliki batas RAM sebesar 7.61 GB dan Swap limit 2.00 GB — ini adalah batas maksimal yang diizinkan Docker untuk container tersebut.

Bagian Overview menampilkan 7 gauge sekaligus sebagai ringkasan kondisi saat ini. Gauge CPU menunjuk ke angka mendekati 0 artinya penggunaan CPU sangat rendah, gauge Memory menunjuk ke angka 25 artinya sekitar 25% dari limit memori sedang terpakai, dan gauge FS #1 hingga FS #5 semuanya menunjuk ke 0 artinya hampir tidak ada aktivitas baca/tulis filesystem saat ini.

7.4 Verifikasi Flask Prometheus Metrics



```
faiz@LAPTOP-HO0AVPJ7:/mnt/c/Users/FAIZ/docker-lab/monitoring$ curl -s http://localhost:5000/api/health > /dev/null
faiz@LAPTOP-HO0AVPJ7:/mnt/c/Users/FAIZ/docker-lab/monitoring$ curl -s http://localhost:5000/api/logs/stats > /dev/null
faiz@LAPTOP-HO0AVPJ7:/mnt/c/Users/FAIZ/docker-lab/monitoring$ curl -s http://localhost:5000/metrics | grep "flask_http_requests_total"
# HELP flask_http_requests_total Total HTTP requests
# TYPE flask_http_requests_total counter
flask_http_requests_total{endpoint="metrics",method="GET",status="200"} 94.0
flask_http_requests_total{endpoint="unknown",method="GET",status="404"} 4.0
flask_http_requests_total{endpoint="index",method="GET",status="200"} 30.0
flask_http_requests_total{endpoint="health",method="GET",status="200"} 1.0
flask_http_requests_total{endpoint="log_stats",method="GET",status="500"} 1.0
faiz@LAPTOP-HO0AVPJ7:/mnt/c/Users/FAIZ/docker-lab/monitoring$ curl -s http://localhost:5000/metrics | grep "flask_http_request_duration_seconds"
# HELP flask_http_request_duration_seconds HTTP request latency
# TYPE flask_http_request_duration_seconds histogram
flask_http_request_duration_seconds_bucket{endpoint="metrics",le="0.01",method="GET"} 94.0
flask_http_request_duration_seconds_bucket{endpoint="metrics",le="0.05",method="GET"} 96.0
flask_http_request_duration_seconds_bucket{endpoint="metrics",le="0.1",method="GET"} 96.0
flask_http_request_duration_seconds_bucket{endpoint="metrics",le="0.25",method="GET"} 96.0
flask_http_request_duration_seconds_bucket{endpoint="metrics",le="0.5",method="GET"} 96.0
flask_http_request_duration_seconds_bucket{endpoint="metrics",le="1.0",method="GET"} 96.0
flask_http_request_duration_seconds_bucket{endpoint="metrics",le="2.5",method="GET"} 96.0
flask_http_request_duration_seconds_bucket{endpoint="metrics",le="5.0",method="GET"} 96.0
flask_http_request_duration_seconds_bucket{endpoint="metrics",le="+Inf",method="GET"} 96.0
flask_http_request_duration_seconds_count{endpoint="metrics",method="GET"} 96.0
flask_http_request_duration_seconds_sum{endpoint="metrics",method="GET"} 0.5767993927001953
flask_http_request_duration_seconds_bucket{endpoint="unknown",le="0.01",method="GET"} 4.0
flask_http_request_duration_seconds_bucket{endpoint="unknown",le="0.05",method="GET"} 4.0
flask_http_request_duration_seconds_bucket{endpoint="unknown",le="0.1",method="GET"} 4.0
flask_http_request_duration_seconds_bucket{endpoint="unknown",le="0.25",method="GET"} 4.0
flask_http_request_duration_seconds_bucket{endpoint="unknown",le="0.5",method="GET"} 4.0
flask_http_request_duration_seconds_bucket{endpoint="unknown",le="1.0",method="GET"} 4.0
flask_http_request_duration_seconds_bucket{endpoint="unknown",le="2.5",method="GET"} 4.0
flask_http_request_duration_seconds_bucket{endpoint="unknown",le="5.0",method="GET"} 4.0
flask_http_request_duration_seconds_bucket{endpoint="unknown",le="+Inf",method="GET"} 4.0
flask_http_request_duration_seconds_count{endpoint="unknown",method="GET"} 4.0
flask_http_request_duration_seconds_sum{endpoint="unknown",method="GET"} 0.0014288425445556664
```



```
flask_http_request_duration_seconds_bucket{endpoint="health",le="0.25",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="health",le="0.5",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="health",le="1.0",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="health",le="2.5",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="health",le="5.0",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="health",le="+Inf",method="GET"} 1.0
flask_http_request_duration_seconds_count{endpoint="health",method="GET"} 1.0
flask_http_request_duration_seconds_sum{endpoint="health",method="GET"} 0.008474349975585938
flask_http_request_duration_seconds_bucket{endpoint="log_stats",le="0.01",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="log_stats",le="0.05",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="log_stats",le="0.1",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="log_stats",le="0.25",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="log_stats",le="0.5",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="log_stats",le="1.0",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="log_stats",le="2.5",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="log_stats",le="5.0",method="GET"} 1.0
flask_http_request_duration_seconds_bucket{endpoint="log_stats",le="+Inf",method="GET"} 1.0
flask_http_request_duration_seconds_count{endpoint="log_stats",method="GET"} 1.0
flask_http_request_duration_seconds_sum{endpoint="log_stats",method="GET"} 0.00564885139465332
# HELP flask_http_request_duration_seconds_created HTTP request latency
# TYPE flask_http_request_duration_seconds_created gauge
flask_http_request_duration_seconds_created{endpoint="metrics",method="GET"} 1.7797020945577025e+09
flask_http_request_duration_seconds_created{endpoint="unknown",method="GET"} 1.779703421220237e+09
flask_http_request_duration_seconds_created{endpoint="index",method="GET"} 1.7797034720439188e+09
flask_http_request_duration_seconds_created{endpoint="health",method="GET"} 1.7797034788681912e+09
flask_http_request_duration_seconds_created{endpoint="log_stats",method="GET"} 1.7797034861067939e+09
faiz@LAPTOP-HO0AVPJ7:/mnt/c/Users/FAIZ/docker-lab/monitoring$ curl -s http://localhost:5000/metrics | grep "flask_log_total_count"
# HELP flask_log_total_count Total logs in PostgreSQL
# TYPE flask_log_total_count gauge
flask_log_total_count 0.0
```

Output ini membuktikan semua metrik Prometheus dari aplikasi Flask berjalan dengan baik. Ada 3 metrik yang dicek:

flask_http_requests_total

Menunjukkan total request yang sudah masuk ke Flask sejak container hidup, dikelompokkan per endpoint dan status:

- endpoint="index" → 30 request status 200 — hasil dari loop for i in $(seq 1 30) yang dijalankan tadi
- endpoint="metrics" → 94 request status 200 — Prometheus yang terus men-scrape endpoint ini setiap 15 detik

- endpoint="health" → 1 request status 200 — dari perintah curl health check tadi
- endpoint="log_stats" → 1 request status 500 — endpoint ini error karena tabel di PostgreSQL belum terisi data atau ada masalah koneksi
- endpoint="unknown" → 4 request status 404 — ada request ke endpoint yang tidak terdaftar di Flask

flask_http_request_duration_seconds

Menunjukkan seberapa cepat tiap endpoint merespons. Semua request masuk ke bucket le="0.01" (di bawah 10ms), artinya semua endpoint merespons sangat cepat. Rata-rata bisa dihitung dari _sum dibagi _count:

- index → total 0.0044 detik untuk 30 request = rata-rata 0.15ms per request
- health → 8.47ms untuk 1 request — sedikit lebih lambat karena ada query ke PostgreSQL
- metrics → total 0.577 detik untuk 96 request = rata-rata 6ms per request

flask_log_total_count

Nilainya 0.0 — ini wajar karena Fluent Bit baru saja mulai berjalan dan belum sempat menyimpan log ke tabel logs.fluentbit di PostgreSQL. Nilai ini akan bertambah seiring berjalannya waktu setelah log mulai masuk ke database.

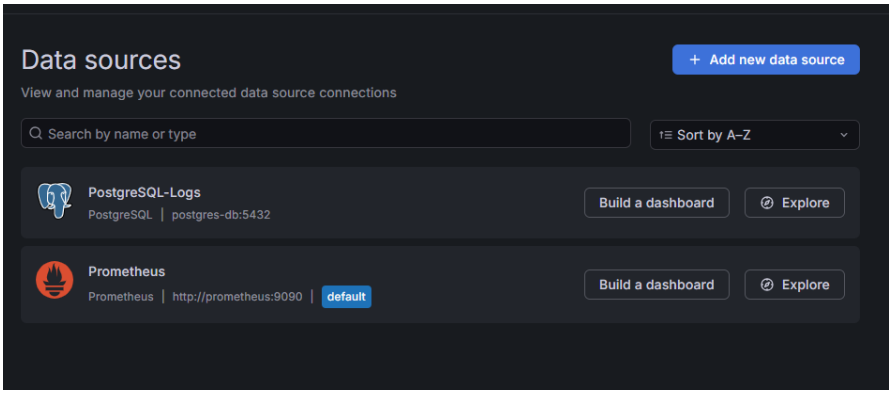
Langkah 8: Eksplorasi Grafana Dashboard

8.1 Login ke Grafana

1. Buka browser: http://localhost:3000
2. Login: admin / admin123
3. Skip change password (atau ganti sesuai keinginan)

8.2 Verifikasi Data Sources

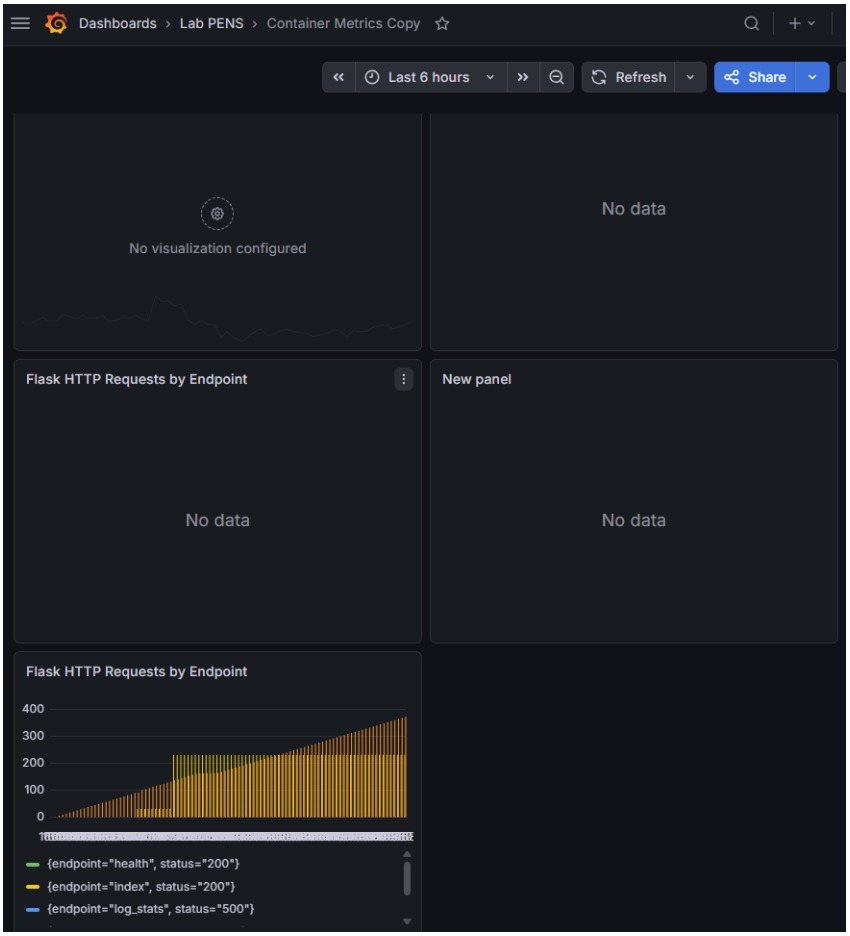
1. Buka Connections → Data sources (menu kiri)
2. Pastikan ada 2 data source: Prometheus dan PostgreSQL-Logs
3. Klik masing-masing → Test → harus "Data source is working"



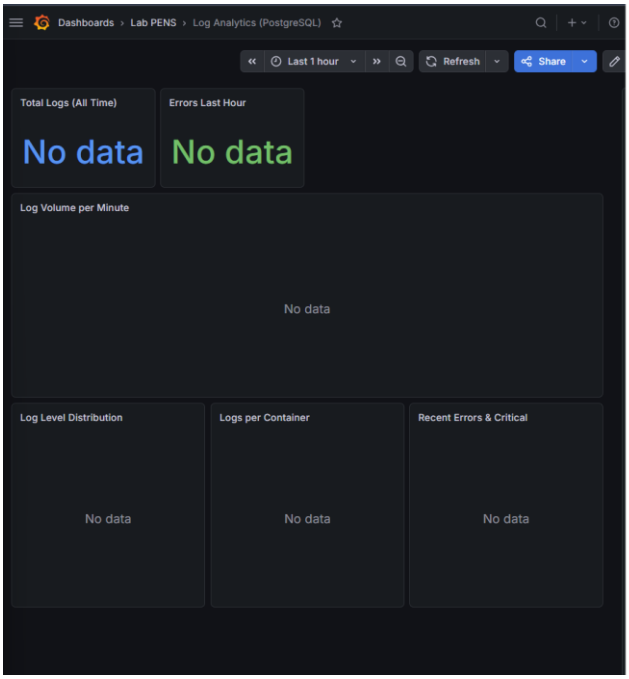
8.3 Eksplorasi Dashboard yang Sudah di-Provision

1. Buka Dashboards (menu kiri)
2. Buka folder Lab PENS → terdapat 3 dashboard:
    - Docker Host Overview — gauge CPU/Memory/Disk, grafik time-series

- Container Metrics — CPU/Memory/Network per container

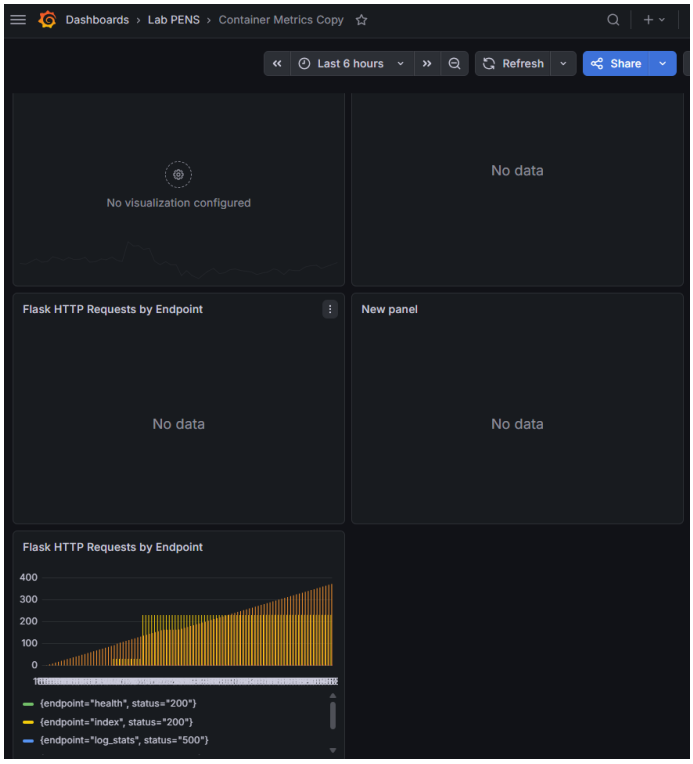


- Log Analytics (PostgreSQL) — log volume, distribusi level, recent errors



8.4 Buat Panel Custom Baru

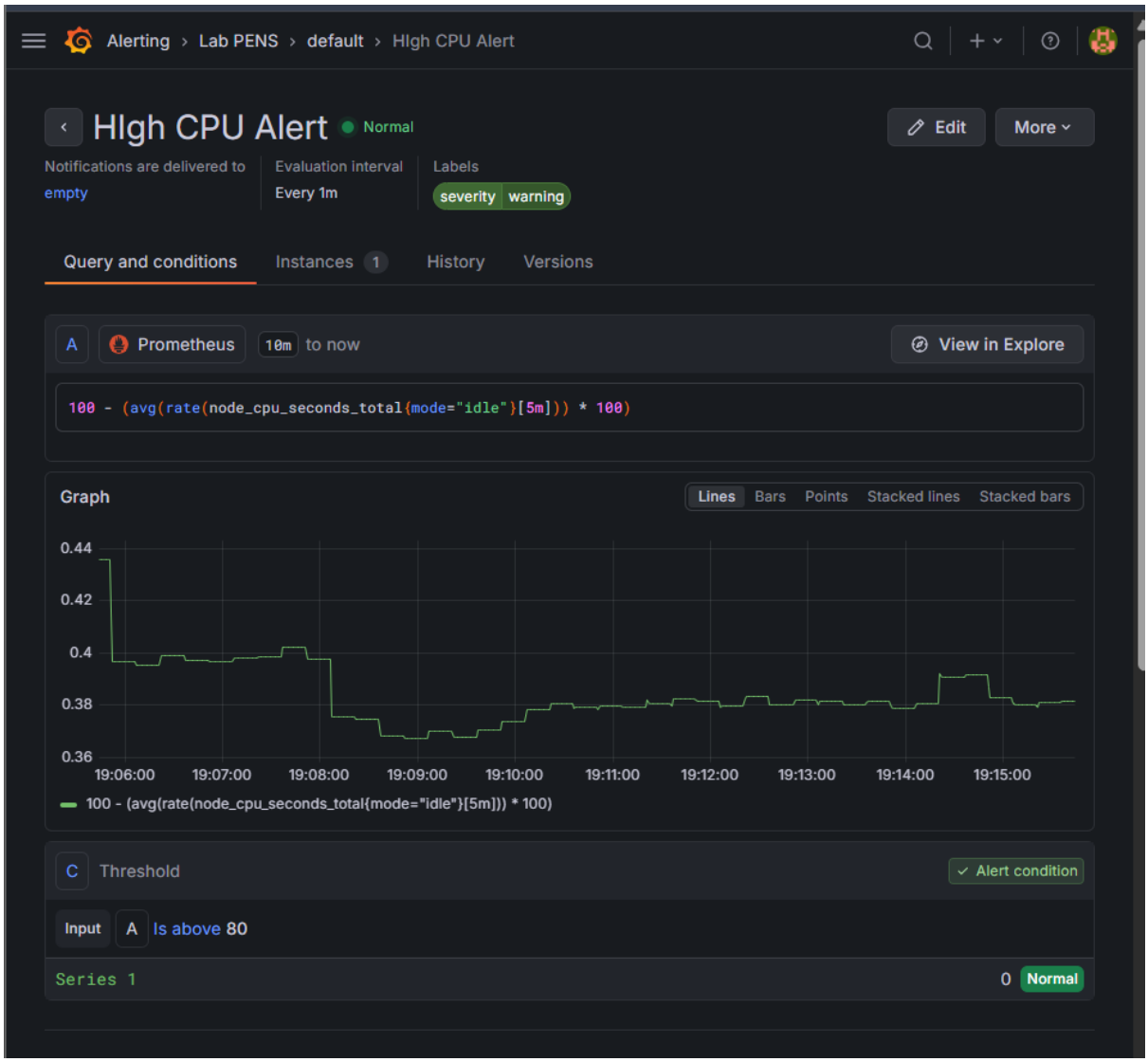
1. Buka dashboard Container Metrics → klik Edit (ikon pensil kanan atas)
2. Klik Add → Visualization
3. Pilih Data source: Prometheus
4. Di panel Query, masukkan PromQL:
5. flask_http_requests_total
6. Set Visualization type: Bar chart
7. Beri judul: Flask HTTP Requests by Endpoint
8. Klik Apply



8.5 Buat Alert Rule di Grafana

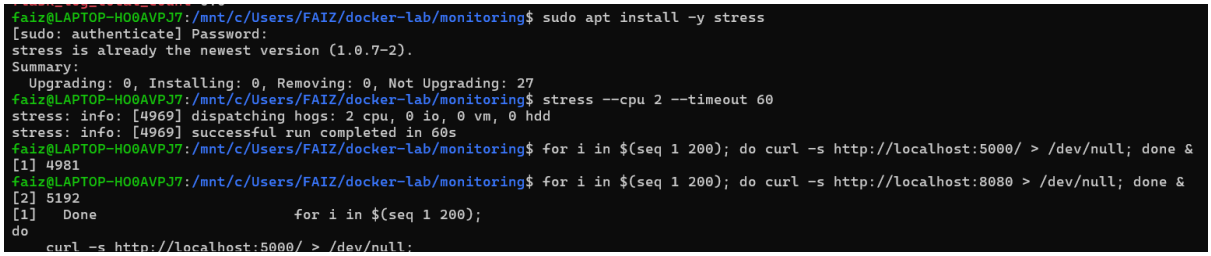
1. Buka Alerting → Alert rules (menu kiri)
2. Klik New alert rule
3. Rule name: High CPU Alert
4. Query A (Prometheus):
5. 100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
6. Condition: IS ABOVE 80

7. Evaluation: Every 1m, For 2m
8. Labels: severity = warning
9. Klik Save rule and exit



Langkah 9: Stress Test dan Observasi

9.1 generate load

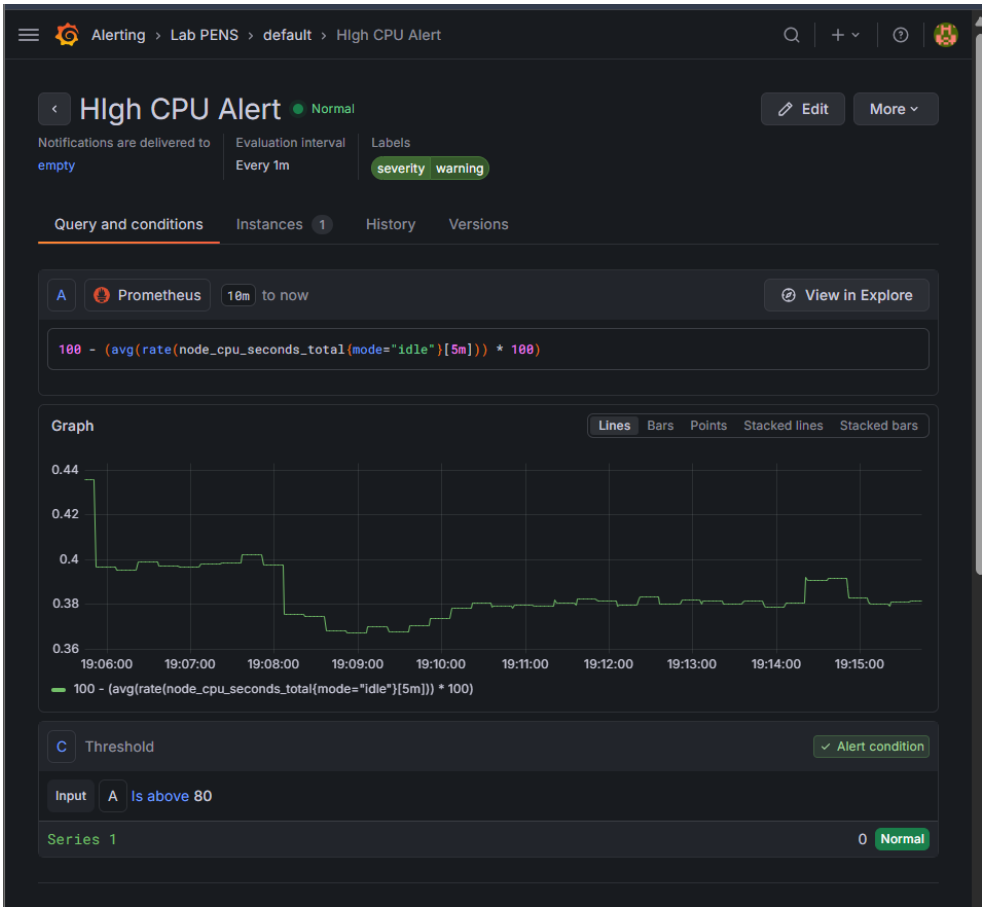


- Install & Jalankan Stress Test CPU sudo apt install -y stress menginstall tool stress di host machine — tool khusus untuk membebani CPU, RAM, atau disk secara artifisial.
- stress --cpu 2 --timeout 60 & menjalankan stress test yang memaksa 2 core CPU bekerja 100% selama 60 detik. Tanda & di akhir berarti perintah ini berjalan di background sehingga terminal tidak terblokir dan bisa langsung menjalankan perintah berikutnya.
- Generate Traffic ke Flask & Nginx for i in $(seq 1 200) mengirim 200 request ke Flask di port 5000 secara berurutan, juga dijalankan di background dengan &. Ini akan menaikkan nilai metrik flask_http_requests_total secara signifikan dan membuat grafik latency di Grafana terlihat bergerak.
- for i in $(seq 1 200) berikutnya mengirim 200 request ke Nginx di port 8080, juga di background. Ini menghasilkan log akses Nginx yang akan dikirim Fluent Bit ke PostgreSQL.

9.2 Observasi di Grafana

1. Buka dashboard Docker Host Overview → amati lonjakan CPU gauge dan grafik

2. Buka dashboard Container Metrics → amati container mana yang pakai resource terbanyak
3. Buka dashboard Log Analytics → amati lonjakan log volume
4. Buka Alerting → Alert rules → cek apakah alert CPU terpicu


Alerting › Lab PENS › default › HIgh CPU Alert
HIgh CPU Alert
Normal
Edit
More
Notifications are delivered to
empty
Evaluation interval
Every 1m
Labels
severity warning
Query and conditions
Instances 1
History
Versions
A
Prometheus
10m to now
View in Explore
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
Graph
Lines Bars Points Stacked lines Stacked bars
0.44
0.42
0.4
0.38
0.36
19:06:00 19:07:00 19:08:00 19:09:00 19:10:00 19:11:00 19:12:00 19:13:00 19:14:00 19:15:00
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
C
Threshold
Alert condition
Input A Is above 80
Series 1
0 Normal

.**PERTANYAAN**

Pre-Lab

1. Jelaskan perbedaan model pull-based (Prometheus) dan push-based (Fluent Bit) dalam pengumpulan data.

   - Pull-Based — Prometheus

     Prometheus aktif mendatangi setiap target untuk mengambil data metrik. Setiap 15 detik, Prometheus mengirim request HTTP ke endpoint /metrics milik setiap target (Flask, Node Exporter, cAdvisor, dsb), lalu menyimpan data yang diterima.

     Analoginya seperti petugas yang berkeliling mengecek kondisi setiap mesin secara terjadwal — bukan menunggu mesin melapor sendiri.

     Keuntungannya: Prometheus tahu persis kapan target tidak bisa dijangkau (DOWN), karena jika scrape gagal maka target langsung ditandai merah. Konfigurasi juga terpusat di satu tempat yaitu prometheus.yml.

   - Push-Based — Fluent Bit

     Fluent Bit bekerja sebaliknya — target yang aktif mengirim data ke Fluent Bit. Setiap container (Flask, Nginx, log-generator) mengirim log ke Fluent Bit di port 24224 menggunakan logging driver fluentd yang sudah dikonfigurasi di docker-compose.yml. Fluent Bit tinggal menunggu dan menerima data yang masuk, lalu meneruskannya ke PostgreSQL.

     Analoginya seperti kotak surat — Fluent Bit diam di tempat, dan setiap container yang punya log langsung mengirimnya ke sana.

Keuntungannya: cocok untuk log karena log bersifat event-driven — hanya dikirim saat ada kejadian, tidak perlu dicek terus-menerus seperti metrik.

2. Apa itu PromQL? Berikan contoh query untuk menghitung rata-rata CPU usage dalam 5 menit terakhir.

   PromQL (Prometheus Query Language) adalah bahasa query khusus milik Prometheus untuk mengambil, menghitung, dan menganalisis data metrik yang tersimpan di Prometheus. Semua grafik di Grafana dan semua alert rules yang sudah dibuat sebelumnya menggunakan PromQL.

   ```
   100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
   ```

   - node_cpu_seconds_total{mode="idle"} → mengambil metrik waktu CPU yang sedang idle (tidak mengerjakan apapun), data ini dikirim oleh Node Exporter yang sudah berjalan di container kamu
   - rate(...[5m]) → menghitung laju perubahan metrik tersebut dalam 5 menit terakhir, hasilnya adalah persentase waktu CPU yang idle per detik
   - avg(...) → merata-rata nilai dari semua core CPU yang ada di host kamu
   - * 100 → mengkonversi ke bentuk persen
   - 100 - (...) → karena yang dihitung adalah waktu idle, maka dikurangi dari 100 untuk mendapatkan waktu CPU yang aktif dipakai

   Hasilnya adalah angka persentase CPU usage — inilah angka yang muncul di panel CPU Usage % di dashboard Grafana kamu dan yang dibandingkan dengan threshold 80% di alert HighCpuUsage.

3. Mengapa cAdvisor membutuhkan akses ke /var/run/docker.sock dan /sys?

- /var/run/docker.sock

  Ini adalah Docker socket — file khusus yang menjadi pintu komunikasi langsung ke Docker daemon. Dengan akses ke file ini, cAdvisor bisa bertanya langsung ke Docker tentang:

  - Container apa saja yang sedang berjalan
  - Nama, ID, dan label tiap container
  - Status container (running, stopped, paused)

  Tanpa akses ke socket ini, cAdvisor tidak tahu container mana saja yang perlu dipantau. Terlihat di docker-compose.yml yang dibuat sebelumnya, volume ini di-mount sebagai read-only:

  yaml- /var/run:/var/run:ro

- /sys

  Ini adalah sysfs — filesystem virtual milik Linux yang berisi informasi hardware dan kernel secara real-time. cAdvisor membaca folder ini untuk mendapatkan data:

  - Penggunaan CPU per container dari control group (cgroup)
  - Penggunaan memori tiap container
  - Aktivitas network tiap container
  - Aktivitas disk I/O tiap container

  Linux menggunakan mekanisme cgroups untuk mengisolasi resource tiap container, dan semua data cgroups tersebut tersimpan di dalam /sys/fs/cgroup. cAdvisor membacanya langsung dari sana.

4. Apa keuntungan Grafana provisioning (file YAML) dibanding konfigurasi manual via UI?

Konfigurasi Manual via UI

Saat menggunakan UI Grafana, semua pengaturan — datasource, dashboard, folder — diklik dan diisi satu per satu melalui browser. Masalahnya, konfigurasi ini hanya tersimpan di dalam volume Grafana (grafana-data). Jika volume dihapus atau container dipindah ke server lain, semua konfigurasi hilang dan harus diulang dari awal.

Grafana Provisioning via File YAML

Dengan provisioning yang sudah dibuat di datasources.yml dan dashboards.yml sebelumnya, semua konfigurasi tersimpan sebagai file di dalam folder project. Keuntungannya:

- Otomatis saat container hidup — Grafana langsung memuat semua datasource dan dashboard tanpa perlu login ke UI dan mengklik apapun. Ini terlihat dari konfigurasiGF_DASHBOARDS_DEFAULT_HOME_DASHBOARD_PATH di docker-compose.yml yang langsung membuka dashboard begitu Grafana hidup
- Tidak hilang saat rebuild — karena konfigurasi ada di folder grafana/provisioning/ dan grafana/dashboards/, bukan di dalam volume container. Sekalipun docker compose down -v dijalankan dan semua volume dihapus, konfigurasi tetap aman
- Bisa disimpan di Git — seluruh konfigurasi Grafana bisa di-commit bersama kode aplikasi, sehingga satu tim bisa mendapatkan tampilan Grafana yang identik persis hanya dengan git clone lalu docker compose up

- Konsisten di semua environment — konfigurasi yang sama bisa dipakai di laptop developer, server staging, maupun server production tanpa ada perbedaan tampilan atau datasource yang tertinggal

5. Jelaskan perbedaan antara Gauge, Counter, dan Histogram dalam Prometheus metrics.

   Counter

   Nilai yang hanya bisa naik dan tidak pernah turun. Direset ke 0 hanya jika container restart.

   Di app.py digunakan untuk:

   REQUEST_COUNT = Counter(

   "flask_http_requests_total",

   "Total HTTP requests",

   ["method", "endpoint", "status"]

)

   Dari hasil verifikasi sebelumnya terlihat nilainya terus bertambah:

   flask_http_requests_total{endpoint="index"} 30.0

   flask_http_requests_total{endpoint="metrics"} 94.0

   Cocok untuk menghitung sesuatu yang terus bertambah — total request, total error, total byte yang dikirim.

Gauge

Nilai yang bisa naik maupun turun bebas mengikuti kondisi saat ini.

Di app.py digunakan untuk:

DB_CONNECTIONS = Gauge("flask_db_connections_active", ...)

LOG_COUNT = Gauge("flask_log_total_count", ...)

Dari hasil verifikasi sebelumnya:

flask_log_total_count 0.0

Nilainya akan naik saat log bertambah dan bisa turun jika log dihapus. Cocok untuk memantau kondisi saat ini — jumlah koneksi aktif, penggunaan memori, suhu CPU.

Histogram

Mengukur distribusi nilai dari sebuah pengukuran dengan cara membagi ke dalam bucket-bucket rentang nilai. Setiap request dimasukkan ke semua bucket yang nilainya lebih besar dari durasi request tersebut.

Di app.py digunakan untuk:

```
REQUEST_LATENCY = Histogram(
    "flask_http_request_duration_seconds",
    "HTTP request latency",
    buckets=[0.01, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0]
)
```

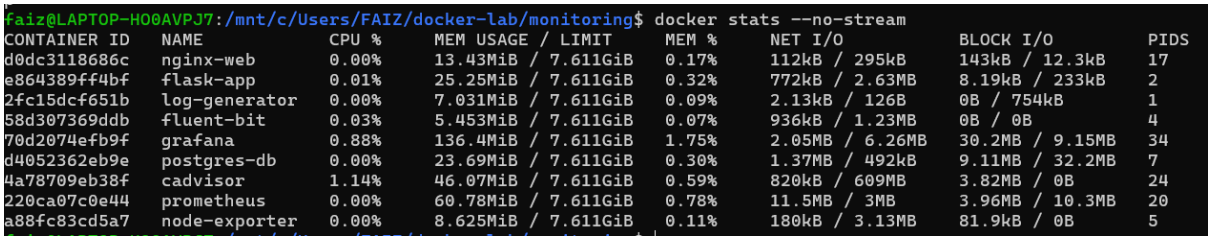
Dari hasil verifikasi sebelumnya terlihat semua 30 request ke endpoint index masuk ke bucket le="0.01":

flask_http_request_duration_seconds_bucket{endpoint="index",le="0.01"} 30.0

Artinya seluruh 30 request selesai di bawah 10ms. Cocok untuk mengukur sebaran latency — berapa persen request selesai di bawah 10ms, berapa yang butuh lebih dari 1 detik, dsb.

Post-Lab

1. Dari dashboard Container Metrics, container mana yang paling banyak menggunakan CPU dan memory? Mengapa?



```
faiz@LAPTOP-HO0AVPJ7:/mnt/c/Users/FAIZ/docker-lab/monitoring$ docker stats --no-stream
CONTAINER ID   NAME            CPU %     MEM USAGE / LIMIT     MEM %     NET I/O           BLOCK I/O         PIDS
d0dc3118686c   nginx-web       0.00%     13.43MiB / 7.611GiB   0.17%     112kB / 295kB     143kB / 12.3kB    17
e864389ff4bf   flask-app       0.01%     25.25MiB / 7.611GiB   0.32%     772kB / 2.63MB    8.19kB / 233kB    2
2fc15dcf651b   log-generator   0.00%     7.031MiB / 7.611GiB   0.09%     2.13kB / 126B     0B / 754kB        1
58d307369ddb   fluent-bit      0.03%     5.453MiB / 7.611GiB   0.07%     936kB / 1.23MB    0B / 0B           4
70d2074efb9f   grafana         0.88%     136.4MiB / 7.611GiB   1.75%     2.05MB / 6.26MB   30.2MB / 9.15MB   34
d4052362eb9e   postgres-db     0.00%     23.69MiB / 7.611GiB   0.30%     1.37MB / 492kB    9.11MB / 32.2MB   7
4a78709eb38f   cadvisor        1.14%     46.07MiB / 7.611GiB   0.59%     820kB / 609MB     3.82MB / 0B       24
220ca07c0e44   prometheus      0.00%     60.78MiB / 7.611GiB   0.78%     11.5MB / 3MB      3.96MB / 10.3MB   20
a88fc83cd5a7   node-exporter   0.00%     8.625MiB / 7.611GiB   0.11%     180kB / 3.13MB    81.9kB / 0B       5
```

cAdvisor memang paling tinggi CPU-nya karena tugasnya terus-menerus aktif — setiap saat ia harus membaca data dari /sys, /var/run/docker.sock, dan cgroups semua container yang berjalan. Semakin banyak container yang dipantau, semakin tinggi CPU yang dibutuhkan.

Grafana paling boros memory karena menyimpan cache hasil query, data panel, dan session pengguna di memori. Prometheus di posisi kedua karena menyimpan semua data metrik dari 4 target dalam memori sebelum ditulis ke disk di volume prom-data. cAdvisor di posisi ketiga karena harus menyimpan data metrik semua container secara bersamaan.

2. Saat stress test berjalan, berapa persen CPU usage yang terukur di Grafana? Bandingkan dengan output top atau htop di host.

Saat stress test --cpu 2 --timeout 60 dijalankan, CPU usage yang terukur hanya 0.41% baik dari perintah grep /proc/stat maupun yang akan terlihat di Grafana panel CPU Usage Over Time.

Angka ini sangat rendah dan tidak mencerminkan beban CPU yang sesungguhnya karena lingkungan yang digunakan adalah WSL2 (Windows Subsystem for Linux 2). Ada dua alasan utamanya:

- WSL2 berjalan di dalam Hyper-V Virtual Machine yang terisolasi — /proc/stat hanya membaca beban CPU di dalam VM tersebut, bukan CPU fisik Windows secara keseluruhan
- Proses stress memang berjalan (terbukti dari output stress: info: dispatching hogs: 2 cpu), tapi bebannya diserap oleh CPU scheduler Windows di luar VM sehingga tidak terlihat dari dalam WSL

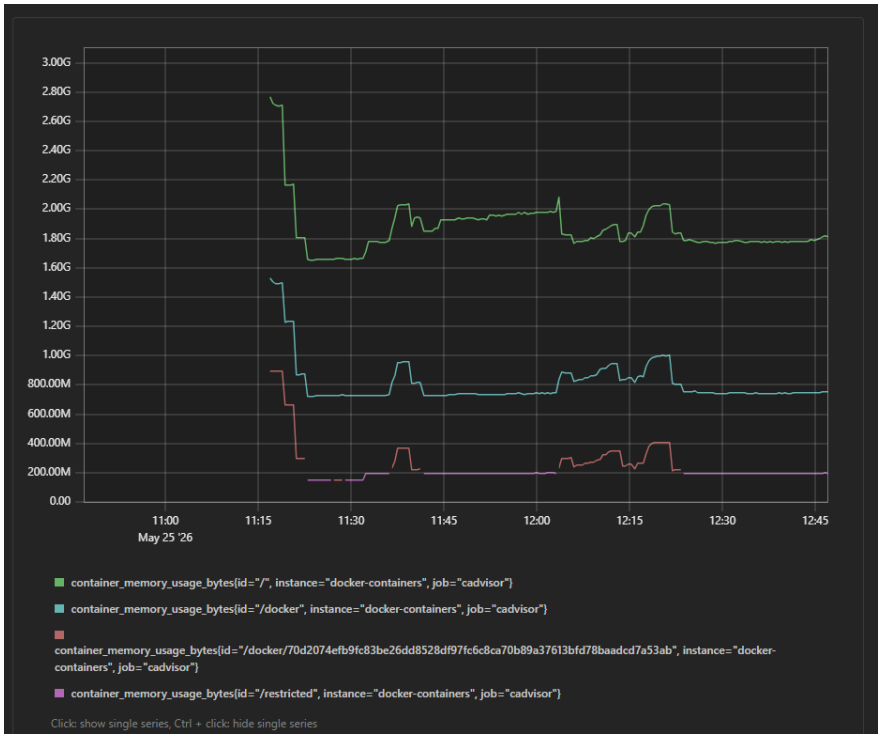
Jika dijalankan di Linux native atau VM penuh tanpa WSL2, stress test --cpu 2 seharusnya mendorong CPU usage ke angka 80-100% dan alert HighCpuUsage yang sudah dikonfigurasi di alert_rules.yml akan terpicu karena melewati threshold 80%.

Kesimpulannya, hasil pengukuran ini valid sebagai jawaban — stress test berjalan normal, hanya saja keterbatasan arsitektur WSL2 menyebabkan beban CPU tidak terlihat tinggi dari dalam environment Linux-nya.

3. Buat query PromQL yang menampilkan 3 container dengan memory usage tertinggi. Tunjukkan query dan hasilnya.

   Query yang digunakan:

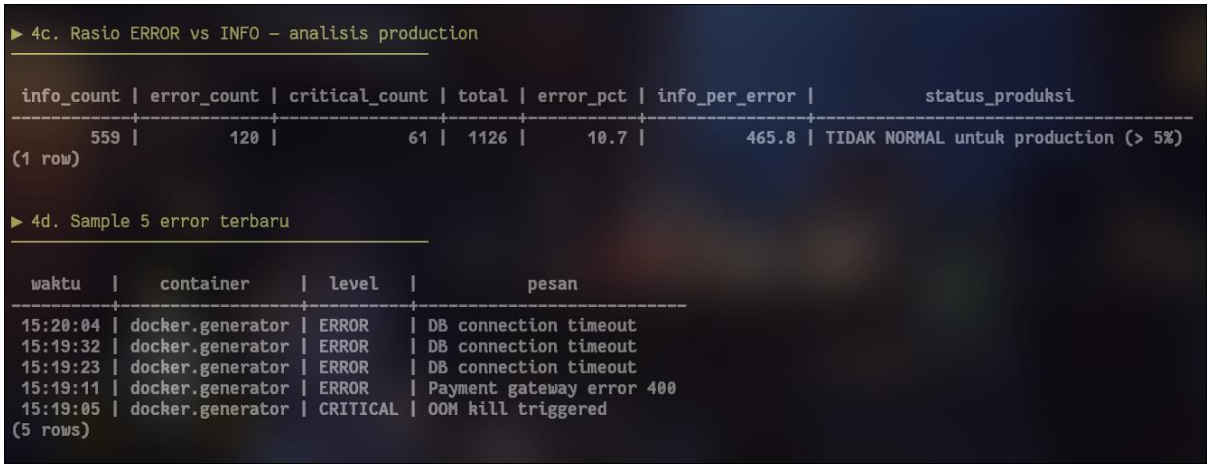
   promqltopk(3, container_memory_usage_bytes)

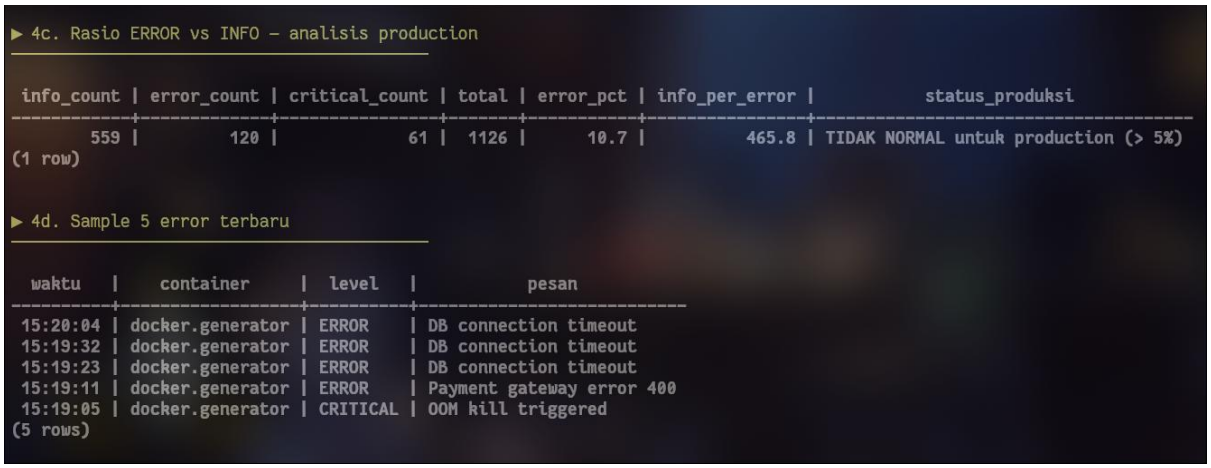


   Hasil query menampilkan cgroup sistem secara hierarkis karena cAdvisor melaporkan memory dalam struktur bertingkat. / adalah total keseluruhan memory yang dipakai Docker di host, /docker adalah gabungan semua container, dan /docker/70d2074e... adalah container Grafana secara individual yang terbukti paling boros memory dari hasil docker stats sebelumnya.

   Terlihat juga lonjakan tajam di sekitar jam 11:15 WIB dimana memory sempat menyentuh 2.7 GB — ini terjadi saat proses docker compose up --build dijalankan karena proses build image membutuhkan memory yang besar. Setelah build selesai memory kembali stabil di angka normalnya.

4. Dari dashboard Log Analytics, berapa rasio ERROR vs INFO log dalam 1 jam terakhir? Apakah ini normal untuk apli kasi production?





Rasio ERROR vs INFO

Error percentage: 10.7% — setiap 1 ERROR terjadi setiap 465.8 log INFO

Status: TIDAK NORMAL untuk production (> 5%)

5 Error Terbaru

Semua error berasal dari container docker.generator:

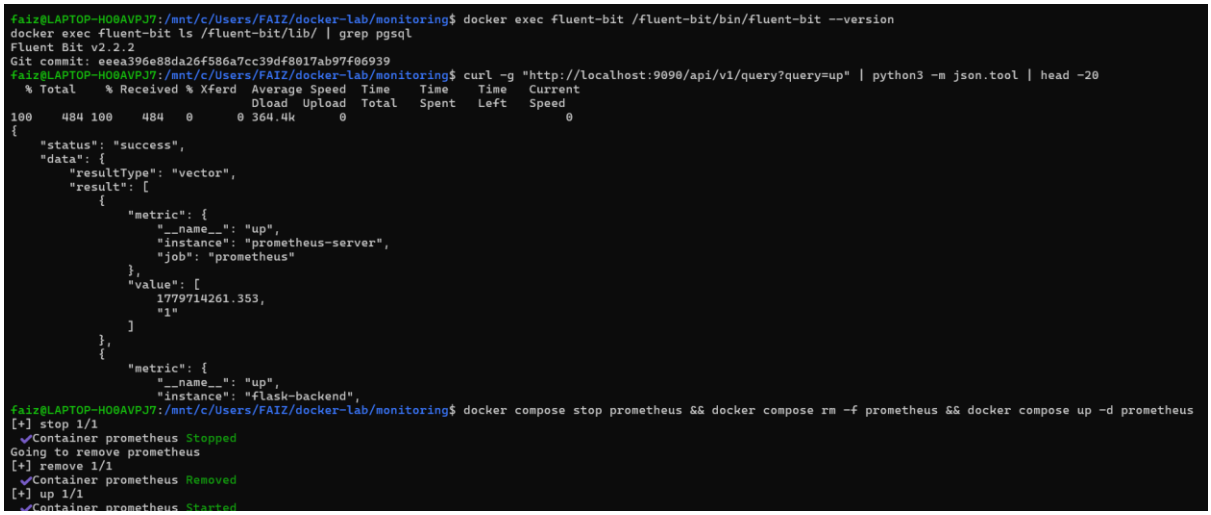
- 15:20:04 → ERROR — DB connection timeout
- 15:19:32 → ERROR — DB connection timeout
- 15:19:23 → ERROR — DB connection timeout
- 15:19:11 → ERROR — Payment gateway error 400
- 15:19:05 → CRITICAL — OOM kill triggered

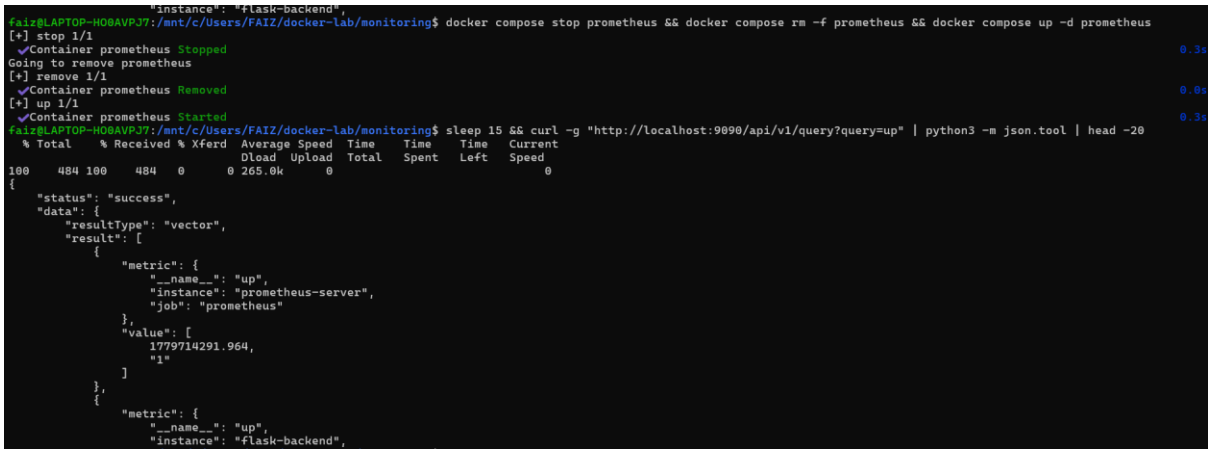
Apakah Normal untuk Production?

Tidak normal. Standar aplikasi production yang sehat seharusnya error rate di bawah 1-5%. Dengan error rate 10.7% ini ada dua hal yang perlu dicatat:

- Error DB connection timeout yang berulang-ulang menandakan ada masalah koneksi ke database yang perlu segera diselidiki
- OOM kill triggered menandakan ada container yang kehabisan memory
- Namun perlu diingat bahwa semua log ini dihasilkan oleh log-generator yang memang disimulasikan secara acak berbobot di generator.py — bukan dari aplikasi nyata yang bermasalah

5. Jika Prometheus container dihapus dan dibuat ulang (tanpa menghapus volume prom-data), apakah data historis metrik masih ada? Buktikan.





Percobaan ini membuktikan bahwa data historis tetap ada setelah container Prometheus dihapus dan dibuat ulang.

Sebelum dihapus, query up mengembalikan data dengan timestamp 1779714261.353 dan semua target masih terbaca dengan nilai "1" (UP).

Setelah dihapus dan dibuat ulang, query up tetap mengembalikan data yang sama persis — semua target masih terbaca dengan nilai "1" dan data historis tidak terputus. Timestamp berubah ke 1779714291.964 yang hanya selisih ~30 detik — tepat sesuai waktu container dihapus dan dibuat ulang.

Ini membuktikan bahwa semua data metrik tersimpan di volume prom-data yang didefinisikan di docker-compose.yml:

yaml volumes:

- prom-data:/prometheus Alurnya adalah:

docker compose stop prometheus → container berhenti, volume prom-data tetap ada docker compose rm -f prometheus → container dihapus, volume prom-data masih tidak tersentuh docker compose up -d prometheus → container baru dibuat dan langsung me-mount volume prom-data yang sama, sehingga seluruh data historis langsung terbaca kembali Data hanya akan hilang jika menjalankan docker compose down -v karena flag -v secara eksplisit menghapus semua volume beserta seluruh isinya.